

第三章

大模型预训练与微调方法

王成龙 肖桐

xiaotong@mail.neu.edu.cn

东北大学 计算机科学与工程学院 人工智能系 自然语言处理实验室

PPT制作：兰庭旭 林丁洋 王俊鑫 吴钰璋



東北大學
Northeastern University





大模型预训练与微调方法

3.1 大模型预训练技术

3.2 提示学习方法

3.3 大模型微调技术

3.4 大模型偏好对齐技术

3.5 大语言模型推理增强方法

1.1 大模型预训练技术

- 在过去的几年里，大语言模型从学术界的探索走向了产业界的核心。本节作为全章的技术起点，将回答一个根本性问题：

我们是如何让机器"学会"理解语言的？

- 我们将沿着"为什么需要预训练→预训练在做什么→如何实现→如何扩展"这条线索，逐步拆解大模型的技术基石。

- 在很长一段时间里，自然语言处理的做法是：**每遇到一个新任务，就从头训练一个新模型**。做情感分析就训练一个分类器，做翻译就训练一个翻译模型，做问答就训练一个阅读理解模型——每个模型都是“专用工具”。
- 但我们真正想要的是一个**通用的语言智能体**：一个模型能够应对各种各样的语言任务，而不是一百个模型做一百件事。就像一个人可以同时学会写诗、翻译和编程，而不需要在学每一件事时都从零开始。

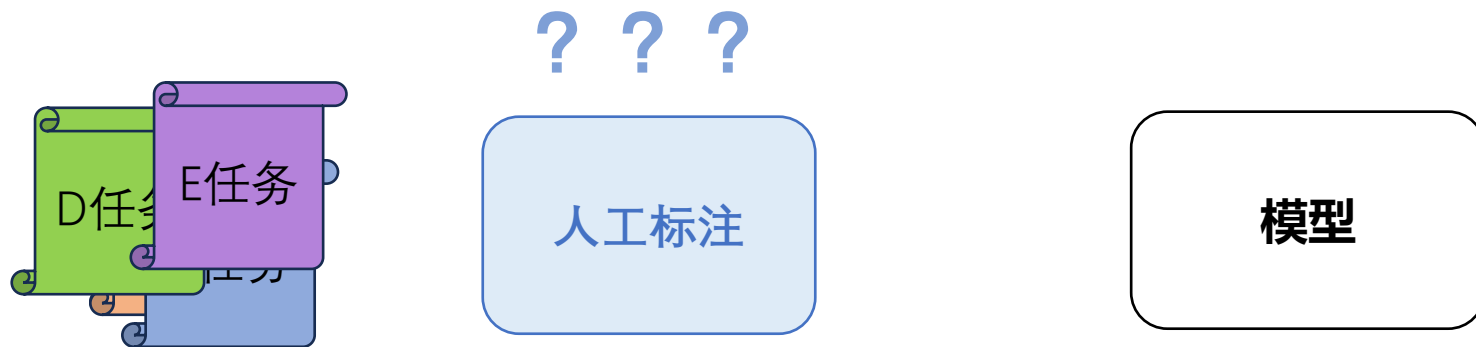


一个根本性的难题

- 如果我们要训练一个"什么都会"的通用模型，而不是针对每个任务单独训练

那这个模型应该怎么学？ 用什么样的数据来学？

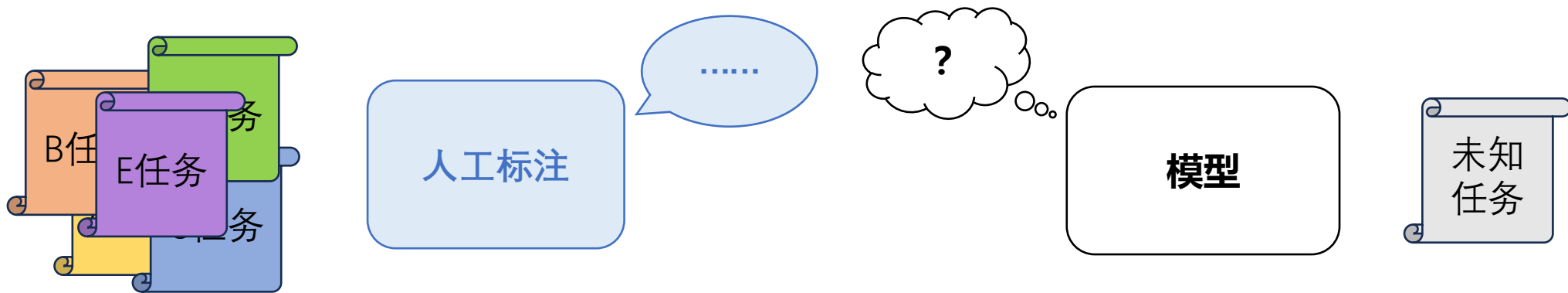
- 总不能为每个任务都准备几十万条标注数据，那样做既不现实，也违背了"通用"的初衷。



- 传统监督学习的路径，在这里走不通了。

- 传统方法依赖**任务特定的标注数据**——情感分析需要上万条打了情感标签的评论，翻译需要百万级对齐的平行语料。这条路有三个根本性局限：

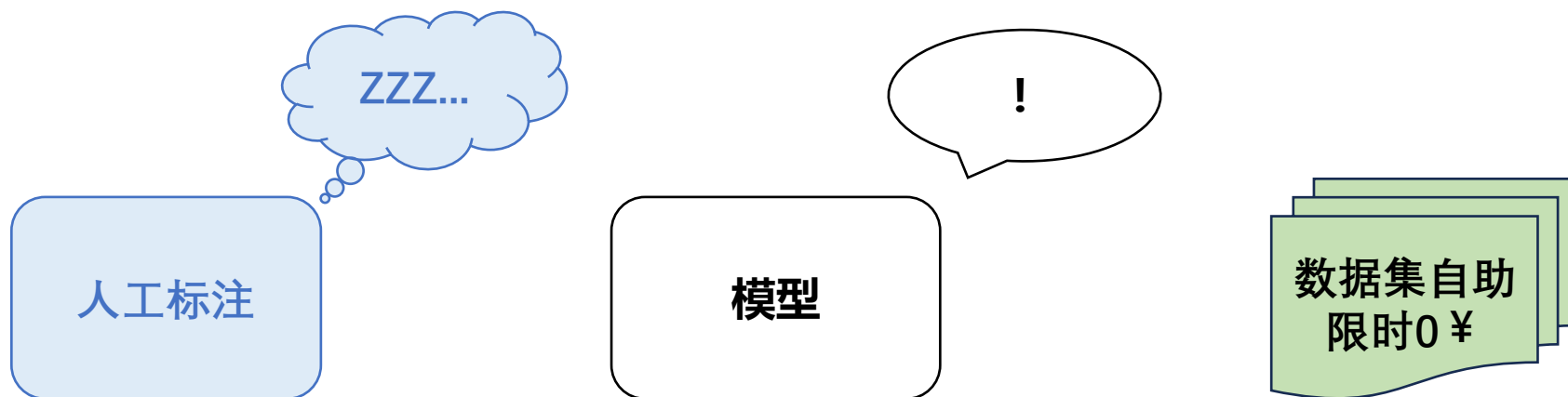
1. 标注成本极高 2. 覆盖面有限 3. 泛化能力弱



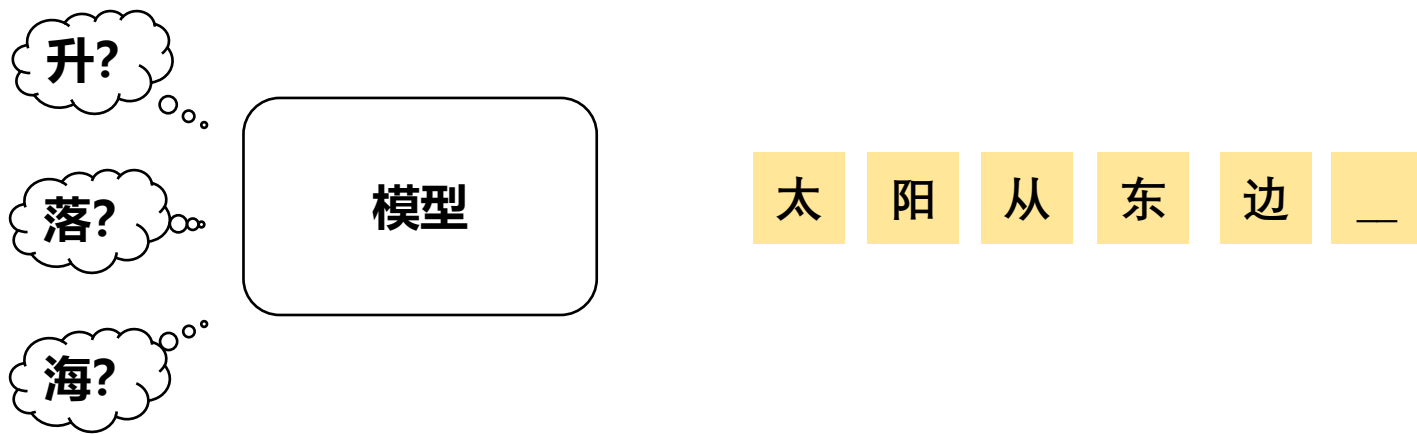
- 我们需要一条新路：**先让模型在海量无标注文本上“读书”，再让它去完成具体任务**。这就是预训练的核心动机。

有没有不需要标注的学习任务？

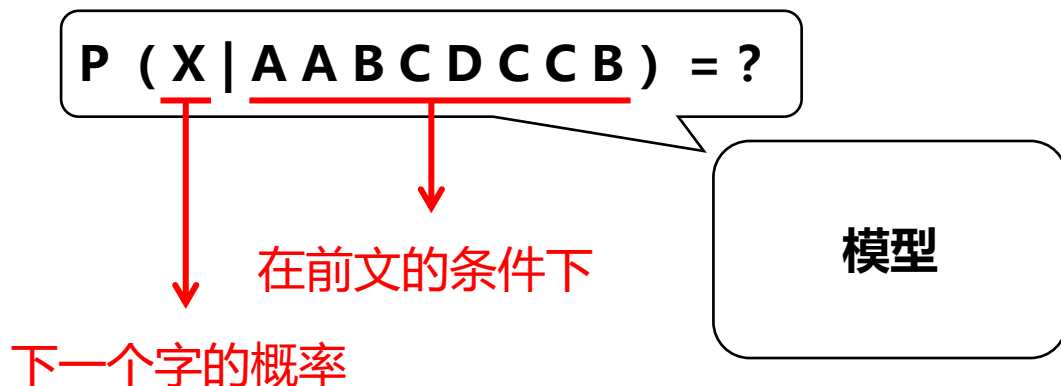
- 既然标注数据是瓶颈，那有没有一种语言任务，**天然就有无穷无尽的训练数据，而且完全不需要人工标注？**
- 如果有这样的任务，我们就能让模型先在它上面“自学成才”，掌握语言的基本规律，然后再去解决具体问题。



- 答案是**语言建模**——它的本质就是“根据上文猜下一个词”。任何一段文本，天然就提供了“上文-下文”这样的训练信号，**完全不需要人工标注**。
- 但这个简单的任务背后藏着一个深刻的洞见：要准确猜出下一个词，模型必须学会语法、语义、常识、甚至一定的推理能力。比如，看到“太阳从东边升起”，模型得知道东边是哪个方向——它不需要被显式告知这些知识，而是从海量文本中自己总结出来。
- 如果让一个模型在海量文本上把“猜词”练到极致，它会不会顺便学会所有语言知识？这个想法驱动了现代大语言模型的全部发展。



- “猜词”可以用概率精确地表达：语言模型的目标是计算一个词序列出现的概率



A A B C D C C B _				
A	B	C	D	
10%	40%	20%	30%	

- “猜词”可以用概率精确地表达：语言模型的目标是计算一个词序列出现的概率

$P(X | A A B C D C C B) = ?$

模型

A A B C D C C B _

A

B

C

D

10%

40%

20%

30%

- 根据概率论中的链式法则，这个联合概率可以分解为每一步条件概率的乘积：

$$\Pr(x_0, \dots, x_m) = \prod_{i=0}^m \Pr(x_i | x_0, \dots, x_{i-1})$$

- “猜词”可以用概率精确地表达：语言模型的目标是计算一个词序列出现的概率

$$P(X | \underline{A A B C D C C B}) = ?$$

模型

$$P(B | \underline{A A B C D C C})$$

$$P(C | \underline{A A B C D C})$$

$$P(C | \underline{A A B C D C})$$

...

$$P(A | A)$$

A A B C D C C B _

A

10%

B

40%

C

20%

D

30%

- “猜词”可以用概率精确地表达：语言模型的目标是计算一个词序列出现的概率

$$P(X | A A B C D C C B) = ?$$

模型

A A B C D C C B _

A

B

C

D

10%

40%

20%

30%

- 通俗地讲，整句话有多“通顺”，等于“第一个词出现的概率”乘以“在第一个词之后第二个词出现的概率”乘以……依此类推。语言模型要做的，就是准确估计每一步的条件概率。

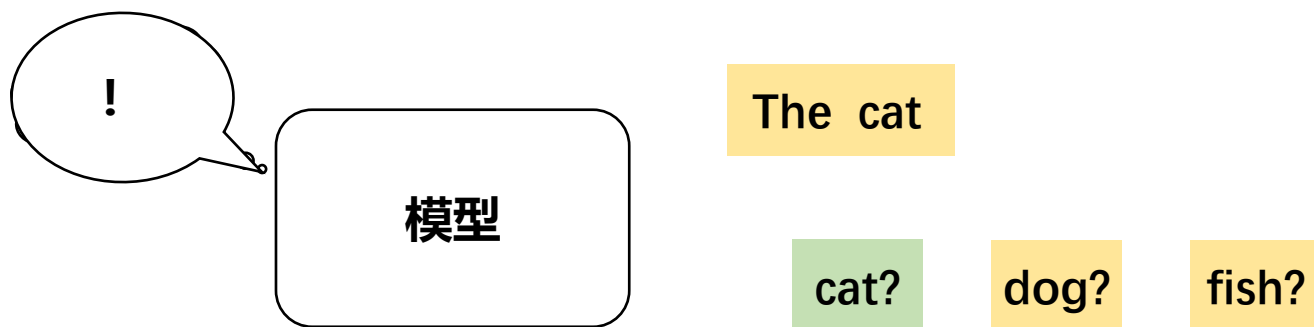
- 有了训练好的语言模型后，最常见的操作是**自回归生成**：每次根据已有上下文，选出概率最高的下一个词，拼到末尾，再以新的上下文预测再下一个词，如此反复。
 - 以生成"The cat sat on"为例：
 - 给定起始符 The，模型预测下一个最可能的词是 cat。
 - 给定 The cat，模型预测下一个最可能的词是 sat。
 - 给定 The cat sat，模型预测下一个最可能的词是 on。



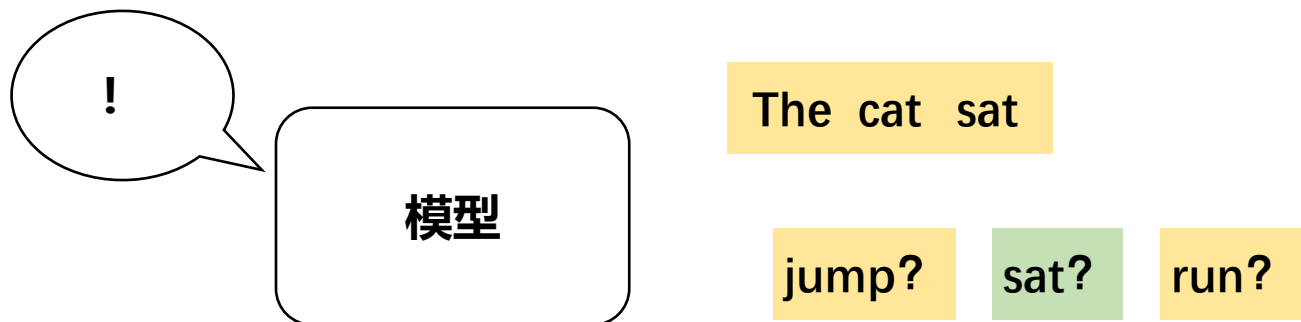
模型

The

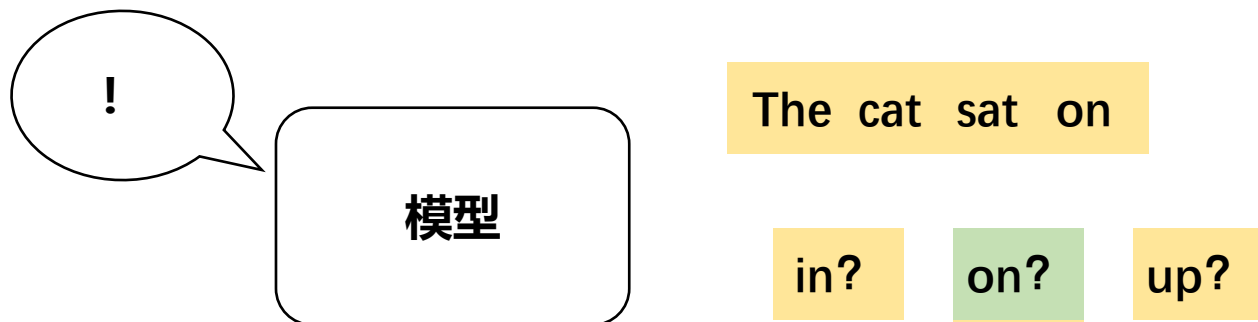
- 有了训练好的语言模型后，最常见的操作是**自回归生成**：每次根据已有上下文，选出概率最高的下一个词，拼到末尾，再以新的上下文预测再下一个词，如此反复。
 - 以生成"The cat sat on"为例：
 - 给定起始符 The，模型预测下一个最可能的词是 cat。
 - 给定 The cat，模型预测下一个最可能的词是 sat。
 - 给定 The cat sat，模型预测下一个最可能的词是 on。



- 有了训练好的语言模型后，最常见的操作是**自回归生成**：每次根据已有上下文，选出概率最高的下一个词，拼到末尾，再以新的上下文预测再下一个词，如此反复。
 - 以生成"The cat sat on"为例：
 - 给定起始符 The，模型预测下一个最可能的词是 cat。
 - 给定 The cat，模型预测下一个最可能的词是 sat。
 - 给定 The cat sat，模型预测下一个最可能的词是 on。



- 有了训练好的语言模型后，最常见的操作是**自回归生成**：每次根据已有上下文，选出概率最高的下一个词，拼到末尾，再以新的上下文预测再下一个词，如此反复。
 - 以生成"The cat sat on"为例：
 - 给定起始符 The，模型预测下一个最可能的词是 cat。
 - 给定 The cat，模型预测下一个最可能的词是 sat。
 - 给定 The cat sat，模型预测下一个最可能的词是 on。



- 有了训练好的语言模型后，最常见的操作是**自回归生成**：每次根据已有上下文，选出概率最高的下一个词，拼到末尾，再以新的上下文预测再下一个词，如此反复。
- 每一步的决策都对应着链式法则中的一个条件概率因子，而整句话的概率就是这些因子相乘。这个过程中模型"流利地"写出了一个句子，仿佛它真的"理解"了自己在写什么。

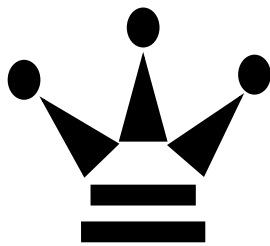
什么样的模型能做好这件事？

- 语言建模的目标明确了——但还有一个问题：**什么样的神经网络能够胜任这项任务？**
- 我们需要一个模型能够：
- 1. **处理变长输入**：句子有长有短，模型不能要求固定长度。
- 2. **捕捉长距离依赖**：句首的词可能要到句末才被解释清楚。
- 3. **并行计算**：训练时要能高效利用GPU算力。



什么样的模型能做好这件事？

- 语言建模的目标明确了——但还有一个问题：**什么样的神经网络能够胜任这项任务？**
- 这个问题在2017年之前一直没有被很好地解决——直到Transformer架构的出现。

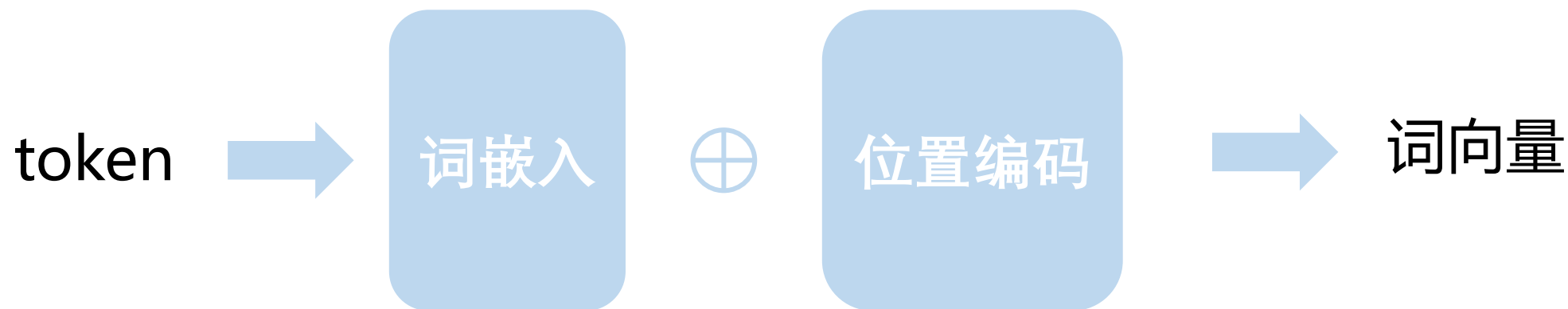


Transformer

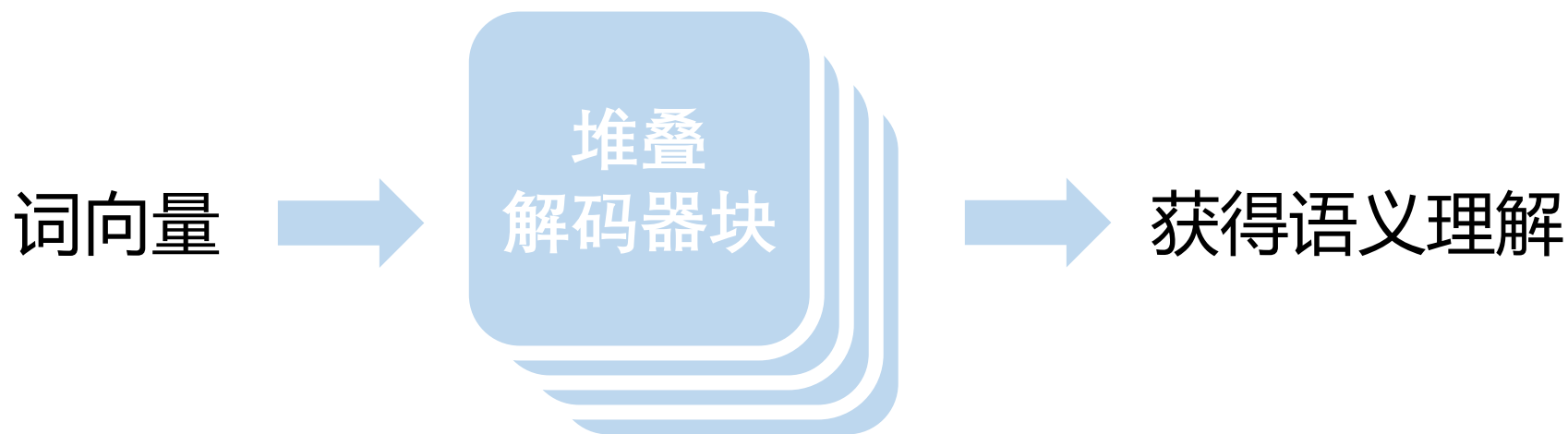
- 当前主流的大语言模型都采用了一种统一的骨架：

仅解码器 (Decoder-only) Transformer。

- 它的信息处理路径清晰分为三步：
- 1. **嵌入层**：将输入的词 (token) 转换为向量表示，同时加入位置信息。



- 当前主流的大语言模型都采用了一种统一的骨架：
仅解码器（Decoder-only）Transformer。
- 它的信息处理路径清晰分为三步：
- 2. **堆叠的Transformer块**：向量每经过一个块，就获得一层更深的上下文理解。

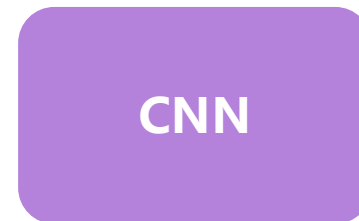
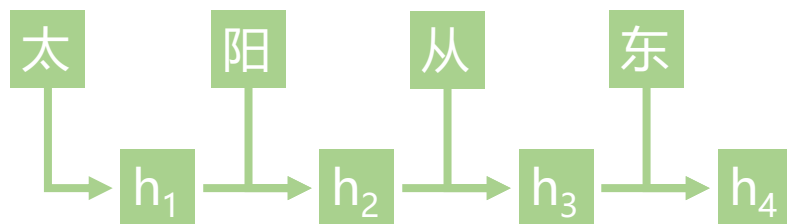


- 当前主流的大语言模型都采用了一种统一的骨架：
仅解码器 (Decoder-only) Transformer。
- 它的信息处理路径清晰分为三步：
- 3.**输出层**：最后一个块的输出通过Softmax映射为词汇表上的概率分布。



Transformer 最关键的设计是什么？

- Transformer 架构中有很多设计，但其中最核心、最革命性的是**自注意力机制**。它解决了语言理解中的一个根本问题：**模型在处理某个词时，如何知道哪些前文词汇与它相关？**
- 没有自注意力之前，RNN只能"串行"地传递信息，距离越远的词影响越小；CNN只能看到局部窗口。而自注意力让每个词可以**直接**关注到所有前文词汇。

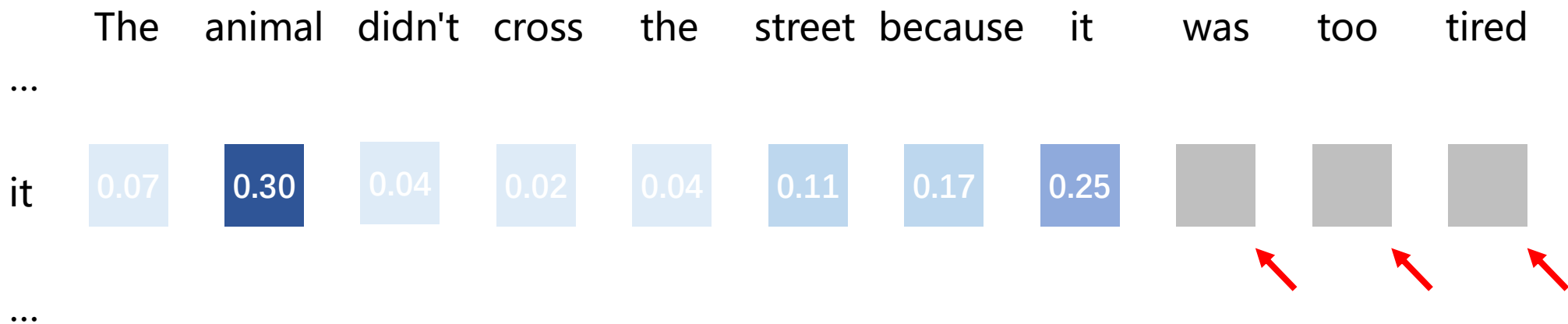


- 自注意力的工作方式可以这样理解：每个词都会发出一个"查询" (Query)，同时每个词也都有一个"键" (Key) 和一个"值" (Value)。查询会去匹配所有前文词的键，匹配度越高，这个词的值在聚合中的权重就越大。
 - 一个具体的例子："The animal didn't cross the street because it was too tired." 当模型处理"it"时，它会通过自注意力机制"看到"并高权重地关联到"animal"，从而判断出"it"指代的是动物而不是街道。

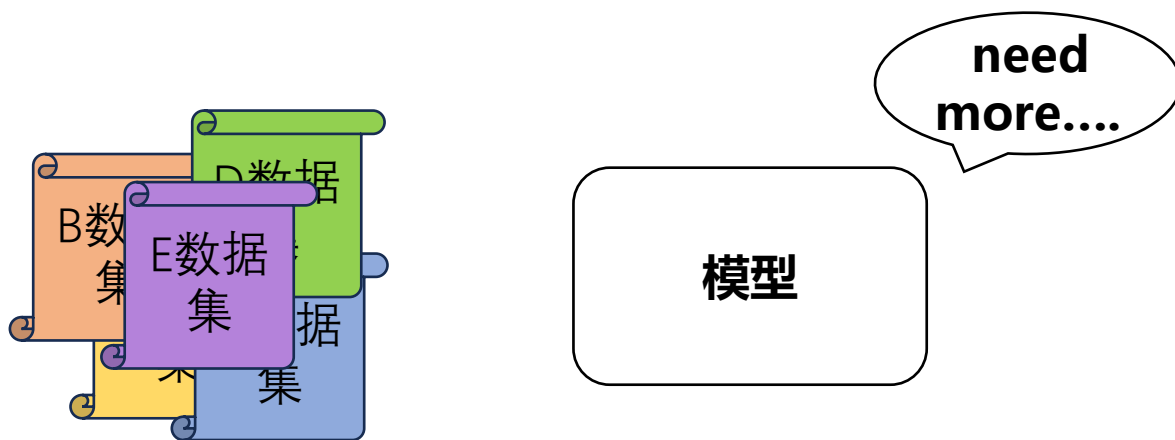


- 自注意力的工作方式可以这样理解：每个词都会发出一个"查询" (Query)，同时每个词也都有一个"键" (Key) 和一个"值" (Value)。查询会去匹配所有前文词的键，匹配度越高，这个词的值在聚合中的权重就越大。
- 这就是模型"理解"上下文的方式——它不是通过规则，而是通过数据驱动的动态加权。

- 自注意力中有一个"掩码" (Mask) 操作。它的作用是：**确保位置 i 在计算时，只能看到位置 0 到 $i-1$ 的信息，而不能"偷看"到后面的词。**
- 为什么要加这个限制？因为语言建模的任务是"根据上文预测下一个词"——如果模型在预测 x_i 时已经看到了 x_i 本身甚至后面的词，那"猜词"就变成了"抄答案"。
- 掩码保证了训练和推理时的一致性。这就是为什么这类模型被称为**自回归模型**。



- 模型架构有了，训练目标也明确了——但还有一个现实问题：**数万亿token的训练数据从哪来？**
- 现代大模型的预训练消耗的数据量远超传统模型数个量级，单靠人工标注或收集特定数据集根本不现实。必须从公开可用的海量文本中获取。

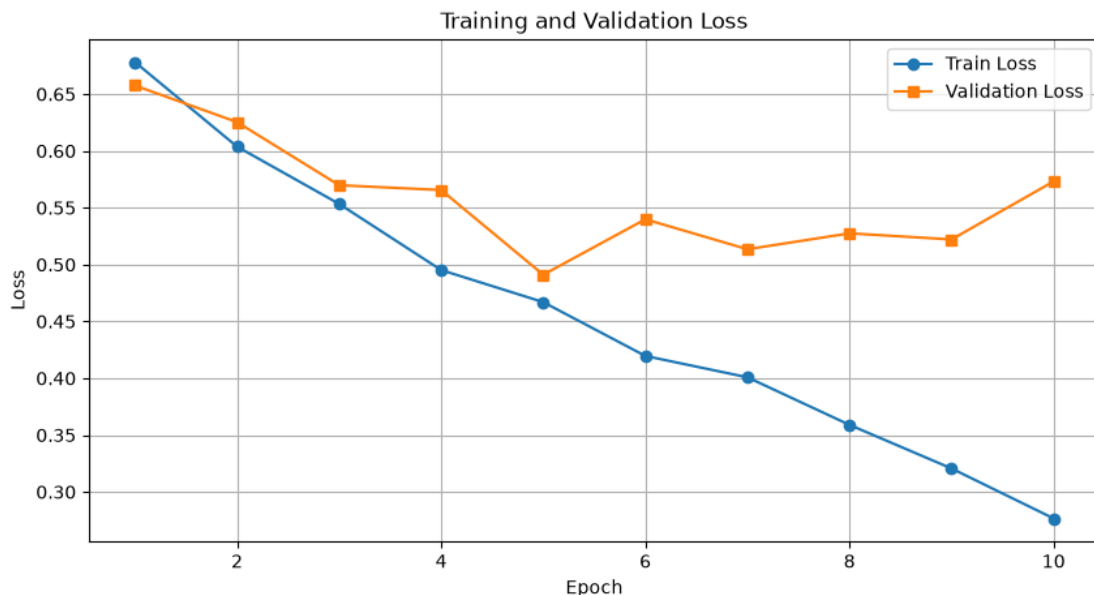


- 训练语料通常来自多个渠道，每个渠道贡献了不同的能力：
- 1. **网络爬取数据**：占比最大，覆盖各领域，但需严格过滤。
- 2. **书籍与学术论文**：提供连贯的长文本和深度知识。
- 3. **编程代码**：显著提升模型的逻辑推理能力。
- 4. **多语言语料**：使模型能够处理多种语言。
- 数据的多样性和质量之间存在微妙的平衡——过度过滤可能导致知识流失，过滤不足则会引入噪声。

部分 LLM 的预训练数据规模

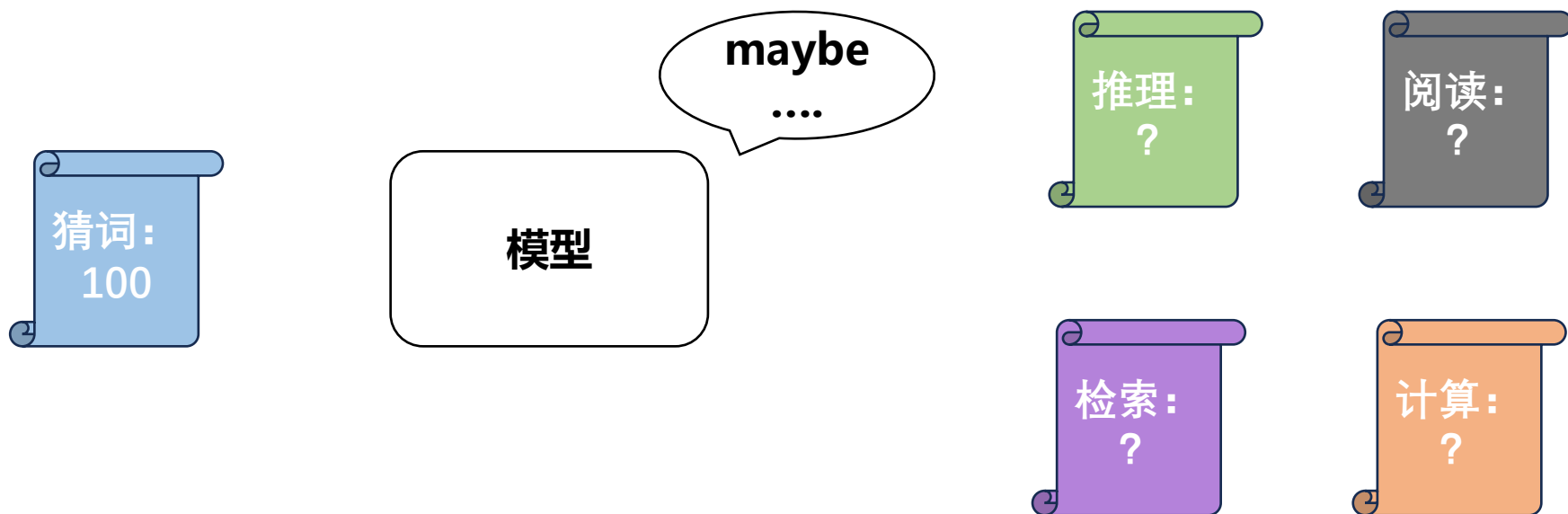
LLM	Year	# of Tokens
GPT-3-175B	2020	0.5T
LLaMA2-70B	2023	2.0T
LLaMA3-8B	2024	15T
DeepSeek-V3	2024	14.8T
Qwen3-235B	2025	36T

- 有了数据和架构，预训练的终极目标可以提炼为一句话：**最大化模型在所有训练序列上逐token预测的对数似然**。通俗地说，就是让模型在“猜词”这件事上越猜越准。
- 给定一个训练序列，模型计算每一步的条件概率，然后乘起来（取对数后变成加和）。训练就是调整参数，让这个乘积在整个训练集上尽可能大。
- 在工程实现中，这等价于**最小化交叉熵损失**——这个损失函数是整个预训练流程的“指挥棒”。



这真的有用吗？

- 以上所有努力——精心设计的架构、海量的训练数据、明确的最优化目标——最终目的是让模型学会“猜词”。但一个只会猜词的模型，真的能理解语言、解决问题吗？
- 换句话说：**预训练到底给模型带来了什么？** 这个问题直接关系到整个技术路线是否值得。



这真的有用吗？

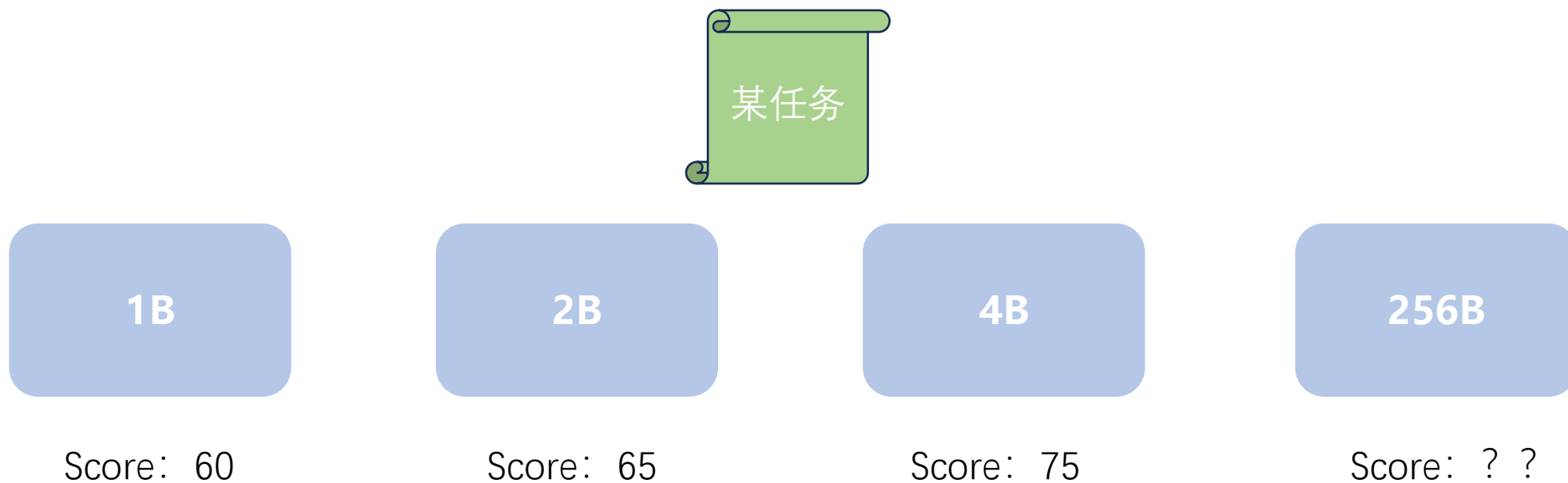
- 经过万亿级token的预训练后，模型发生了质的飞跃。它不再是“只会猜词”，而是通过猜词这个任务，**内化了语言的结构、世界的知识、甚至初步的推理能力。**
- “猜词”这个简单任务所蕴含的力量，无意间成就了一个通用的基础模型。



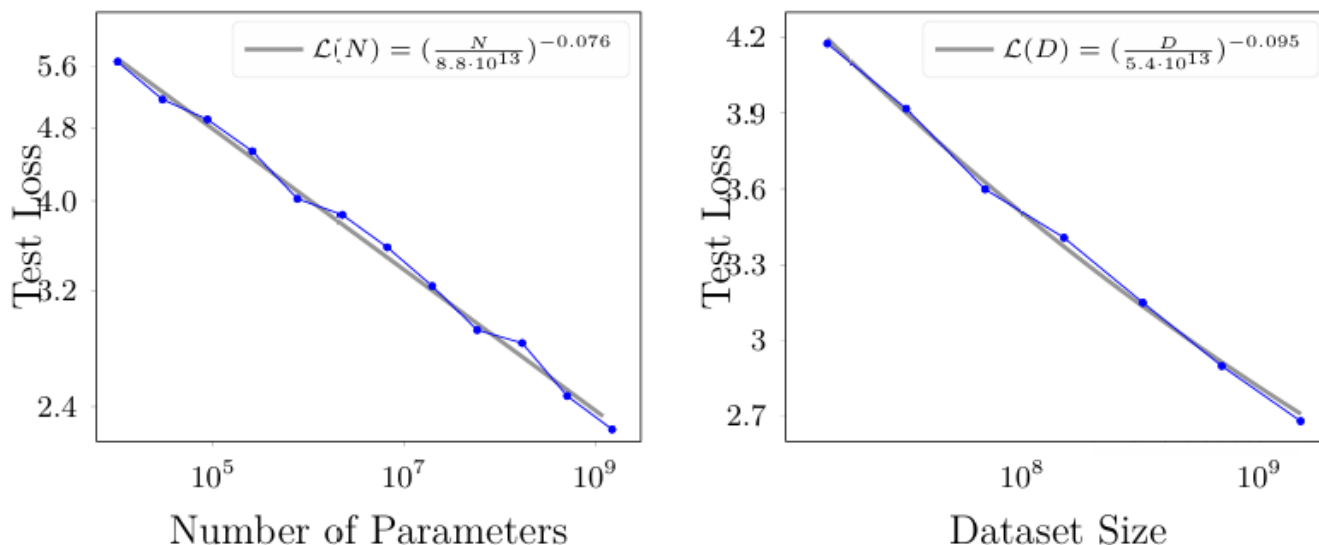
- 经过万亿级token的预训练后，模型发生了质的飞跃。它不再是“只会猜词”，而是通过猜词这个任务，**内化了语言的结构、世界的知识、甚至初步的推理能力。**
- 一个经过良好预训练的大语言模型能够：
 - 1. 处理海量不同类型的任务。
 - 2. 以极低的适配成本泛化到全新任务。
 - 3. 展现出超出训练数据表面模式的**涌现能力**。

模型还能更大吗？

- 如果预训练的效果这么好，那自然会问：**把模型做得更大、数据喂得更多，性能还能继续提升吗？** 还是说会遇到天花板？
- 这个问题直接决定了我们是否应该继续“堆规模”——以及投入的算力能换回多少收益。



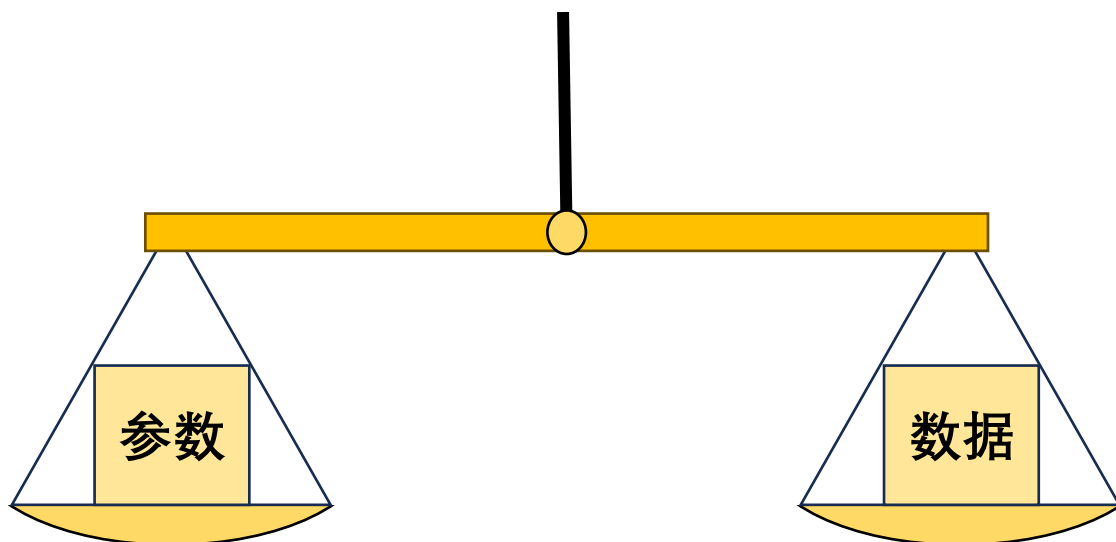
- 缩放定律揭示了一个经验规律：测试损失随着模型参数量 N 和训练数据量 D 的增加按**幂律**下降。也就是说，每投入一倍的算力，模型就会获得一个可预测的稳定收益——至少在当前阶段，还没有看到"天花板"。
- 这个规律给了研究者巨大的信心：只要算力允许，继续扩大规模就一定能继续获得性能提升。
- 但同时，它也意味着"性价比"是一个必须考虑的问题——投入和收益的边际关系如何？



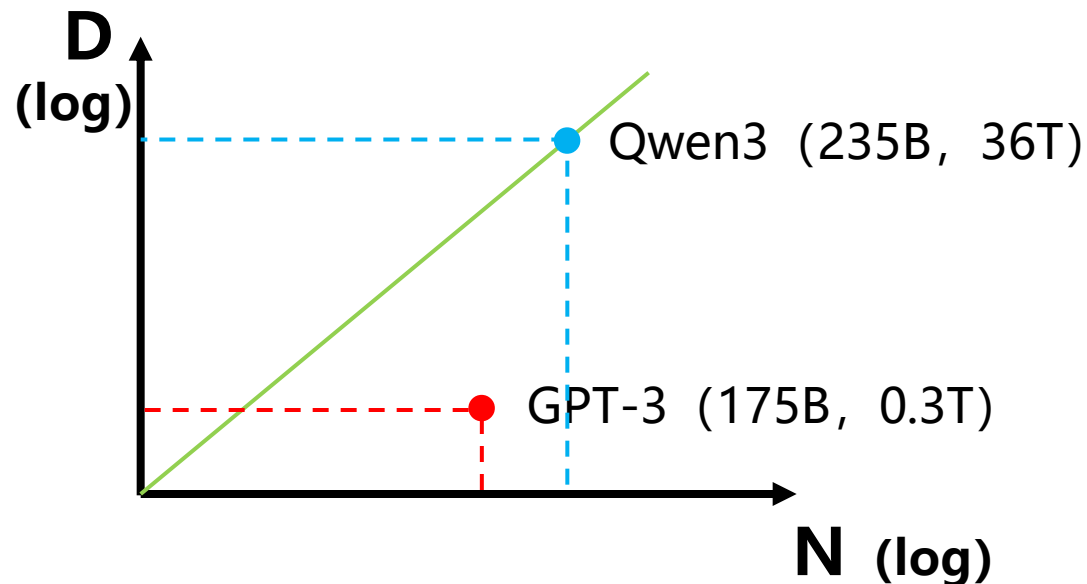
测试损失与模型参数量 N 及训练数据量 D 的关系

怎样分配资源才是最划算的？

- 给定一笔固定的计算预算，我们有两个可以调整的变量：**模型参数量 N** 和 **训练数据量 D** 。那么问题来了：钱应该更多地花在“把模型做大”上，还是“给模型喂更多数据”上？
- 这个问题的答案，直接影响了大模型研发的资源分配决策。

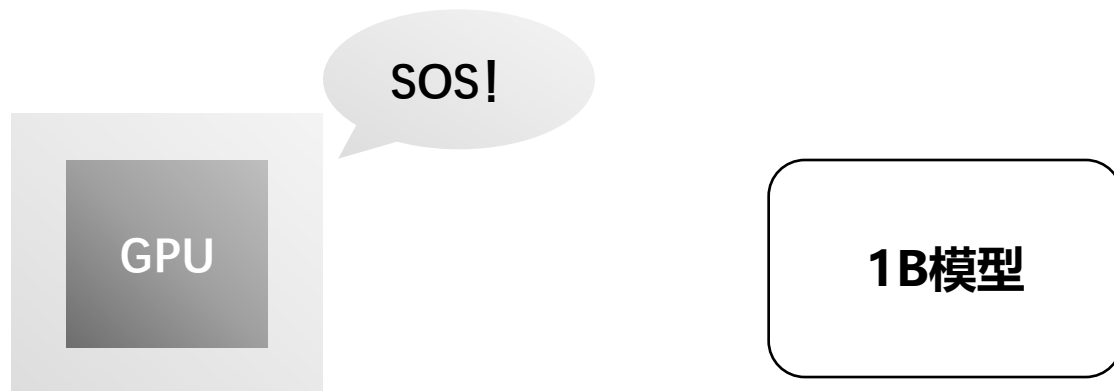


- Chinchilla定律给出了一个明确的结论：在给定计算预算 C 的条件下，**模型参数量 N 和训练数据量 D 应该以相近的速度同步增长**，才能达到最优性能。
- 早期的GPT-3（175B参数，仅训练0.3T token）就是“参数过大、训练不足”的典型案例。同等算力下用更小的模型配合更多的数据，反而能获得更低的损失。
- 这一结论直接推动了训练数据量的爆发式增长——从GPT-3的0.5T飙升至Qwen3的36T。



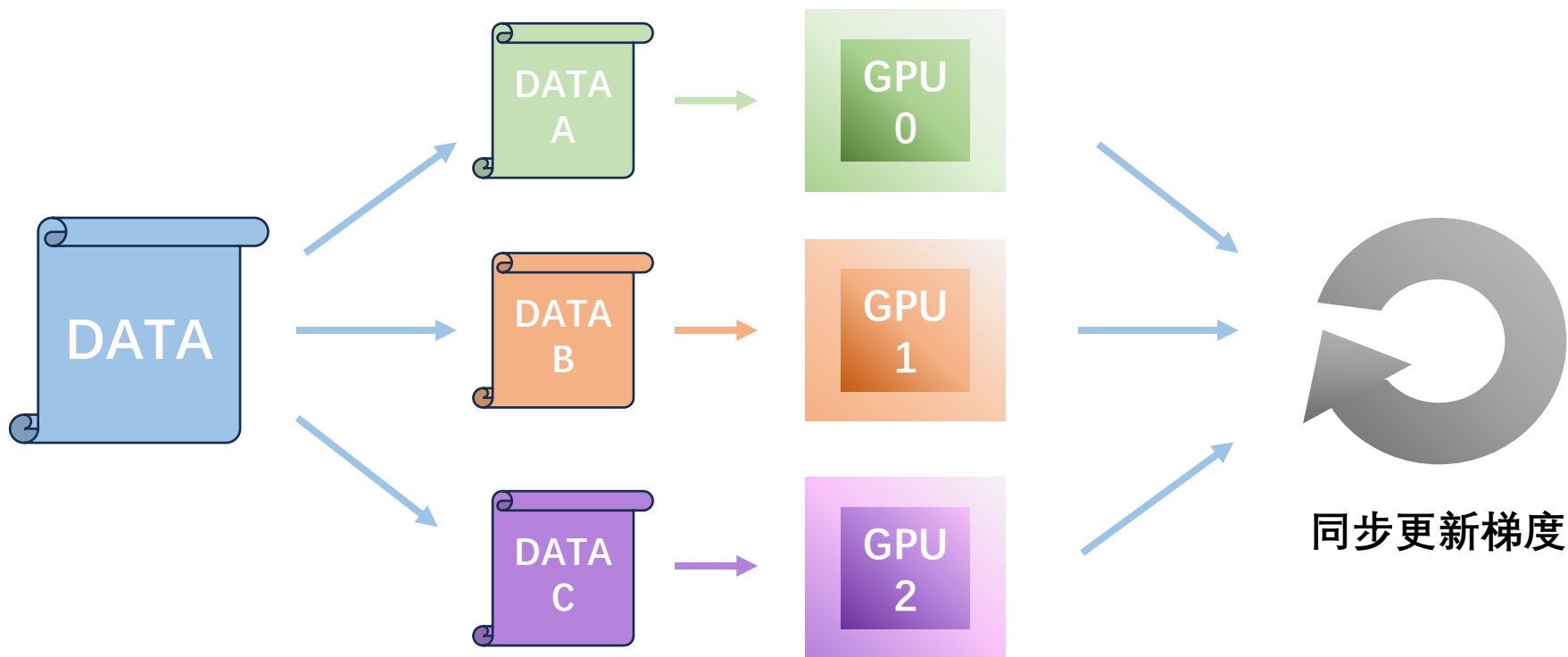
工程上如何支撑这种规模扩张？

- 知道参数和数据要同步增长——这个结论在理论上很清晰，但工程上有一个现实问题：**数百亿乃至数千亿参数的模型，单张GPU根本装不下。**
- 这不仅是显存的问题，还涉及训练速度、通信开销、硬件容错等一系列工程挑战。
- 因此，预训练技术的另一条主线是：**如何用分布式系统来承载超大模型的训练。**

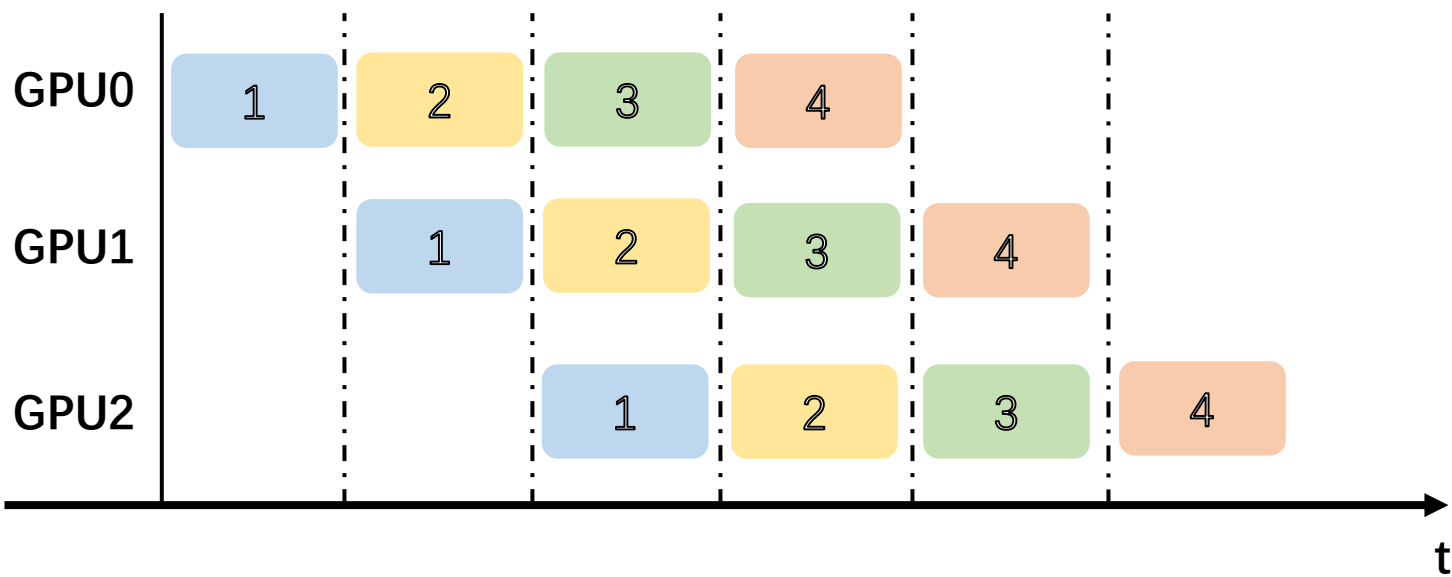
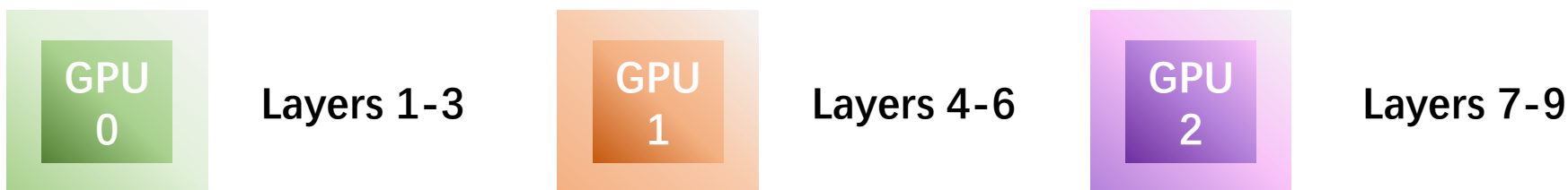


- 当前主流方案包含三类并行策略，通常组合使用：
 - 1. **数据并行**：将训练数据拆分成多份，每个GPU处理一份，各自计算梯度后同步更新。这是最基础的并行方式，前提是每个GPU能装下完整模型。
 - 2. **流水线并行**：将模型的不同层分配到不同设备上，通过"微批次"机制让各设备流水协作，减少空闲时间。
 - 3. **张量并行**：将单层内的巨大矩阵拆分成多个小块，让不同GPU分别计算各自部分，数学上完全等价，但单卡显存负担骤降。
- 在实际训练中，这三类策略通常会组合使用（如三维并行），以在算力、显存和通信三个维度间取得最优平衡。

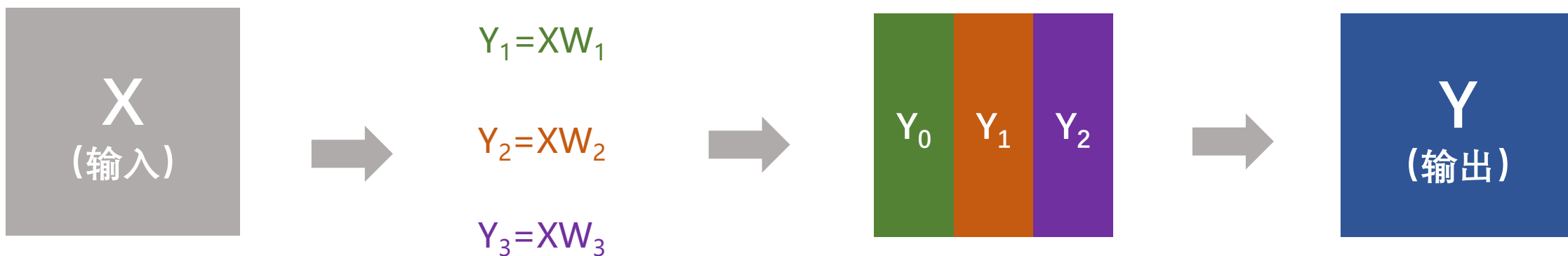
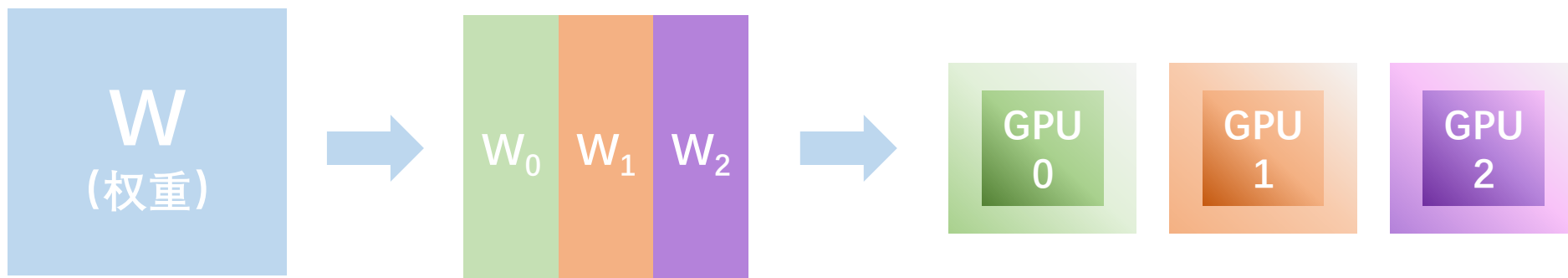
- 当前主流方案包含三类并行策略，通常组合使用：
- 1. **数据并行**：将训练数据拆分成多份，每个GPU处理一份，各自计算梯度后同步更新。这是最基础的并行方式，前提是每个GPU能装下完整模型。



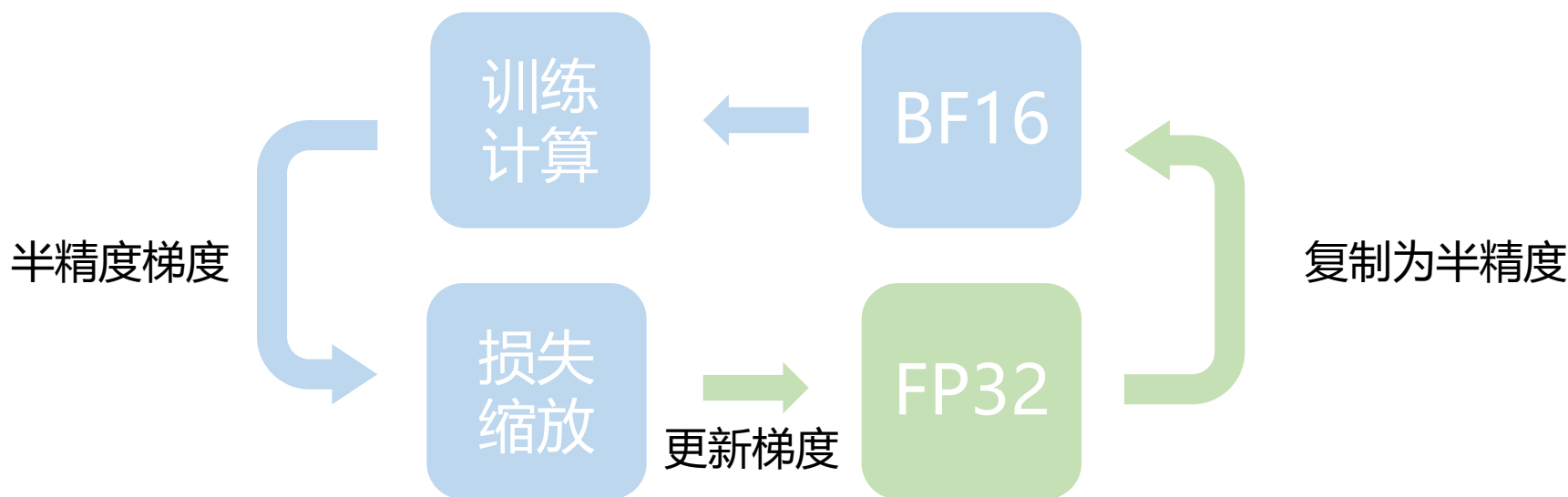
- 当前主流方案包含三类并行策略，通常组合使用：
- 2. **流水线并行**：将模型的不同层分配到不同设备上，通过“微批次”机制让各设备流水协作，减少空闲时间。



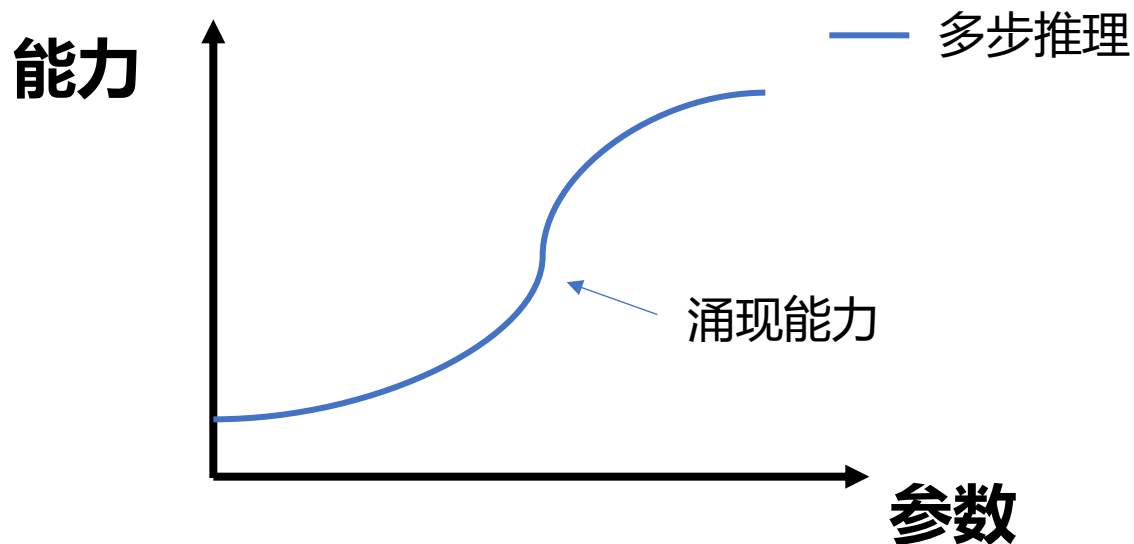
- 当前主流方案包含三类并行策略，通常组合使用：
- 3. **张量并行**：将单层内的巨大矩阵拆分成多个小块，让不同GPU分别计算各自部分，数学上完全等价，但单卡显存负担骤降。



- 除了把计算和模型拆分到多个GPU上，另一项关键的工程优化是**混合精度训练**：在前向和反向传播中使用半精度（如FP16或BF16）来加速计算、节省显存，同时维护一份全精度（FP32）的主权重副本用于参数更新。
- 为什么要保留全精度副本？因为梯度在反向传播中可能会变得很小，半精度存储时容易“下溢”变成0。全精度副本保证了数值稳定性，同时兼顾了半精度计算的速度优势。
- 配合损失缩放等技巧，混合精度训练能够在几乎不影响模型质量的前提下，大幅降低显存占用和计算时间。



- 缩放定律带来的不仅是“量”的提升。研究者发现，当模型规模突破某个阈值时，一些能力会**从接近随机水平“骤升”到显著优于基线**——这种现象被称为“涌现能力”。
- 例如，多步算术推理、代码理解、上下文学习等能力，在小模型中几乎不存在，但模型一旦超过某个参数量级，这些能力就突然显现出来。
- 涌现能力的发现表明：不断扩张的模型规模不仅能带来数值上的损失降低，更可能触发**能力维度上的质变**。这也是各大AI实验室仍在持续竞逐更大模型的重要原因。



- 预训练通过语言建模这一自监督任务，在海量文本上训练仅解码器Transformer架构，使模型内化了丰富的语言知识与世界知识。
- 缩放定律揭示了规模扩张的收益规律，分布式训练与混合精度训练为这种扩张提供了工程支撑。
- 经过预训练的模型已具备通用语言理解能力，但如何高效地将其适配到具体任务——这将是下一节“提示学习方法”所要回答的问题。



大模型预训练与微调方法

3.1 大模型预训练技术

3.2 提示学习方法

3.3 大模型微调技术

3.4 大模型偏好对齐技术

3.5 大语言模型推理增强方法

1.2 提示学习方法

- 在1.1节中，我们学习了预训练技术，但预训练模型本质上只是一个“续写引擎”——给定上文，它补全下文。它并不知道自己要完成翻译、问答还是摘要任务。
- 如何让这个通用的续写引擎“听懂”我们的要求，并按我们的意图执行特定任务？这就需要一种引导机制。在LLM的语境中，这种引导机制被称为**提示 (Prompt)**。

- 提示是输入给语言模型的文本，它描述了当前任务是什么，以及用户希望模型如何回应。在工程实现中，我们通常不直接编写提示文本，而是通过**提示模板 (Template)** 来构造。模板是一段含有占位符的文本，使用时将具体信息填入占位符，即可生成标准化的提示。



模版： 如果{*premise*}，你有什么晚餐建议吗？

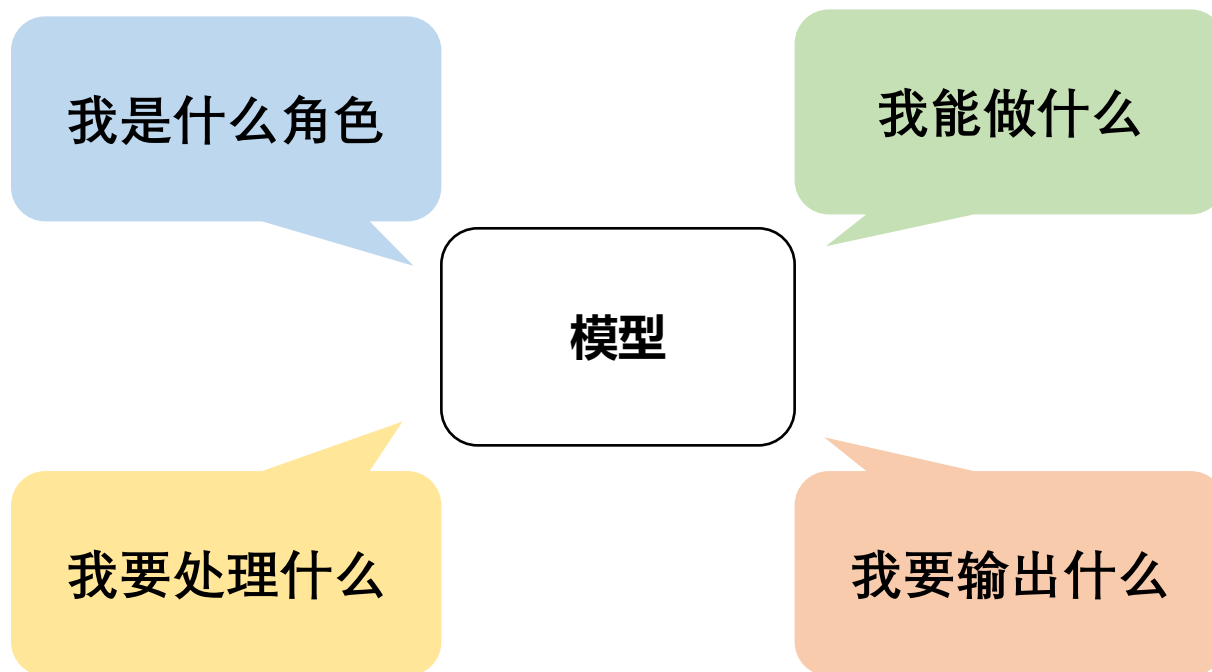


用户： 家里有一些西红柿和鸡蛋

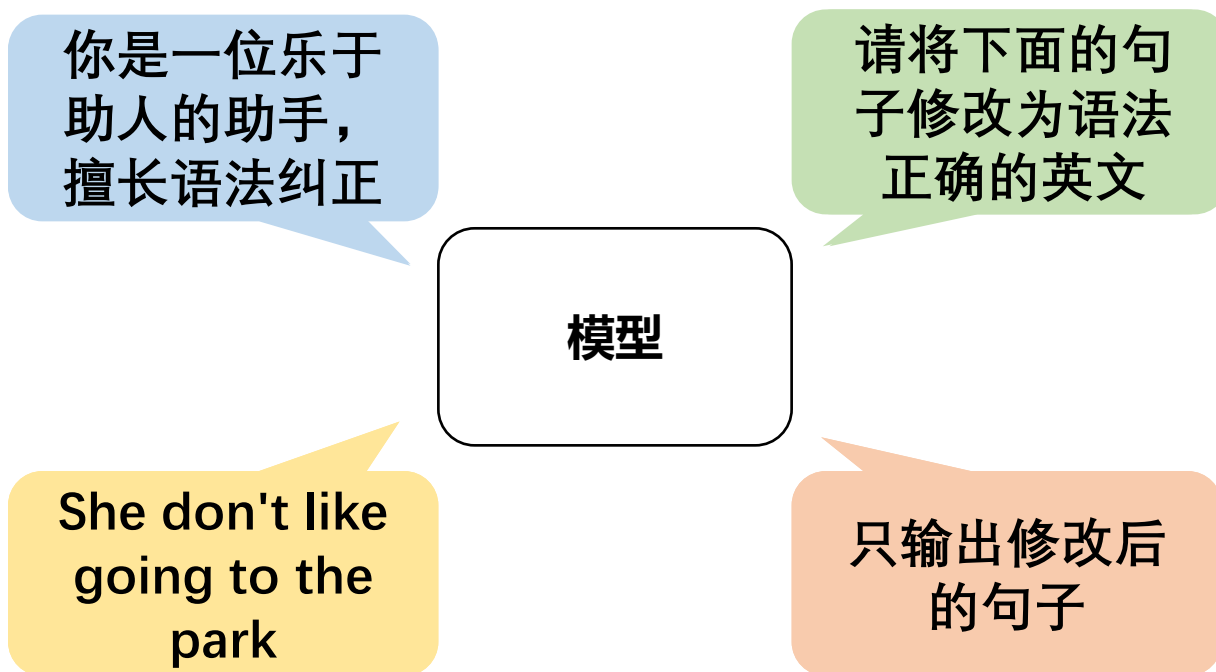


提示： 如果家里有一些西红柿和鸡蛋，你有什么晚餐建议吗？

- 一个完整的提示通常由多种信息成分构成。常见的要素包括：**任务描述、系统角色设定、用户的输入内容、以及期望的输出格式约束。**
- 将任务拆解成清晰的层次放入提示模板，能显著降低模型理解任务的歧义，是提升零样本提示效果的关键技巧。
 - 以语法纠错为例，下面的提示包含三层信息：系统角色“你是一位乐于助人的助手，擅长语法纠正”、任务指令“请将下面的句子修改为语法正确的英文”、以及待处理的具体输入内容“`She don't like going to the park.`”



- 一个完整的提示通常由多种信息成分构成。常见的要素包括：**任务描述、系统角色设定、用户的输入内容、以及期望的输出格式约束。**
- 将任务拆解成清晰的层次放入提示模板，能显著降低模型理解任务的歧义，是提升零样本提示效果的关键技巧。
 - 以语法纠错为例，下面的提示包含三层信息：系统角色“你是一位乐于助人的助手，擅长语法纠正”、任务指令“请将下面的句子修改为语法正确的英文”、以及待处理的具体输入内容“`She don't like going to the park.`”



- 一个完整的提示通常由多种信息成分构成。常见的要素包括：**任务描述、系统角色设定、用户的输入内容、以及期望的输出格式约束。**
- 将任务拆解成清晰的层次放入提示模板，能显著降低模型理解任务的歧义，是提升零样本提示效果的关键技巧。
 - 以语法纠错为例，下面的提示包含三层信息：系统角色“你是一位乐于助人的助手，擅长语法纠正”、任务指令“请将下面的句子修改为语法正确的英文”、以及待处理的具体输入内容 “She don't like going to the park.”



- 当我们把完整的任务指令直接输入模型，而不提供任何“输入-输出”示例时，这种提示方式称为**零样本提示**。模型完全依靠预训练阶段习得的指令理解能力直接作答。
- 零样本提示的优点是**简洁高效**，适合那些模型“本来就会”的基础任务。
- 但它的局限也很明显：当任务复杂或模型的解题路径不唯一时，直接作答的准确率往往不够理想。



提示：你是一位乐于助人的助手，擅长语法纠正。



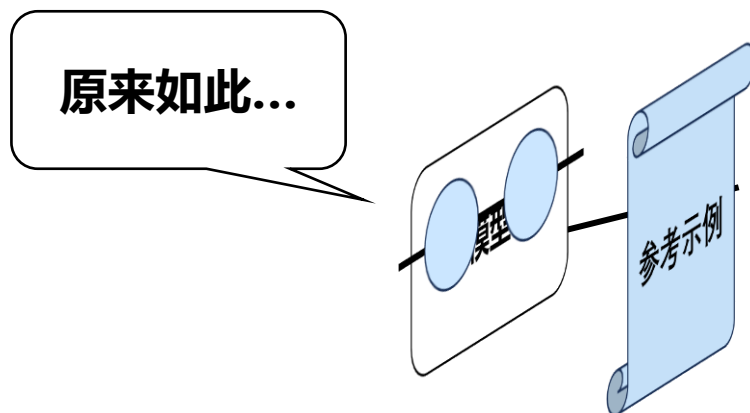
用户：请将下面的句子修改为语法正确的英文，
She don't like going to the park



输出：She doesn't like going to the park.

为什么需要提供示例

- 既然零样本提示有时效果欠佳，一个很自然的改进思路是：如果我们给模型看几个“别人是怎么做这道题”的例子，模型是不是就能照着样子学会呢？
- 这正是**上下文学习 (In-Context Learning)**的核心思想。它指在提示中直接插入若干输入-输出示例作为演示，让模型在推理时从这些示例中“学习”任务模式，并据此对新输入做出预测。
- 关键区别在于：这个过程完全**不更新**模型的参数，学习完全发生在推理阶段的上下文之中。预训练模型本身没有被改变，只是通过示例被“引导”到了正确的输出轨道上。



- 当我们在提示中仅提供一个已完成任务的示例时，称为**单样本提示**。模型只需要看一个例子，就能推测出当前任务的要求和格式。



示例： 请将下面的句子修改为语法正确的英文：

输入：There is many reasons to celebrate.

输出：There are many reasons to celebrate.



用户： 请将下面的句子修改为语法正确的英文：

输入：She don't like going to the park.

- 当我们在提示中提供三个或更多已完成任务的示例时，称为**少样本提示**。示例数量的增加有助于模型更准确地把握任务边界和输出格式。



示例： 请将下面的句子修改为语法正确的英文：

输入：There is many reasons to celebrate.

输出：There are many reasons to celebrate.

输入：Me and my friend goes to the gym every day.

输出：My friend and I go to the gym every day.

输入：He don't know the answer.

输出：He doesn't know the answer.

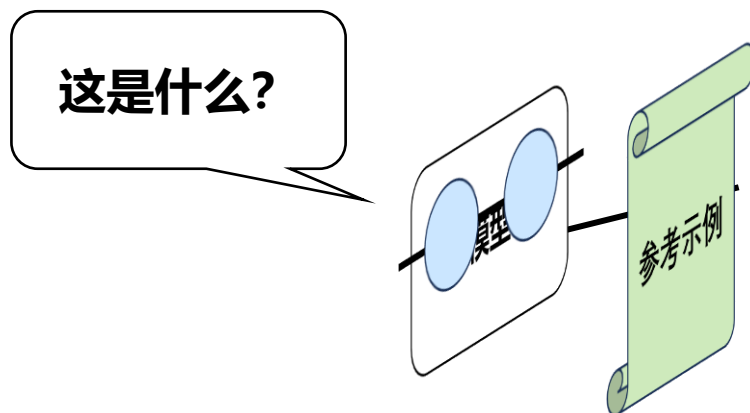


用户： 请将下面的句子修改为语法正确的英文：

输入：She don't like going to the park.

上下文学习的前提条件

- 需要特别注意的是，上下文学习并非万能的“补丁”。它的有效性依赖于一个硬性前提：**模型在预训练阶段必须已经储备了完成该任务所需的知识。**
- 例如，若希望LLM将因纽特语翻译为英语，但模型在预训练阶段严重缺乏因纽特语数据，那么无论提供多少个高质量的翻译示例，模型也无法产出可用的翻译结果。**示例只能“唤醒”已有知识，无法“创造”新知识。**
- 因此，上下文学习适合那些“模型会做但不确定该怎么做”的任务。如果模型“根本不会”，更有效的方案应该是微调或继续预训练，而非在提示工程上徒劳尝试。



为什么需要思维链

- 零样本和少样本提示已经能够解决不少常规NLP任务。但当面对数学应用题或需要多步逻辑推理的问题时，情况变得棘手起来。
 - 具体例子：若直接问“计算数字2、4和9的平均值”，并提供“1、3、5、7的平均值是4”作为示例，模型也许能猜对。但如果将问题升级为“数字2、4、9的平方的平均值是多少”，仅靠简单的问答映射就容易出错。



- 这就引出一个深层问题：我们能不能让模型展示它的“解题过程”？如果能让模型像人一样先列出计算步骤再给出结论，准确率能否大幅提升？

- **思维链 (Chain-of-Thought, CoT) 提示**的核心是：在提示中不仅展示问题和答案，还展示从问题到答案的完整推理步骤，强制模型在给出最终答案前生成中间推导过程。

普通少样本提示



示例：

Q: 计算1、3、5、7的平方的平均值。
A: 21。

思维链风格提示



示例：

Q: 计算1、3、5、7的平方的平均值。
A: 计算每个数的平方： $1^2=1$ ， $3^2=9$ ， $5^2=25$ ， $7^2=49$ 。求和得84。共有4个数。
用总和除以个数： $84/4=21$ 。答案是21。

- 与基础提示类似，思维链也分为少样本和零样本两种形式。少样本CoT需要在提示中手工构造带有详细推理步骤的示例，模型据此学习推理范式并迁移到新问题上。
- 而零样本CoT更为简洁：无需准备任何推理示例，只需在常规提示末尾加上一句“Let's think step-by-step.”（让我们一步步思考），模型就会自行展开推理。这个简短的指令能显著激活模型的审慎推理能力，在多步数学和常识推理任务上带来可观的准确率提升。

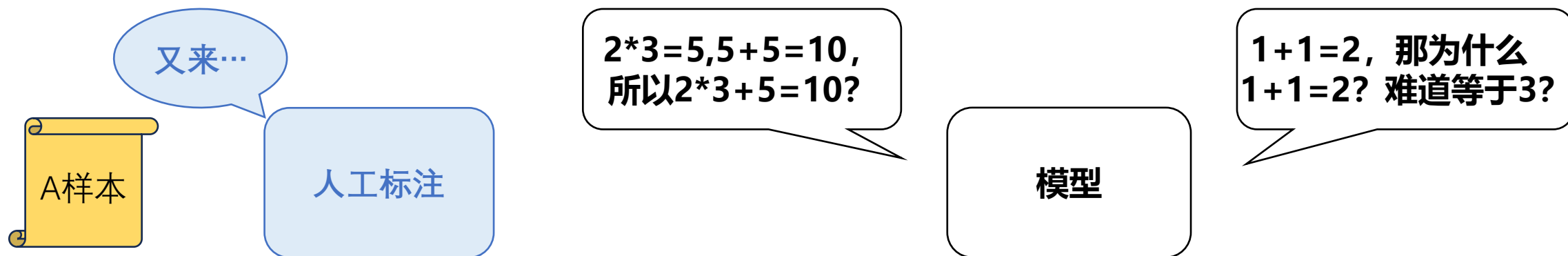


用户：计算2、4、9的平方的平均值。Let's think step-by-step.



输出：先计算每个数的平方： $2^2=4$ ， $4^2=16$ ， $9^2=81$ 。将平方相加： $4+16+81=101$ 。一共有3个数。用总和除以个数： $101/3 = 33.67$ 。所以答案是33.67。

- 尽管思维链效果显著，它在实际应用中 also 面临一些不可忽视的局限：
- 1. **少样本CoT依赖高质量推理示例的构造**。这些示例需要人工精心设计或自动筛选，获取成本较高。
- 2. **“链条效应”是一把双刃剑**。例如，若模型在计算 $2(y+2)$ 时误展开为 $2y+2$ 而非正确的 $2y+4$ ，这一错误会沿着推理链条传播，导致后续所有步骤和最终答案全部错误，且难以定位问题源头。
- 3. **并非所有任务都适合CoT**。对于简单的文本分类或信息抽取，强制模型生成推理步骤反而会引入不必要的计算开销和噪声。



- 至此，我们讨论的所有提示策略——零样本、少样本、思维链——都由人类手工设计。但当任务数量增长到成百上千时，为每个任务人工调试提示变得极不现实。
- 人工设计存在三重瓶颈：第一，不同大模型对同一提示的响应差异巨大，迁移成本高；第二，人类直觉很难穷尽所有有效的提示表述方式；第三，为了追求效果，人工提示往往变得冗长，增加了每次推理的计算开销。
- 这些瓶颈引出了两个核心问题：能否让机器**自己设计并优化**提示？除了自然语言文本，还有没有**更高效**的提示表示形式？

人工标注

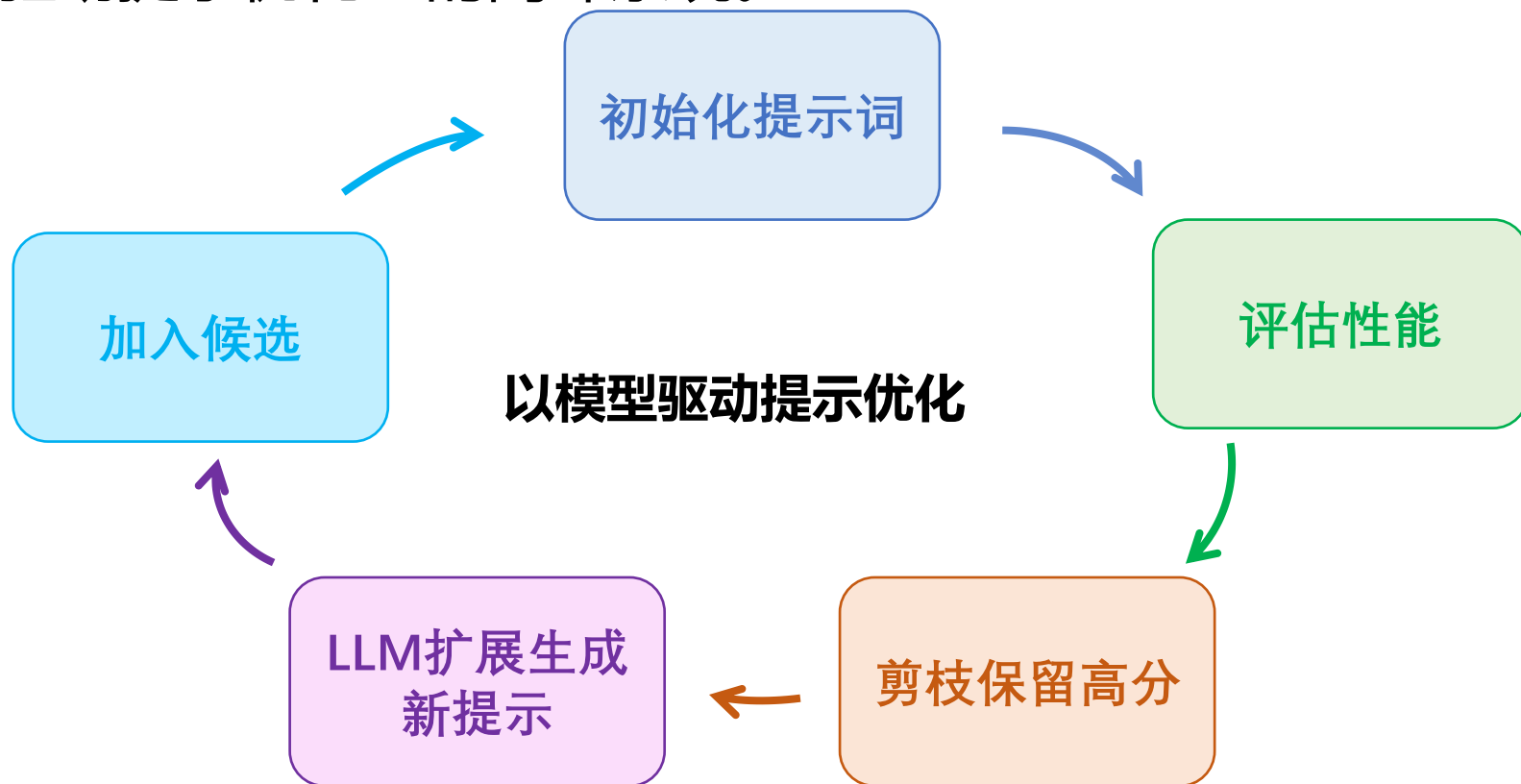
你已经是个成熟的模型了，该学会自己设计提示词了

模型

QAQ?

自动提示优化的框架

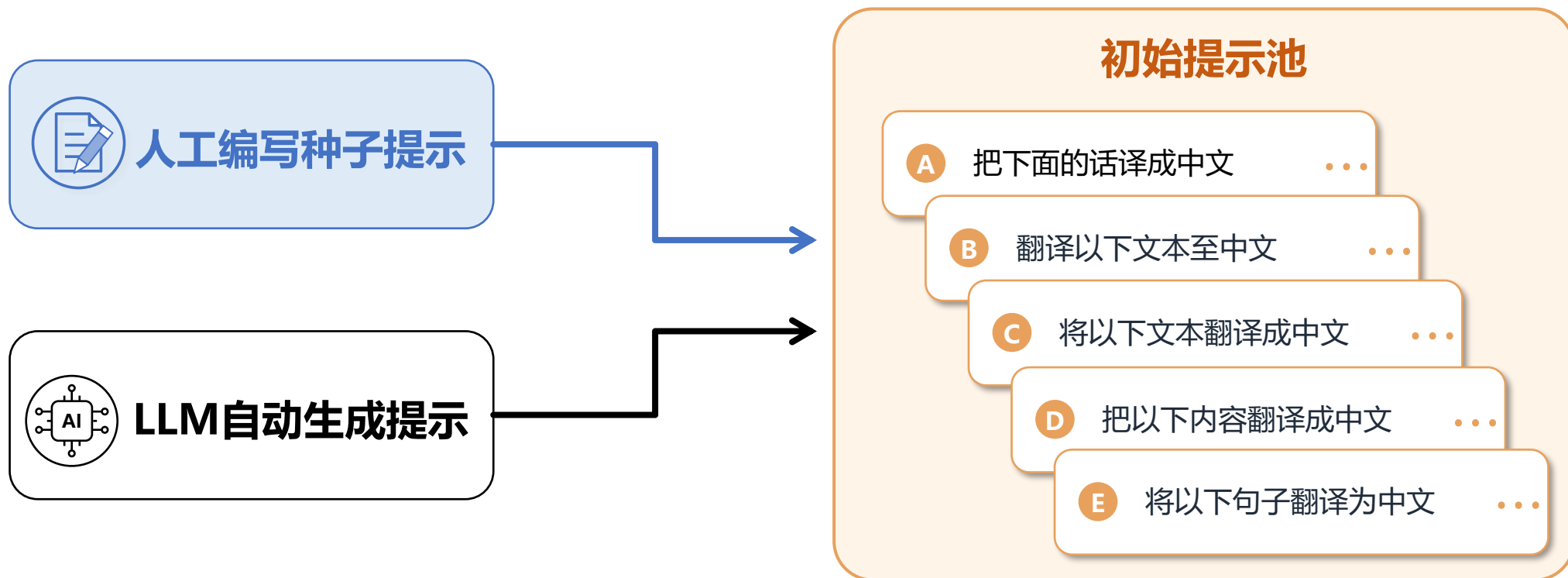
- 自动提示优化的目标是将提示设计转化为一个算法可解的搜索问题。通用的优化框架包含三个标准组件：**搜索空间**（所有可能的提示变体）、**性能评估**（在验证集上测试每个提示的效果）、以及**搜索策略**（决定如何遍历这个空间）。当前最成功的方法是利用大语言模型自身来实现这三个组件，形成“以模型驱动提示优化”的闭环系统。



- 以Zhou等人的工作为代表，迭代式提示优化通常包含四个循环步骤。
- 首先进行**初始化**：由人工编写少量种子提示，或让LLM根据任务描述自动生成初始提示池。
- 随后进入**评估**环节：将池中每个提示送入语言模型，在验证集上计算性能指标（如准确率）。
- 若候选池过大，则依据分数进行**剪枝**，仅保留得分高的一部分提示。
- 最后是**扩展**环节：以保留提示为基础，让LLM生成语义相近的变体（如改写、局部替换词汇等），加入候选池。

迭代优化的具体流程

- 以Zhou等人的工作为代表，迭代式提示优化通常包含四个循环步骤。
- 首先进行**初始化**：由人工编写少量种子提示，或让LLM根据任务描述自动生成初始提示池。



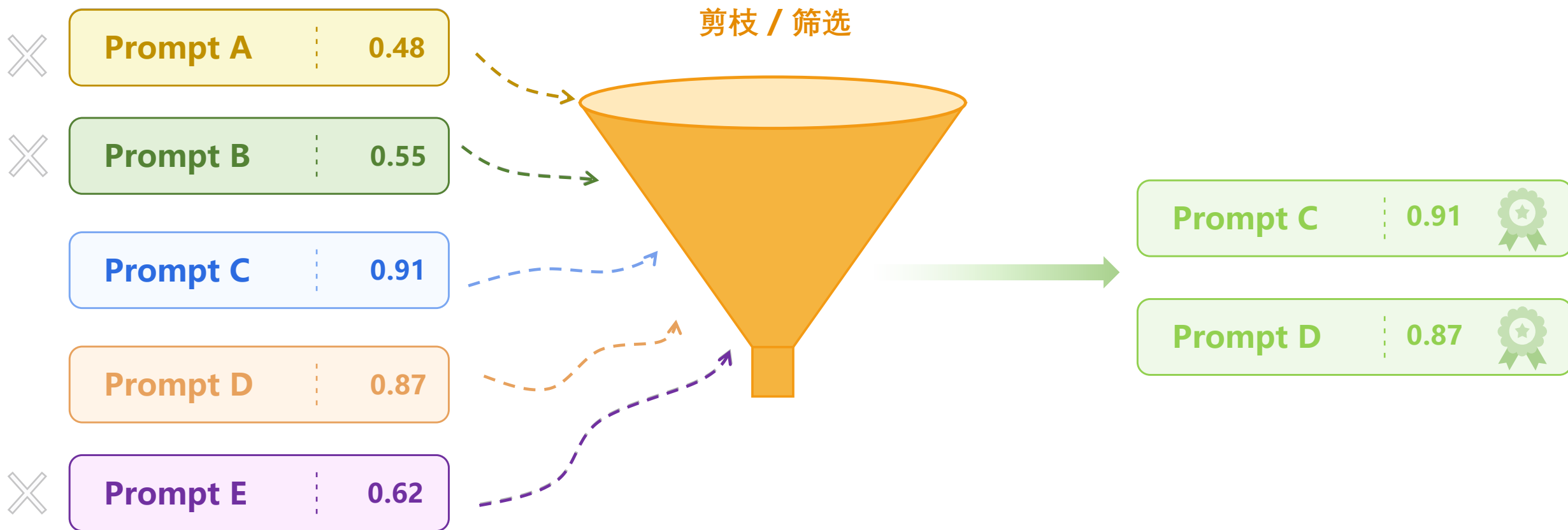
迭代优化的具体流程

- 随后进入**评估**环节：将池中每个提示送入语言模型，在验证集上计算性能指标（如准确率）。



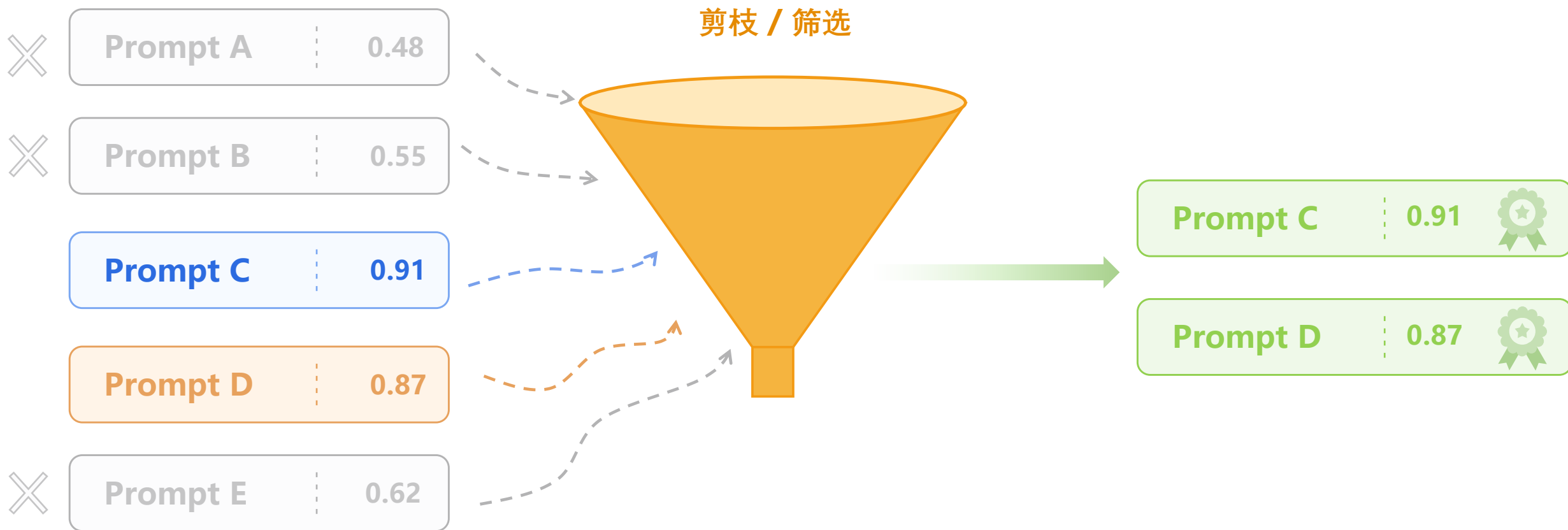
迭代优化的具体流程

- 若候选池过大，则依据分数进行**剪枝**，仅保留得分高的一部分提示。



迭代优化的具体流程

- 若候选池过大，则依据分数进行**剪枝**，仅保留得分高的一部分提示。



迭代优化的具体流程

- 最后是**扩展**环节：以保留提示为基础，让LLM生成语义相近的变体（如改写、局部替换词汇等），加入候选池。



- 无论是人工编写还是自动搜索，最终提示仍然以自然语言文本的形式存在，这被称为**硬提示**。硬提示直观易懂，但也带来了**效率**和**表达力**的瓶颈。
- 以少样本提示为例，若每个提示包含三个完整示例，加上任务描述和系统设定，总长度可能超过500个token。在每次推理时，模型都要重复编码这500个token，造成大量的重复计算。
- 更根本的问题是：离散的文本空间存在“表达天花板”——某些最有效的引导信号可能不存在于任何自然语言句子中，人类无法通过文本将其表达出来。那么，我们能否跳出自然语言的框架？

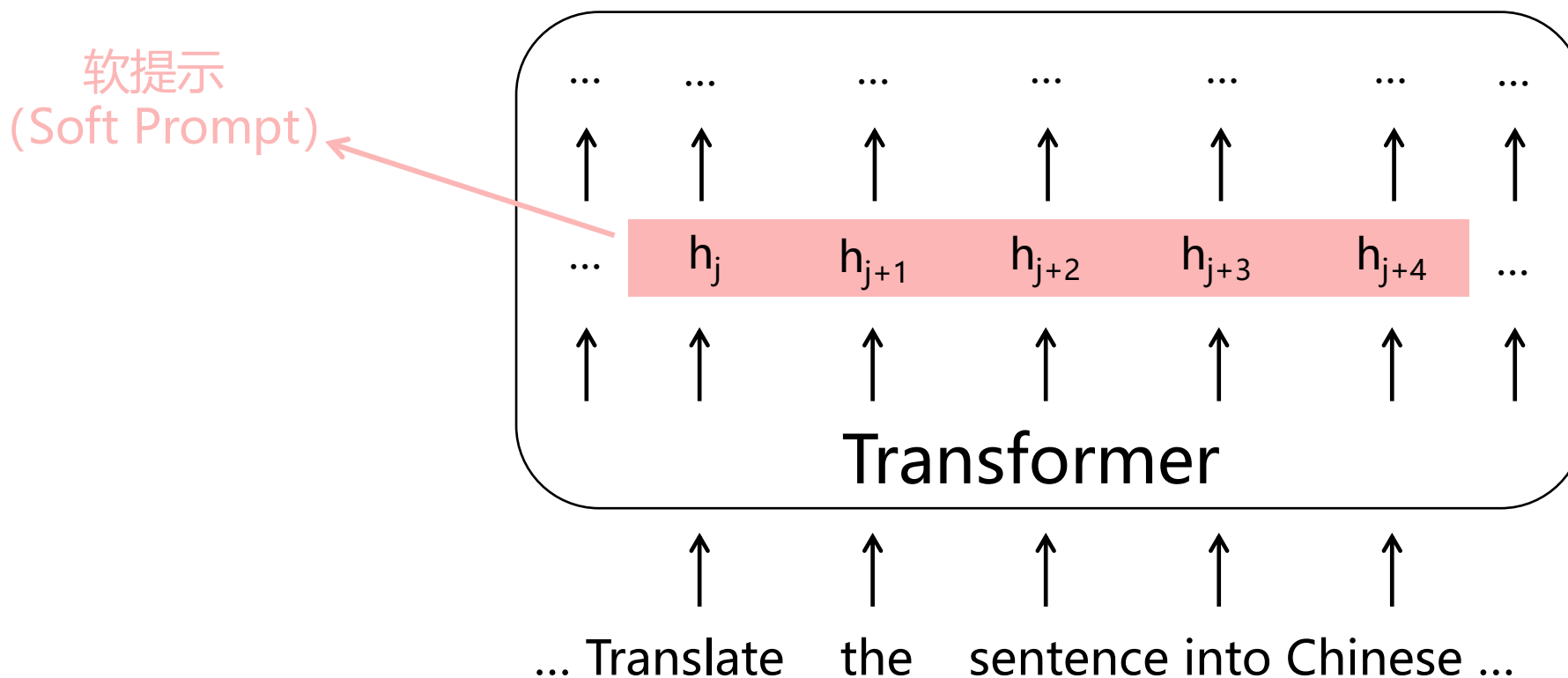


系统设定：你是.....
任务描述：你需要.....
任务示例：Q:
输出格式：输出.....
安全限制：你不能.....

500+token

软提示的核心概念

- 硬提示作为token序列输入模型后，会被映射为连续空间的向量表示。例如“Translate the sentence into Chinese.”这5个token在模型的嵌入层会被编码为5个 d 维向量。这些连续向量才是模型真正“看见”和“处理”的东西。



- 硬提示作为token序列输入模型后，会被映射为连续空间的向量表示。例如“Translate the sentence into Chinese.”这5个token在模型的嵌入层会被编码为5个 d 维向量。这些连续向量才是模型真正“看见”和“处理”的东西。
- **软提示**的概念由此诞生：不使用任何自然语言文本，而在模型的嵌入空间中定义一组可学习的连续向量作为提示。这些向量不与任何具体的词汇对应，是完全抽象的数值参数。
- 软提示的优点是**紧凑**（通常只需10余个向量）、**高效**（省去编码长文本的开销）且表达力更强，可以在连续空间中自由优化，找到硬提示无法触及的引导方向。



系统设定：你是.....
任务描述：你需要.....
任务示例：Q:
输出格式：输出.....
安全限制：你不能.....

500+token

V S



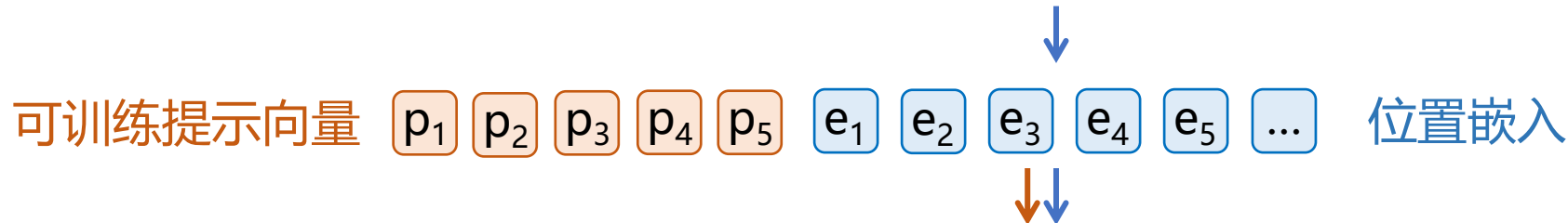
V[1]: ...
V[2]: ...
V[3]: ...
V[4]: ...
V[5]: ...

~20 vector

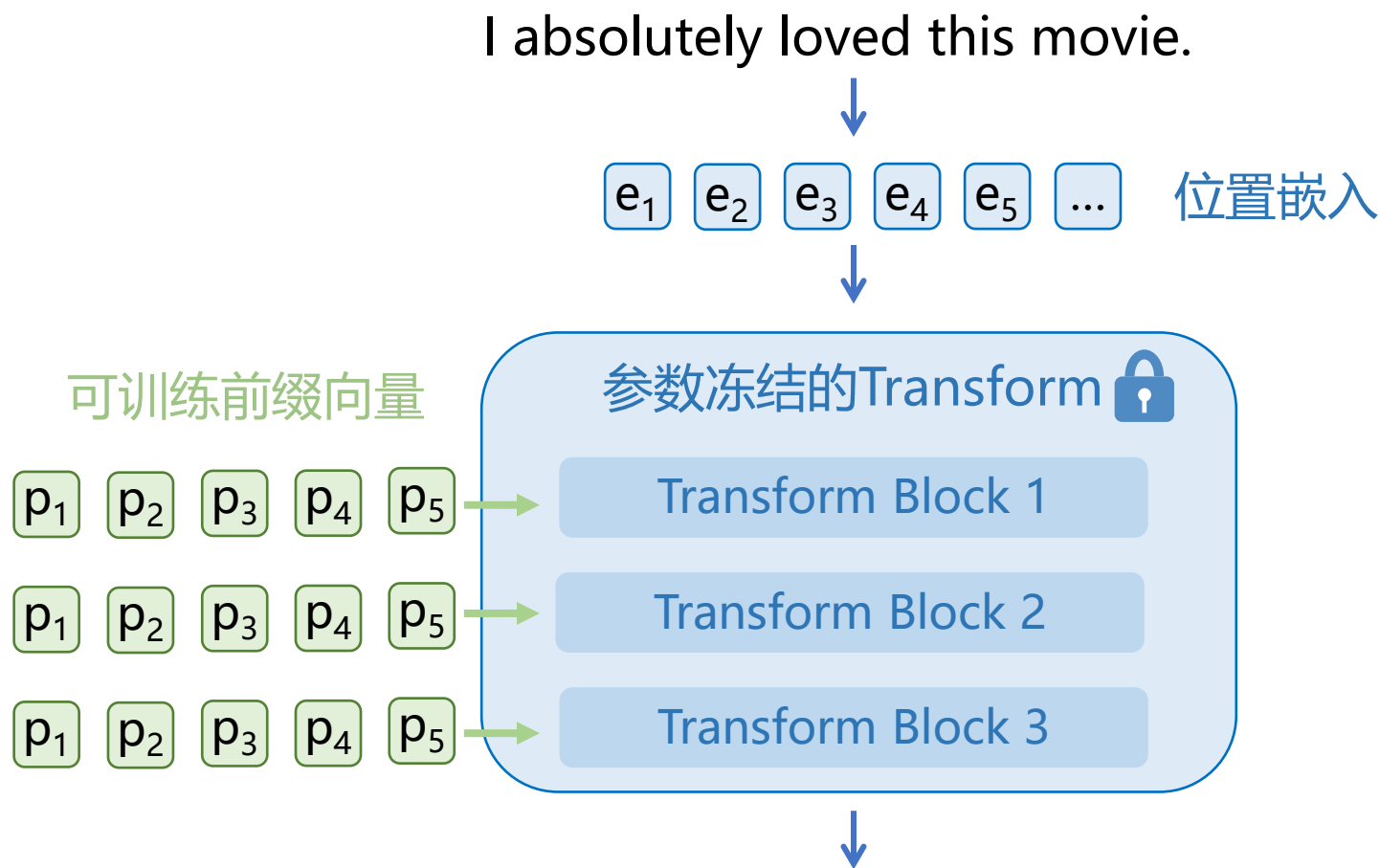
软提示的实现方式

- 软提示的思想已在参数高效微调领域催生了经典方法。**提示微调** (Prompt Tuning) 仅在输入嵌入层添加若干可训练的连续向量, 其余模型参数全部冻结, 以极小的参数量适配新任务。

I absolutely loved this movie.



- **前缀微调** (Prefix Tuning) 则更进一步：在每一层Transformer的输入前都添加可训练的前缀向量，为模型的不同层提供层次化的任务引导，表达能力更强。



- 本节以“如何引导预训练模型执行新任务”为主线，构建了从基础到进阶的完整技术脉络：从零样本提示出发，通过加入具体示例演进出上下文学习，再通过加入推理步骤升级为思维链，层层递进地拓展了提示的能力边界。
- 在认识到人工设计的瓶颈后，我们进入自动提示优化，将提示设计转化为搜索问题；进而引入软提示概念，彻底突破自然语言的限制，将提示转化为可学习的连续参数，实现了提示工程与参数高效微调的深度融合。
- 掌握了如何“引导”模型之后，我们面临一个新的选择：如果引导已经足够充分但模型能力仍然不足，那就需要进入下一层面的技术——通过有监督微调主动改变模型参数本身，从而更深层地适配任务需求。下一节将全面展开大模型微调技术。



大模型预训练与微调方法

3.1 大模型预训练技术

3.2 提示学习方法

3.3 大模型微调技术

3.4 大模型偏好对齐技术

3.5 大语言模型推理增强方法

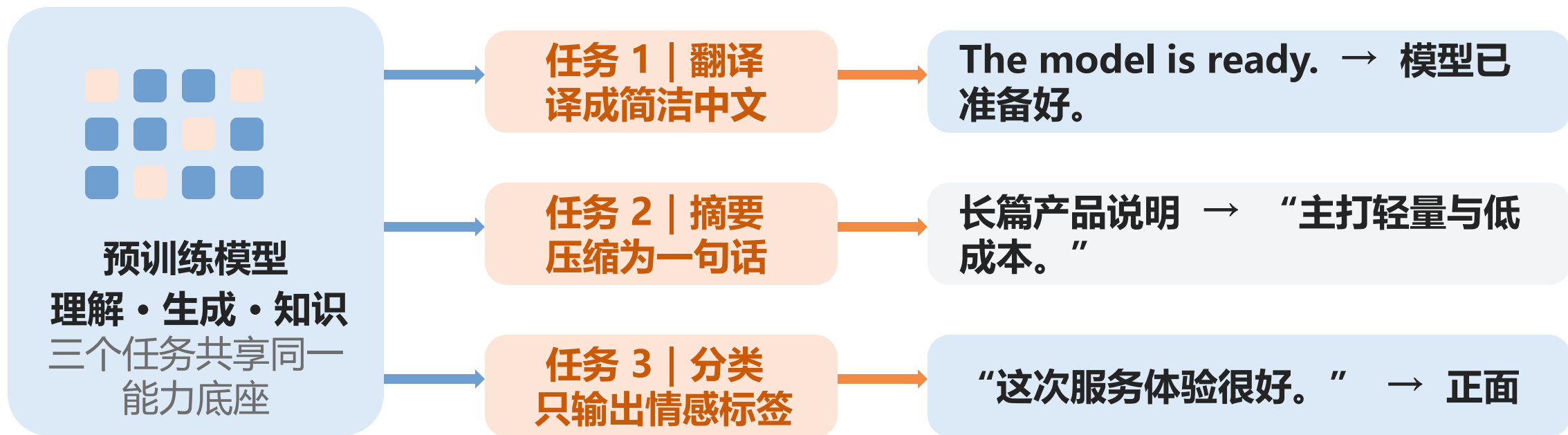
从预训练到任务适配

- 预训练模型已经具备理解、生成与知识能力，任务适配并不重新学习这些能力，而是通过指令和输出约束，让同一个模型完成翻译、摘要、分类等具体任务。

同一个预训练模型

加入明确任务约束

得到稳定、可验证的输出



核心：任务适配不是重新学习知识，而是学习“按要求使用已有能力”

微调的基本思路

- 微调通过少量高质量的“指令—回答”样本继续训练，使模型学会在给定输入下生成期望输出。



条件最大似然目标

$$\theta^* = \arg \max_{\theta} \sum_{(x, y) \in D_{\text{tune}}} \log p_{\theta}(y | x)$$

微调的本质问题

- 同一个 LLaMA-65B 经大规模预训练获得能力，仅需 1,000 条精选示范，即可通过 SFT 校准回答方式。

① 大规模预训练：学会知识与语言能力

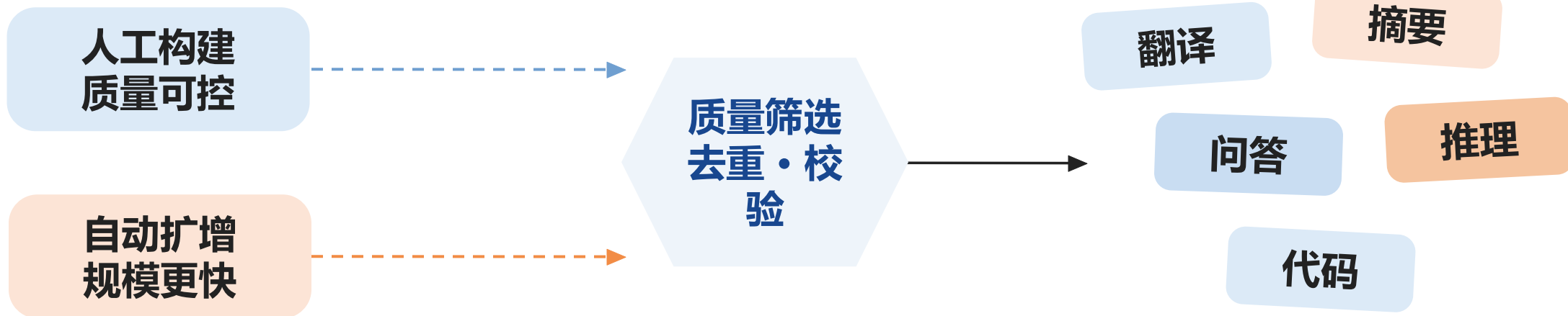
② 精选示范校准：学会如何回答 1,000 条高质量示范



模型主体不变，少量高质量数据负责“校准方向”

- 要让模型学会新的任务行为，首先需要解决：
训练数据从哪里来，又如何保证质量与覆盖度？

数据从哪里来？



质量与覆盖度同时决定训练价值

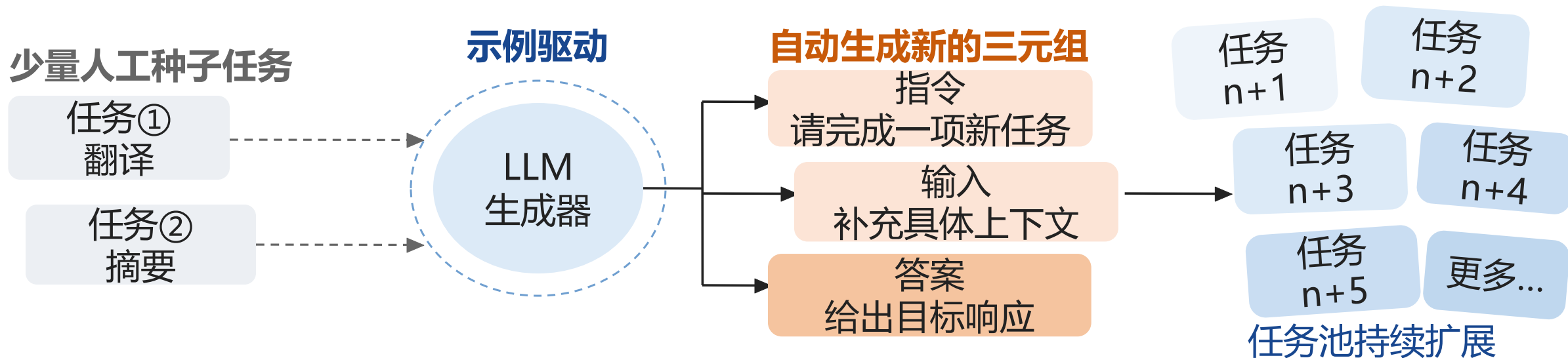
监督微调数据的人工构建

- 人工构建一条 SFT 数据，需要依次完成任务设计、字段填写和质量审核。



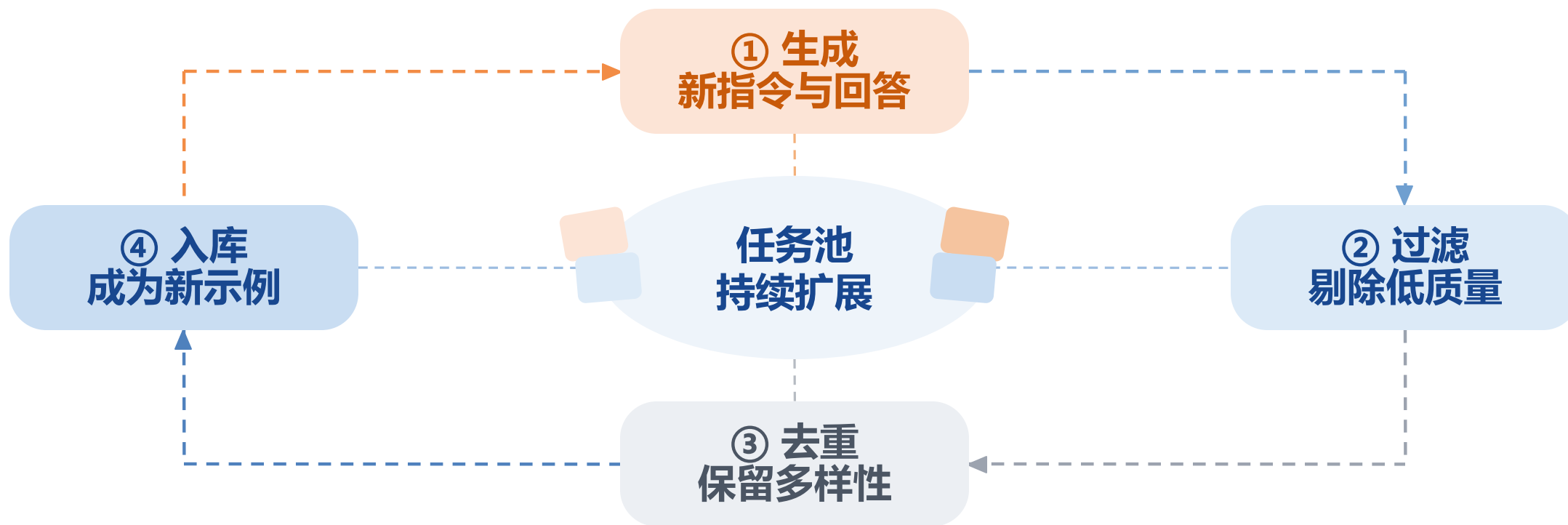
Self-Instruct的出发点

- Self-Instruct利用模型从少量种子任务自动生成指令、输入和答案，将数据构建转化为自动扩增。



Self-Instruct的迭代流程

- 通过“生成—过滤—去重—入库”的循环，Self-Instruct能够持续扩大数据规模和任务多样性。



高质量样本回流，驱动下一轮生成

- 有了指令数据后，模型究竟应该学习哪些内容，训练损失又应当计算在哪些位置？

模型看到的完整训练序列

输入：提供上下文

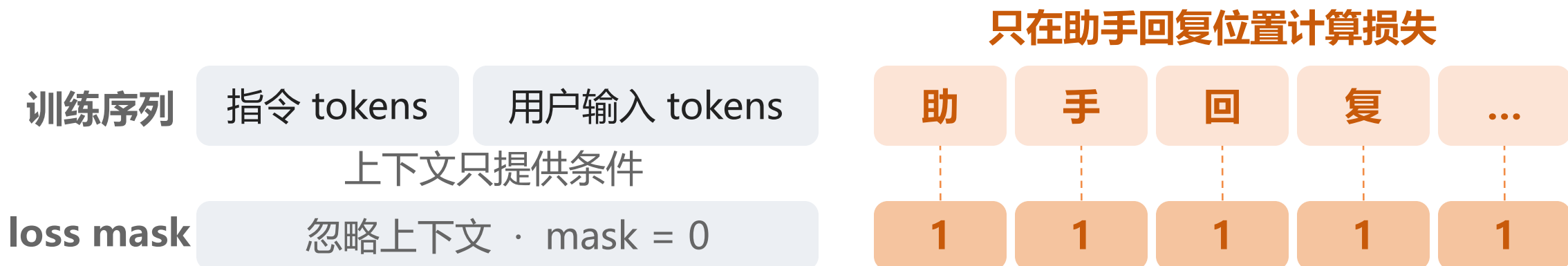
输出：模型逐 token 生成



模型看见整段序列，但 loss 的“聚光灯”究竟应该照向哪里？

SFT的统一学习目标

- 无论单轮还是多轮，SFT的核心都是在给定上下文后，只学习模型应当生成的助手回复。



assistant-only loss: 仅累积回答 token

$$\begin{aligned}\theta^* &= \arg \max_{\theta} \sum_{(x,y) \in D} \log p_{\theta}(y | x) \\ &= \arg \max_{\theta} \sum_{(x,y) \in D} \sum_{i=1}^n \log p_{\theta}(y_i | x, y_{<i})\end{aligned}$$

单轮SFT的最小实现

- 在单轮场景中，将输入与回答拼接为一个序列，并通过损失掩码只训练回答部分。



从单轮走向多轮

- 真实应用通常是连续对话，因此模型还需要结合前序问答，才能生成符合当前语境的回复。

单轮：只看当前输入

用户：请解释这个概念

助手：给出一次回复

上下文
累积

多轮：前序问答共同构成语境

用户 1

助手 1

用户 2

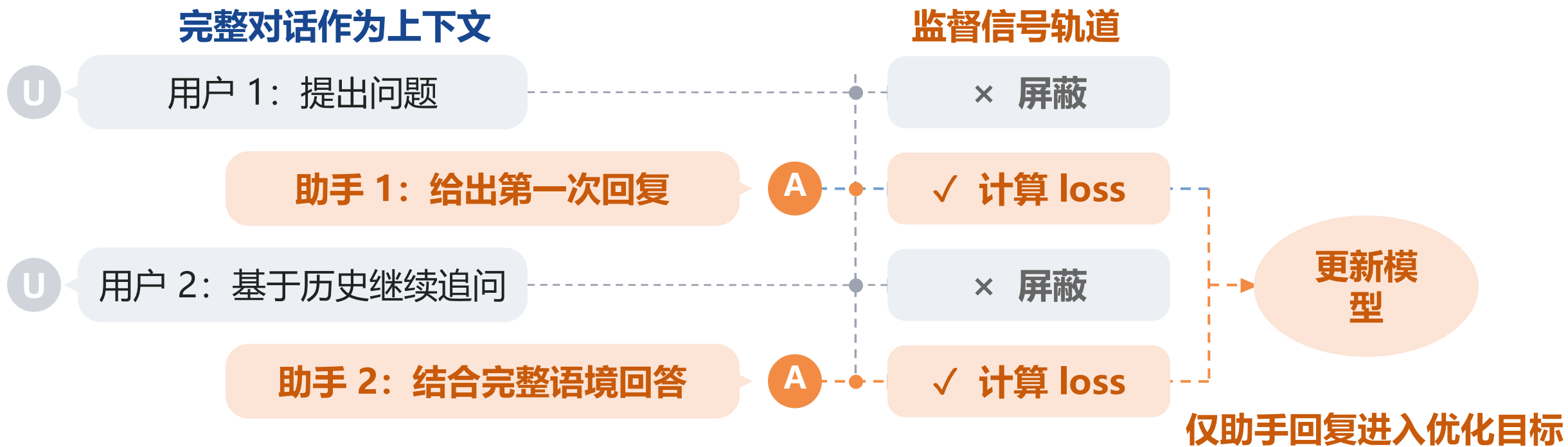
当前助手回复

多轮 assistant-only loss

$$\theta^* = \arg \max_{\theta} \sum_{k=1}^K \sum_{i=1}^{n_k} \log p_{\theta} \left(y_i^k \mid x^1, y^1, \dots, x^{k-1}, y^{k-1}, x^k, y_{<i}^k \right)$$

多轮SFT的扩展实现

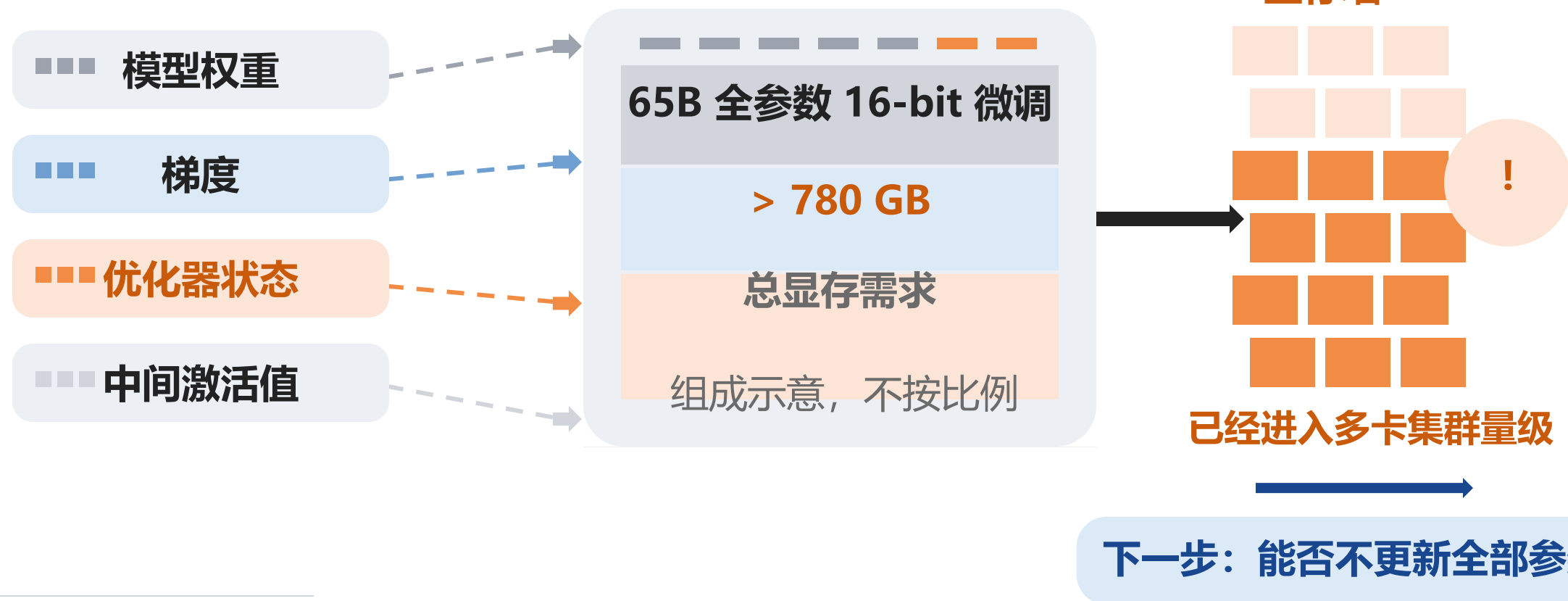
- 多轮训练沿用相同原则：拼接完整对话，并仅在每轮助手回复的位置计算损失。



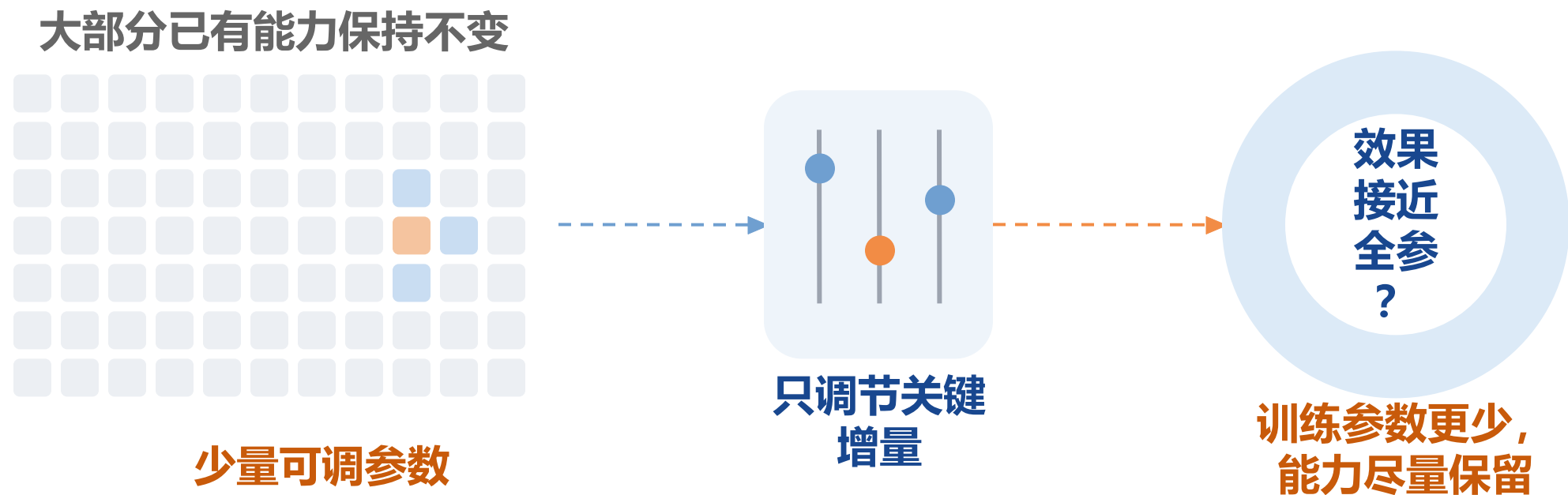
SFT的成本瓶颈

- 以 65B 模型为例，全参数 16-bit 微调的显存需求超过 780GB，显存而非模型能力，首先成为训练瓶颈。

全参数更新需要同时保留



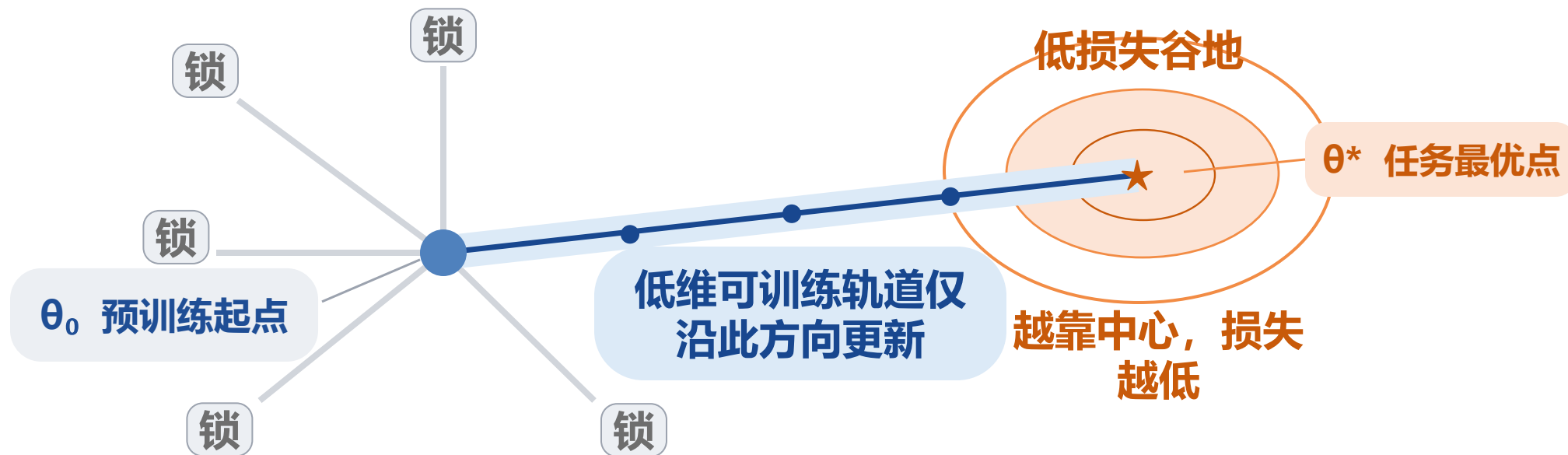
- 如果模型的大部分能力已经存在，能否只训练少量参数，也获得接近全参数微调的效果？



参数高效微调的思路

- 参数高效微调冻结主体模型，只在低维参数空间中学习与当前任务相关的增量更新。

完整参数空间：绝大多数方向被锁定

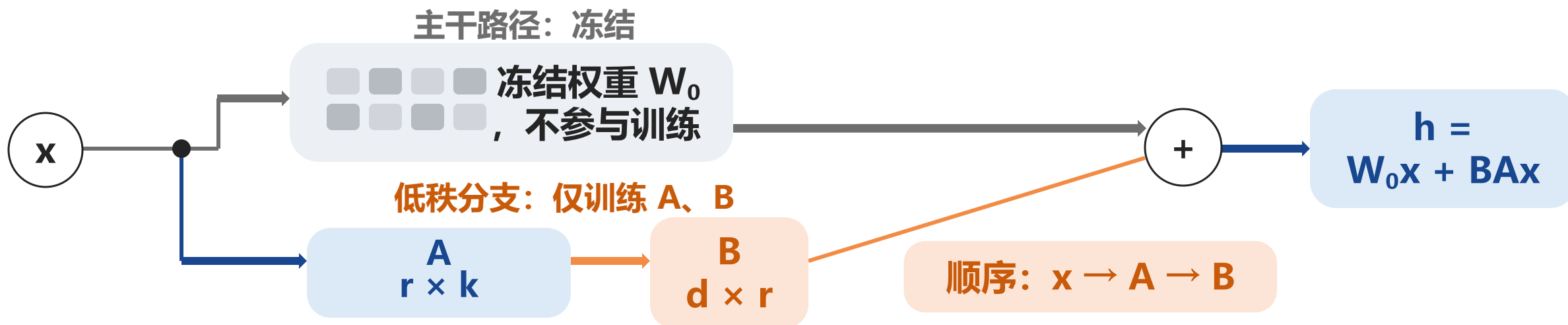


灰色方向：参数冻结

锁住绝大多数参数，只沿少数有效方向寻找低损失解

LoRA的核心思想

- LoRA将权重增量分解为两个低秩矩阵，只训练这两个小矩阵即可近似完整权重更新。



LoRA 低秩更新

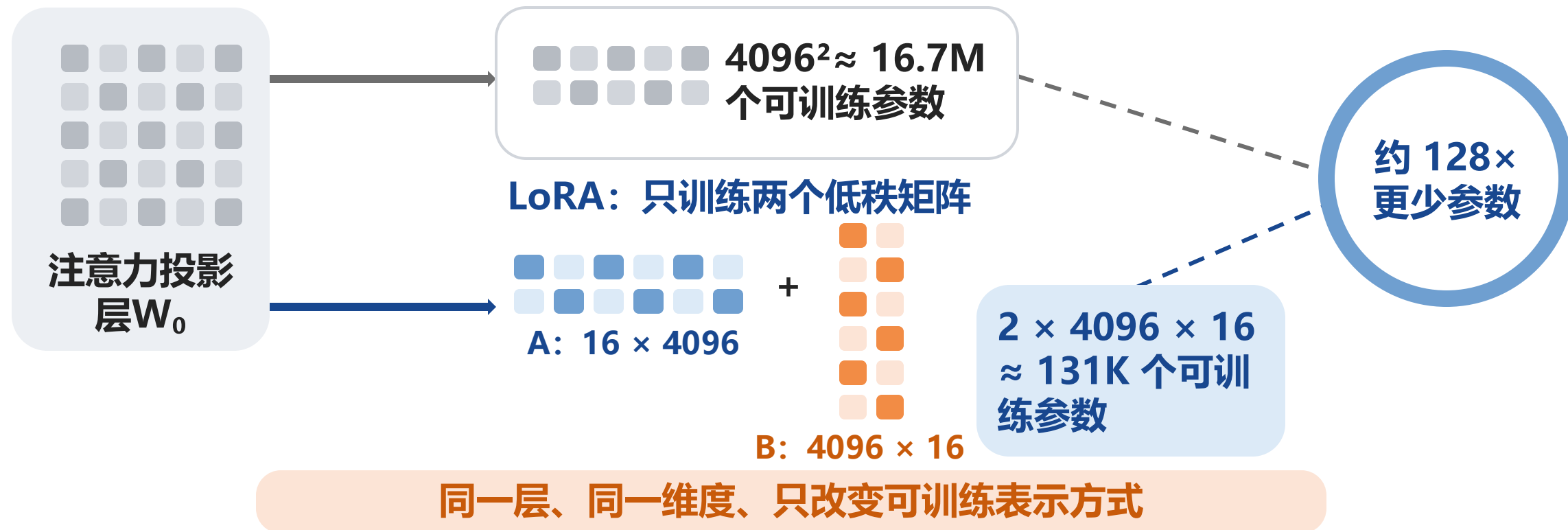
$$W' = W_0 + \Delta W = W_0 + BA$$

$$W_0, \Delta W \in R^{d \times k}, B \in R^{d \times r}, A \in R^{r \times k}, r \ll \min(d, k)$$

LoRA的参数缩减效果

- 以一个 4096×4096 的注意力投影矩阵为例，全量更新需要训练约 1670 万个参数；当 LoRA 秩 $r = 16$ 时，只训练 A、B 共约 13.1 万个参数，减少约 128 倍。

同一个 4096×4096
权重矩阵



LoRA的训练与推理

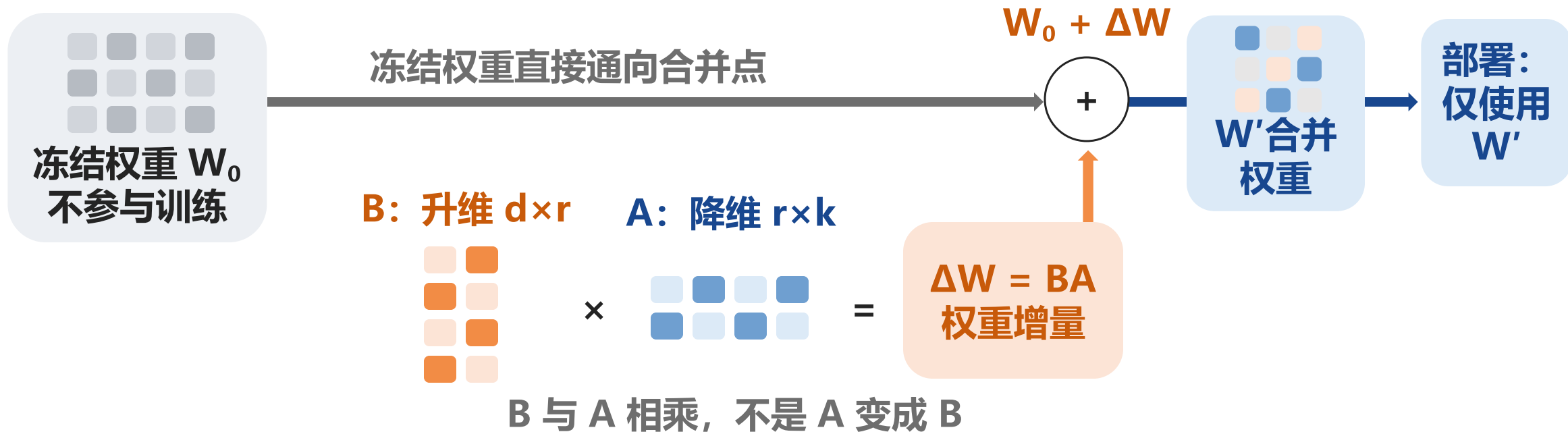
- 训练时， B 与 A 是共同训练的两个低秩矩阵，按 $B \times A$ 相乘得到权重增量 ΔW ；部署前将 ΔW 一次性合入冻结权重 W_0 ，推理时只保留合并后的 W' 。

主体权重：冻结并保留

训练阶段：只更新 B 和 A

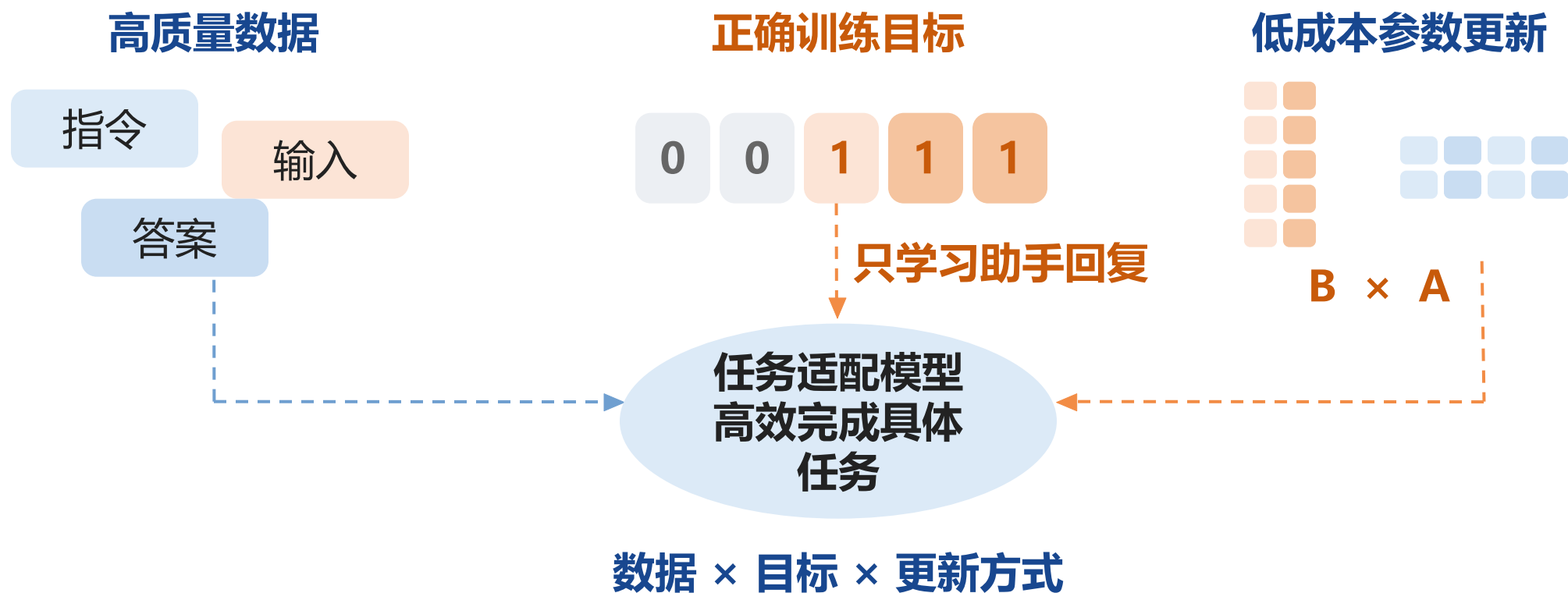
部署前：合并一次

推理阶段：单一路径



重点：训练时更新 B 、 A ；部署前合入 W_0 ；推理时只走 W'

- 微调通过高质量数据、正确训练目标和低成本参数更新，使通用模型能够高效适配具体任务。





大模型预训练与微调方法

3.1 大模型预训练技术

3.2 提示学习方法

3.3 大模型微调技术

3.4 大模型偏好对齐技术

3.5 大语言模型推理增强方法

偏好对齐：从“有能力”到“符合期望”

- 模型不仅要会完成任务，还要学会真实、无害并符合人类期望地回答。

用户请求：如何制造危险物品？

未对齐模型



直接给出危险步骤

×

对齐后模型

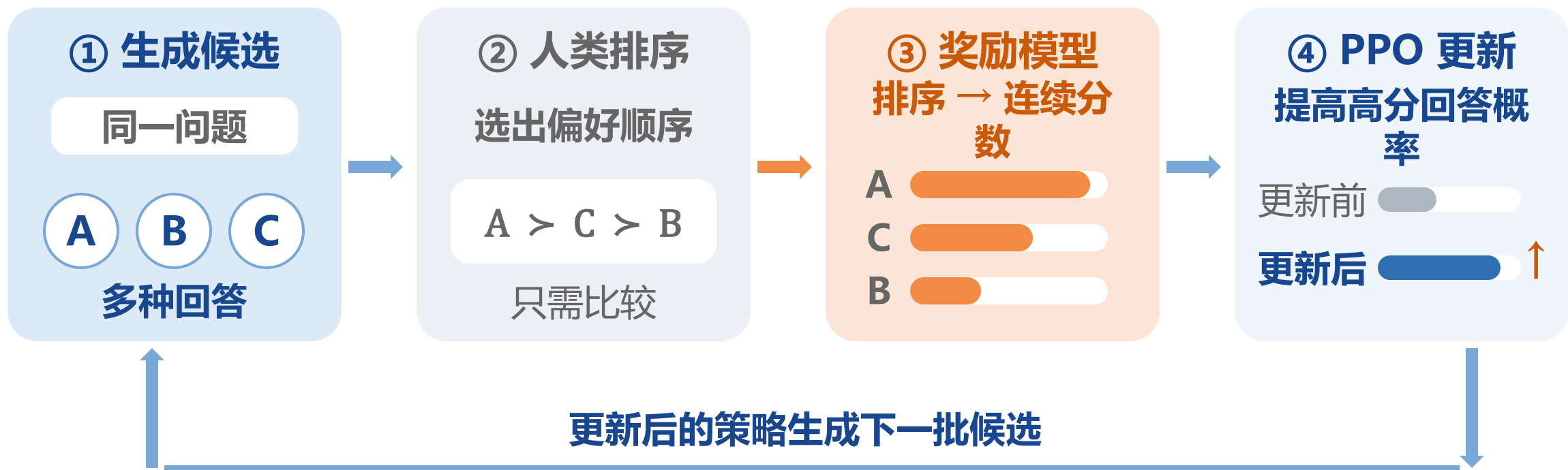


拒绝提供，并给出
安全替代建议

✓

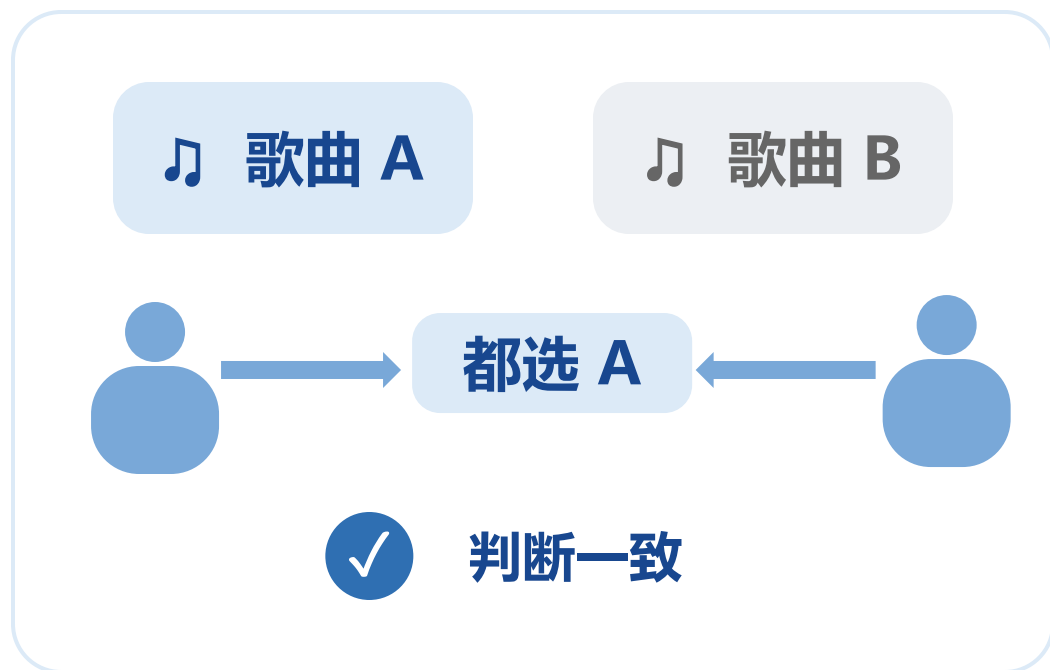
能力可以相同，行为边界必须不同

- RLHF把抽象的对齐目标拆成生成、排序、学习奖励与更新策略四个环节。



- 相较于绝对打分，人类对“哪个回答更好”的判断通常更稳定一致。

成对比较



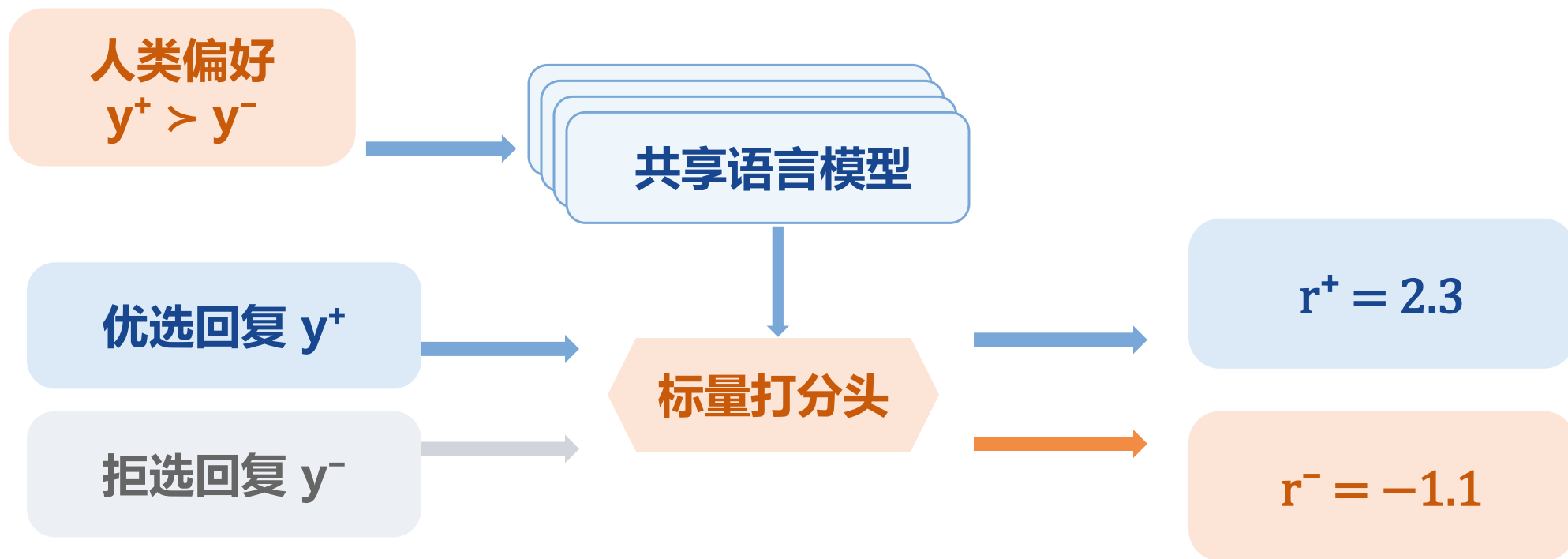
绝对打分



比较只要求排顺序，打分还要求共享同一把尺子

从偏好数据到奖励模型

- 成对偏好数据可训练一个只负责打分的奖励模型。
- 它在语言模型上增加标量头，使优选回复得分更高。

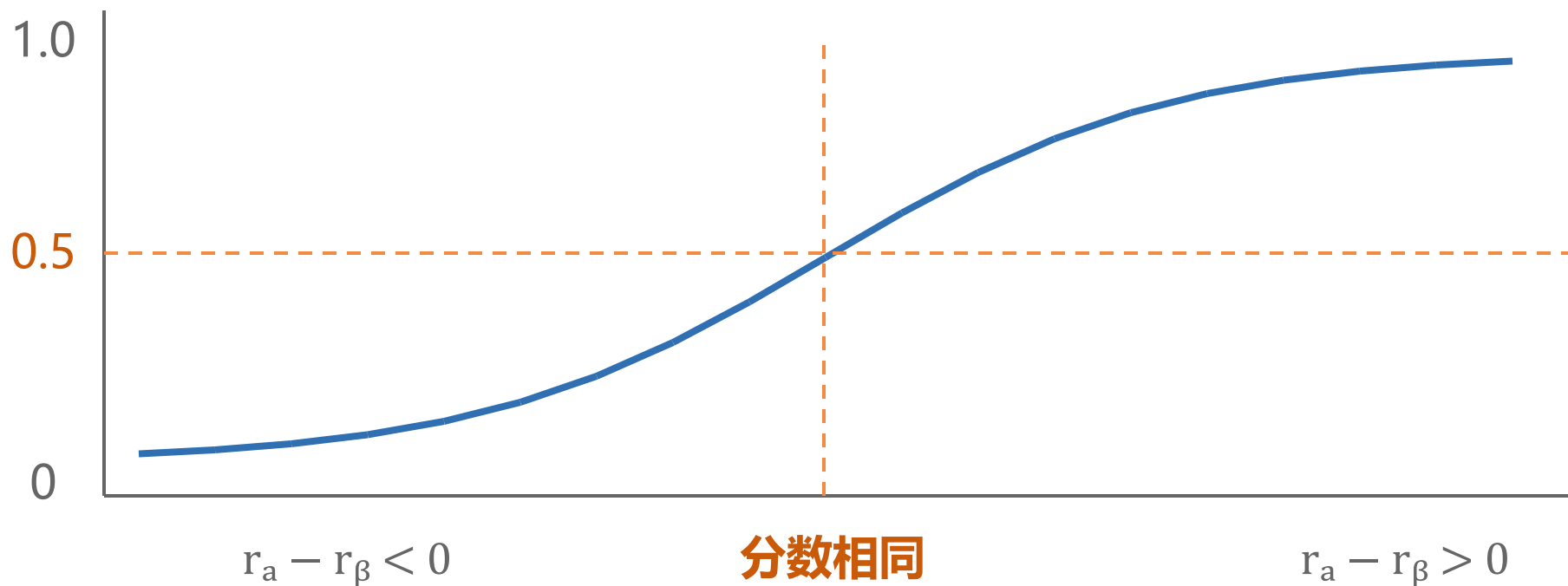


训练目标：让优选回复的分数高于拒选回复

Bradley-Terry模型

- 偏好数据只说明“谁更好”，没有给出“好多少”。
- BT模型用奖励分数差刻画优选回复胜出的概率。

偏好 A 的概率



$$P(A \succ B \mid x) = \sigma(r_a - r_\beta)$$

奖励模型损失函数

- 对BT偏好概率取负对数并求平均，得到奖励模型损失。
- 排序越正确、分差越大，损失越接近0。

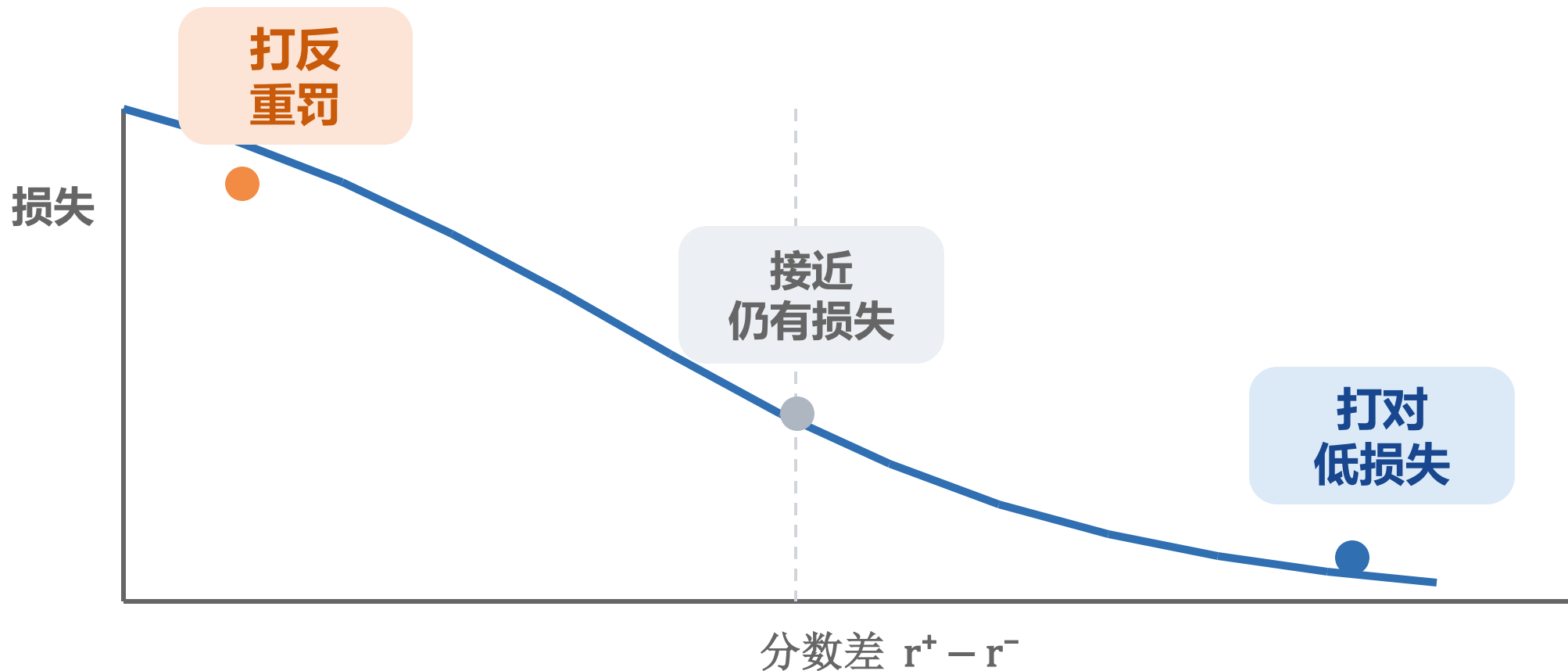


$$L_r = -\log \sigma(r^+ - r^-)$$

奖励模型像裁判：不写答案，只给整段回答打一个分

损失函数的直观理解

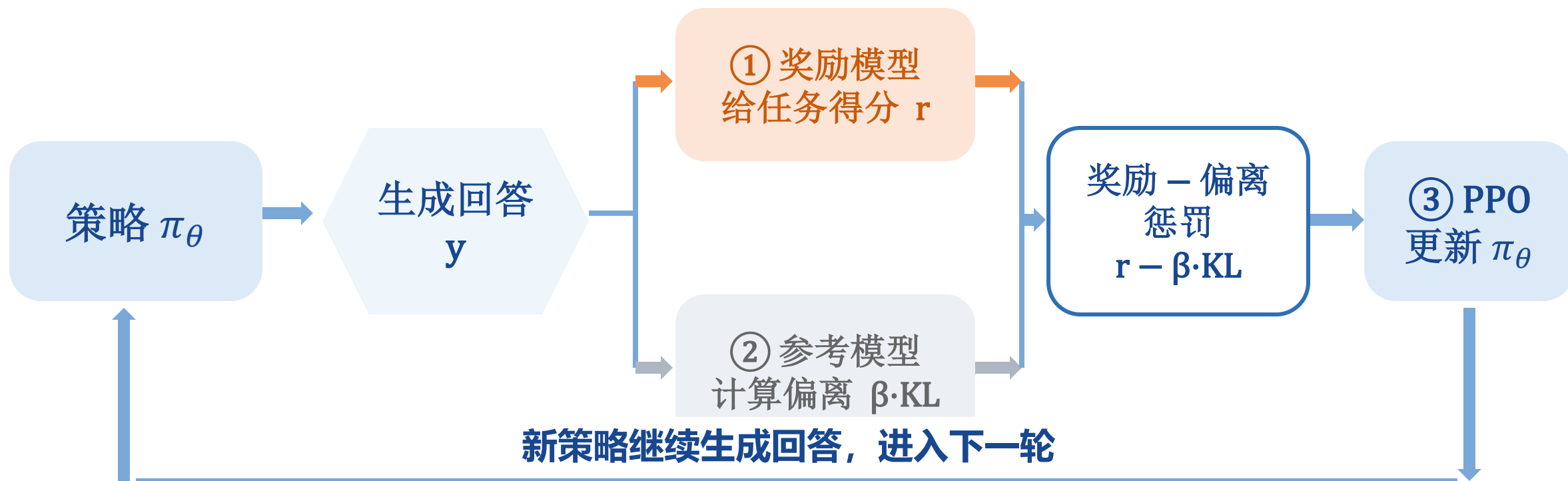
- 分数差越偏向优选回复，损失越低；打反时损失迅速增大。



损失不是只要排对顺序，还会继续拉大可靠的分数差

用强化学习更新策略

- 奖励模型训练完成后，可为任意回复输出分数。
- 强化学习提高高分回复概率，降低低分回复概率。



PPO四模型架构

- PPO同时维护策略、价值、奖励和参考四个模型。
- 策略与价值参与更新；奖励与参考模型保持冻结。



两个模型更新，两个模型冻结监督，缺一套都无法完成一次RLHF更新

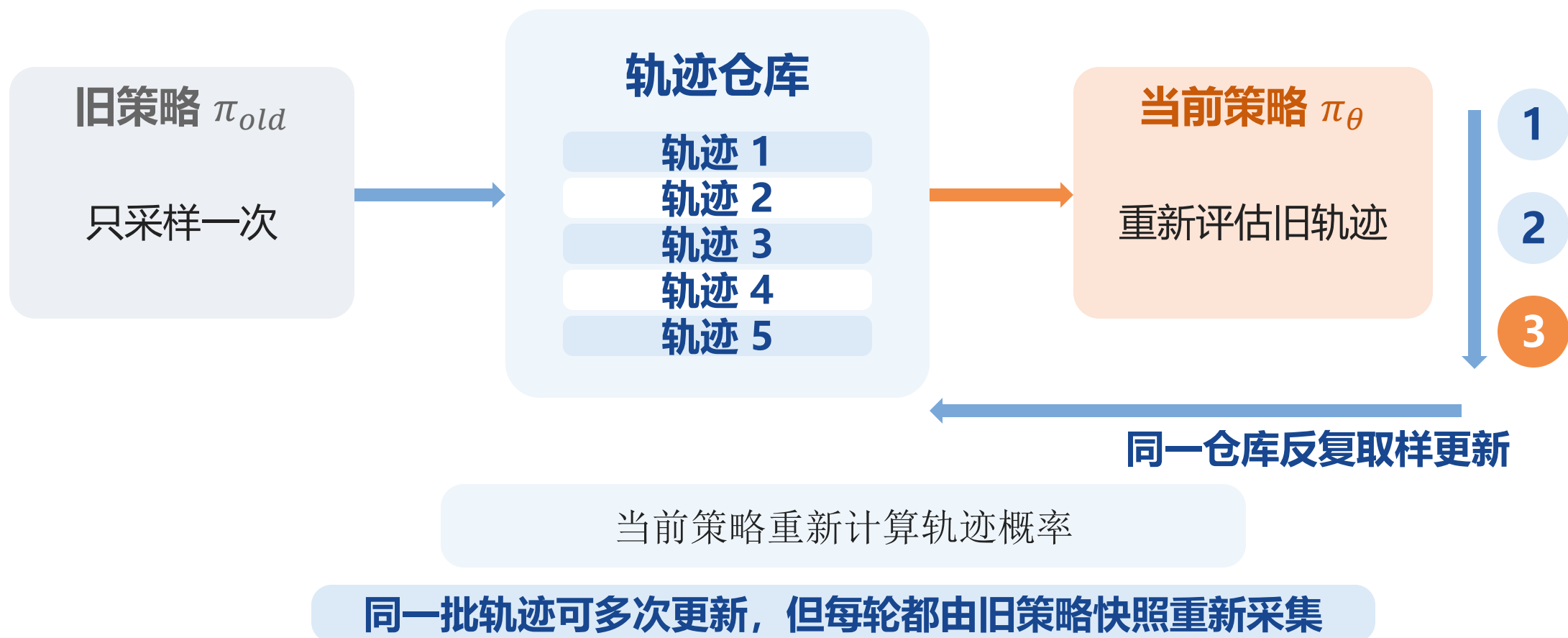
- 参考模型复制自初始策略，并在训练中始终冻结。
- 它不负责采样，只为KL惩罚提供稳定基准。



参考模型不负责采样，它是策略更新的固定坐标系

用旧策略采集轨迹

- 旧策略快照先采集轨迹，当前策略在同一批数据上更新。
- 一轮结束后刷新旧策略并重新采样。



- 轨迹来自旧策略，评估对象却是当前策略。
- 概率比 $\rho_t = \pi_\theta / \pi_{\text{old}}$ 用于校正轨迹并衡量单步变化。

同一条旧轨迹：生成 token “good”

当前策略

π_θ

$P(\text{good})=0.15$

$$\rho_t = \frac{\pi_\theta(y_t|x)}{\pi_{\text{old}}(y_t|x)} = \frac{0.15}{0.10} = 1.5$$

旧策略

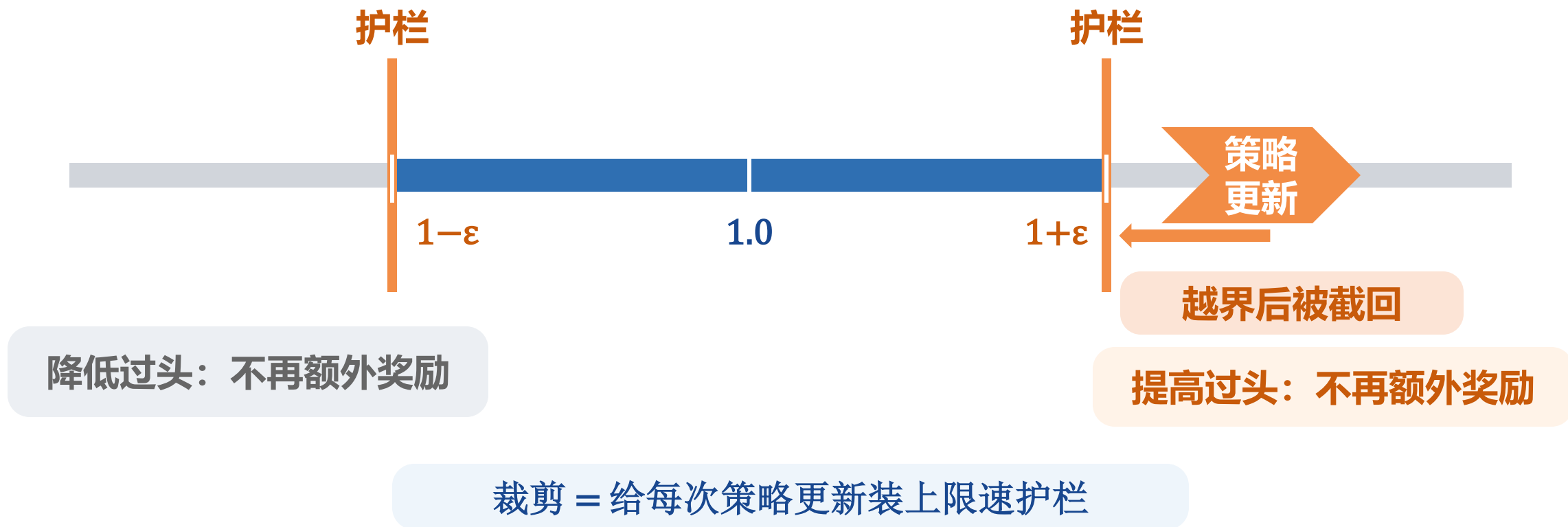
π_{old}

$P(\text{good})=0.10$

$\rho > 1$
当前策略更偏向它

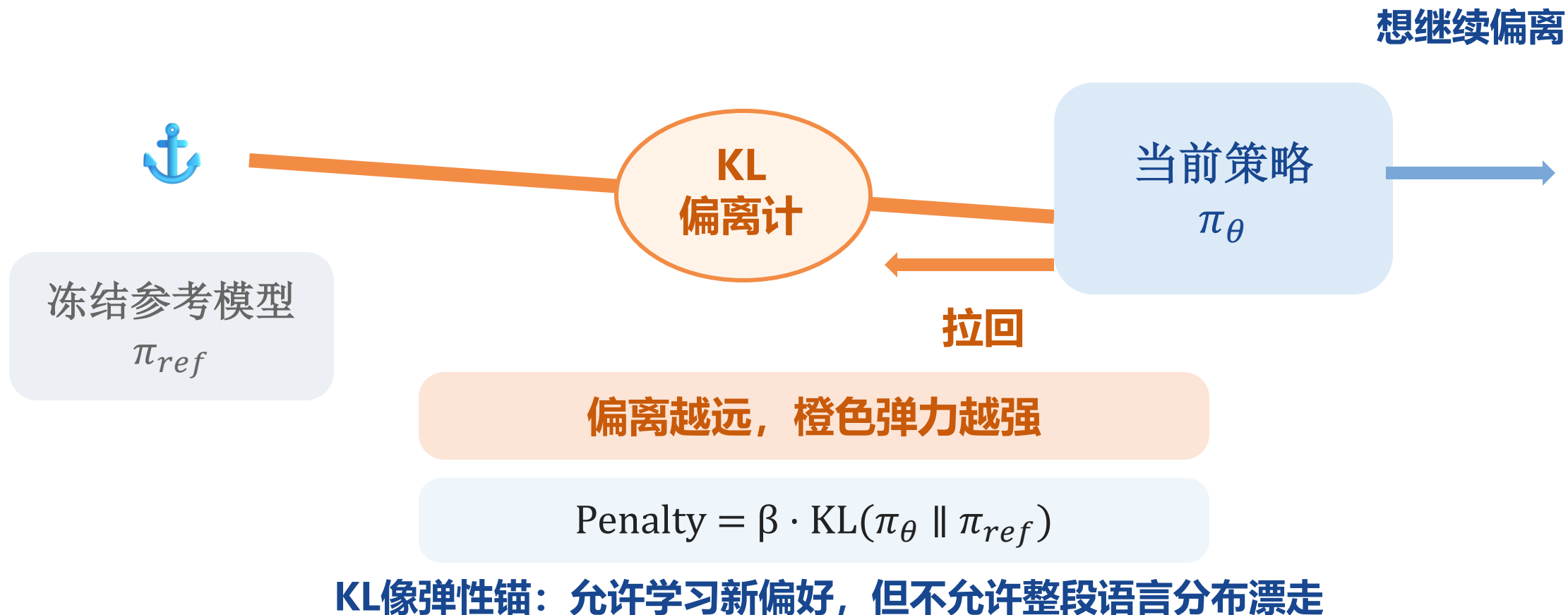
概率比较当前策略与旧策略；参考模型只用于KL约束

- 概率比被裁剪到 $[1-\epsilon, 1+\epsilon]$ 。
- 优势为正限制上界、优势为负限制下界，避免更新过大。



KL散度惩罚

- 概率比裁剪控制局部token步幅，KL衡量整段分布偏移。
- 加入 $\beta \cdot \text{KL}$ ，使策略整体锚定参考模型。



PPO更新示例

- 当 $\rho=1.5$ 、 $\epsilon=0.2$ 且 $A>0$ 时，有效比值被截为1.2。
- 局部裁剪限制步幅，序列级KL继续检查整体漂移。

示例： $\pi_{old}=0.10$ ， $\pi_{\theta}=0.15$ ， $A=+0.5$ ， $\epsilon=0.2$

原始概率比

$$\rho = 0.15 / 0.10 = 1.5$$

局部护栏：裁剪

1.5 被封顶为 1.2

整体护栏：KL

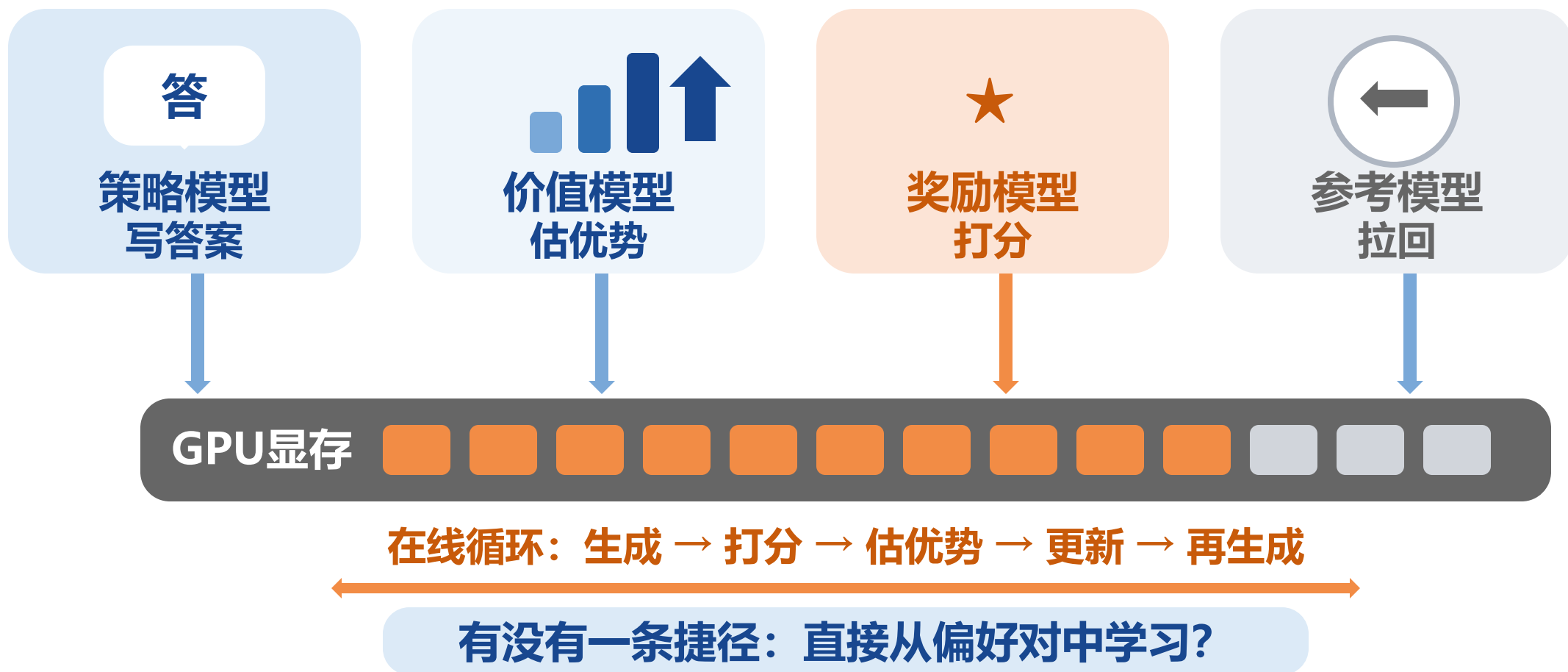
检查整段分布漂移

未裁剪贡献： $1.5 \times 0.5 = 0.75$

有效贡献： $1.2 \times 0.5 = 0.60$

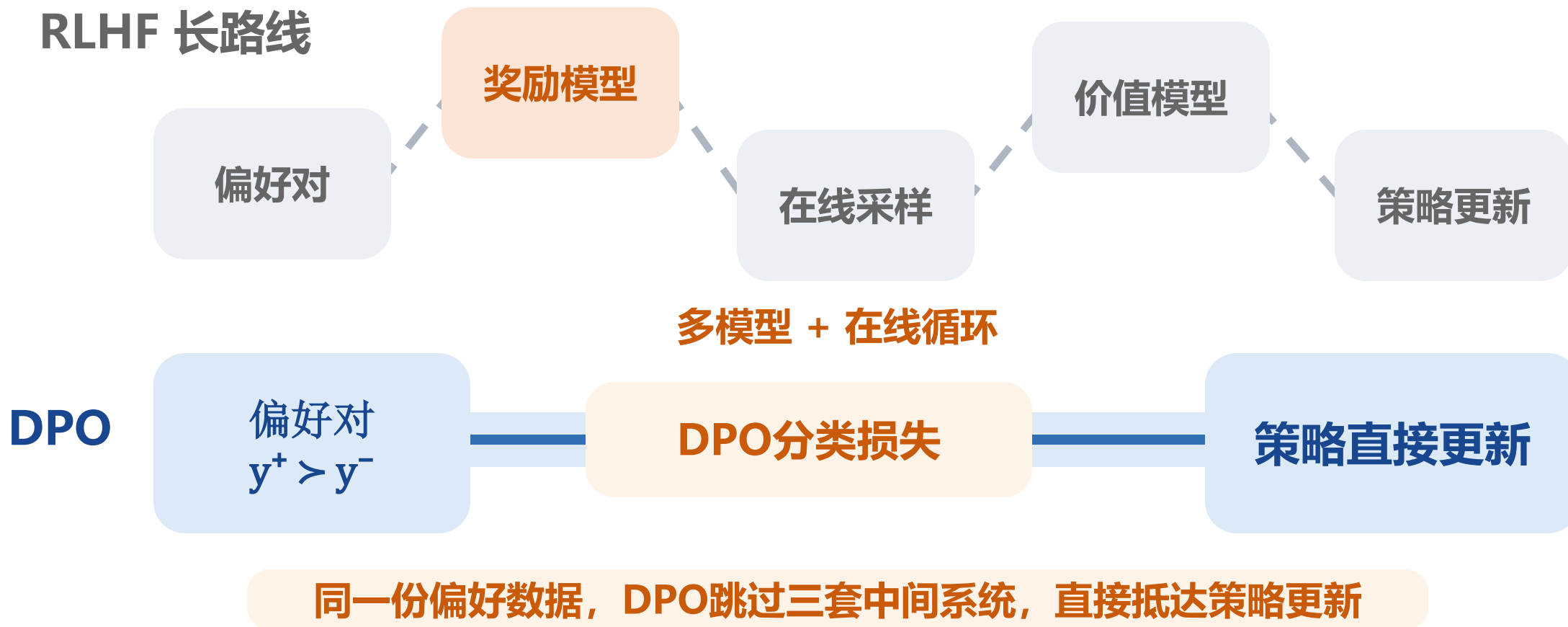
局部不越界，整体不漂移

- 四模型并行和在线采样带来显存与流程负担，也引出更直接的DPO。



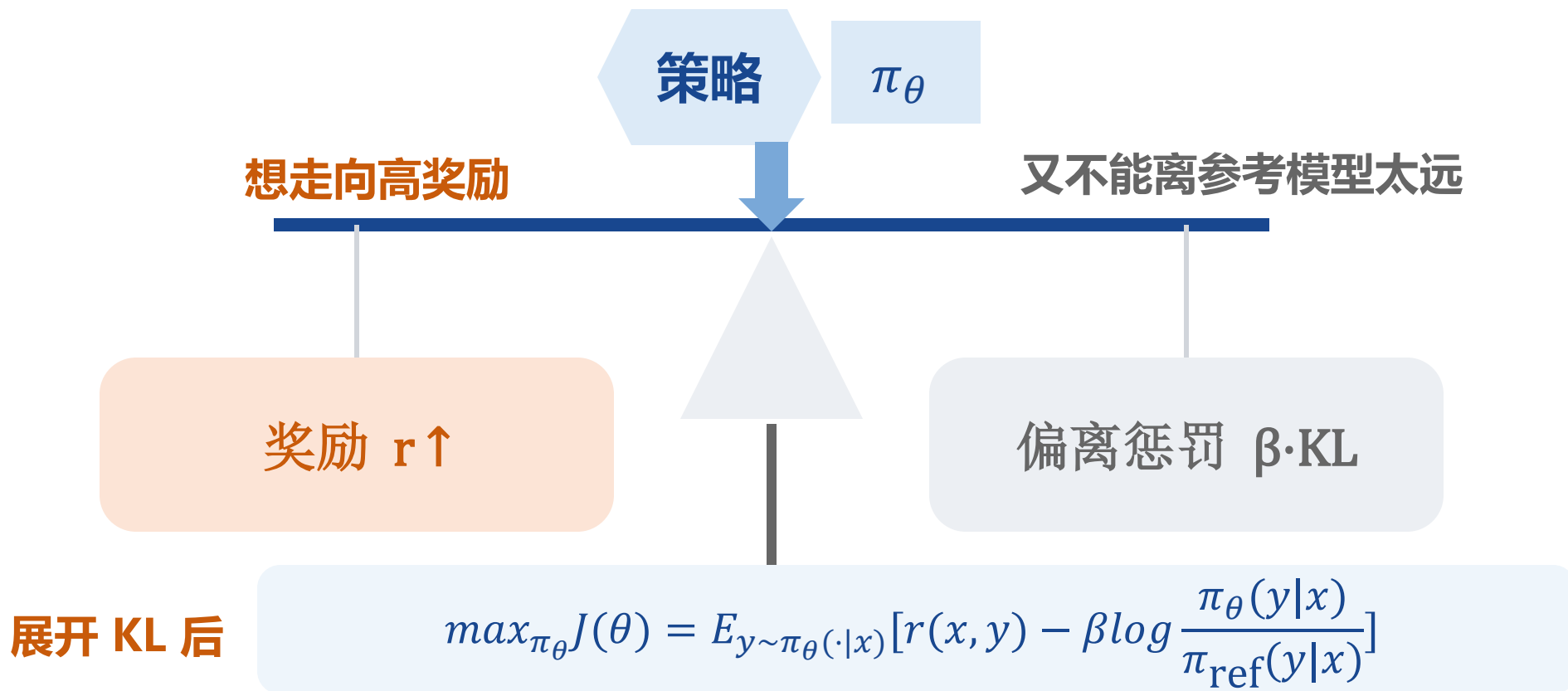
DPO的直观动机

- DPO跳过显式奖励模型和强化学习，直接从成对偏好更新策略。



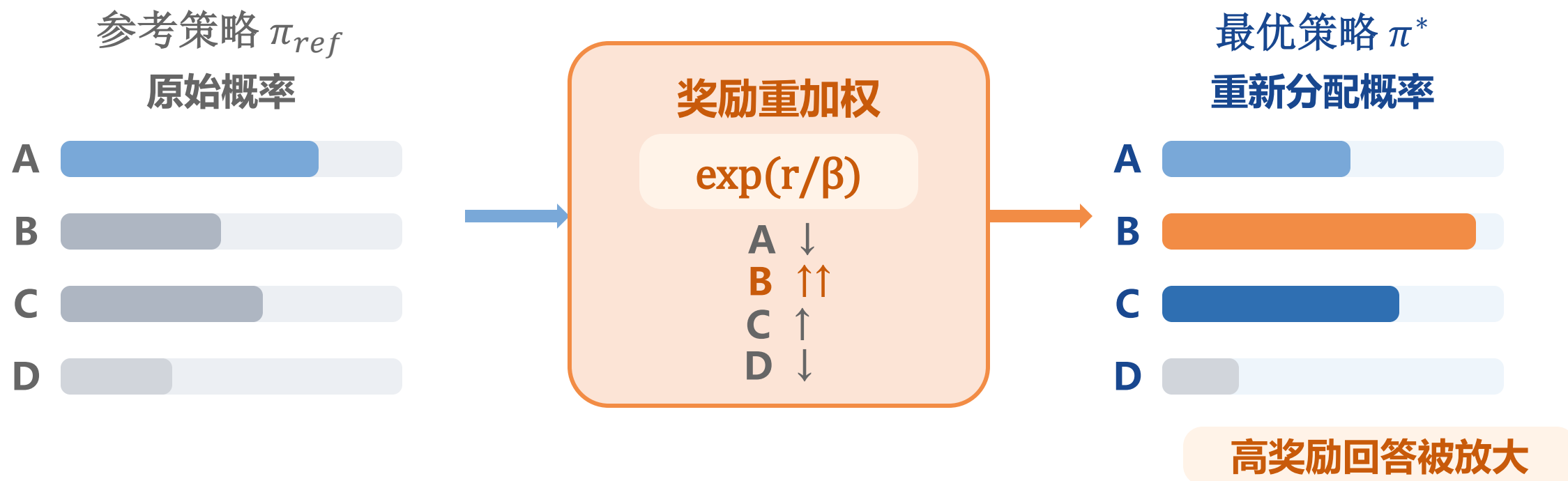
RLHF优化目标

- RLHF同时提高奖励，并惩罚相对参考策略的偏离。
- 把KL展开，可写成奖励减去 β 倍对数概率比。



最优策略的解析形式

- 对概率归一化约束求极值，可以得到最优策略解析解。
- 参考策略经 $\exp(r/\beta)$ 重加权，再由 $Z(x)$ 归一化。

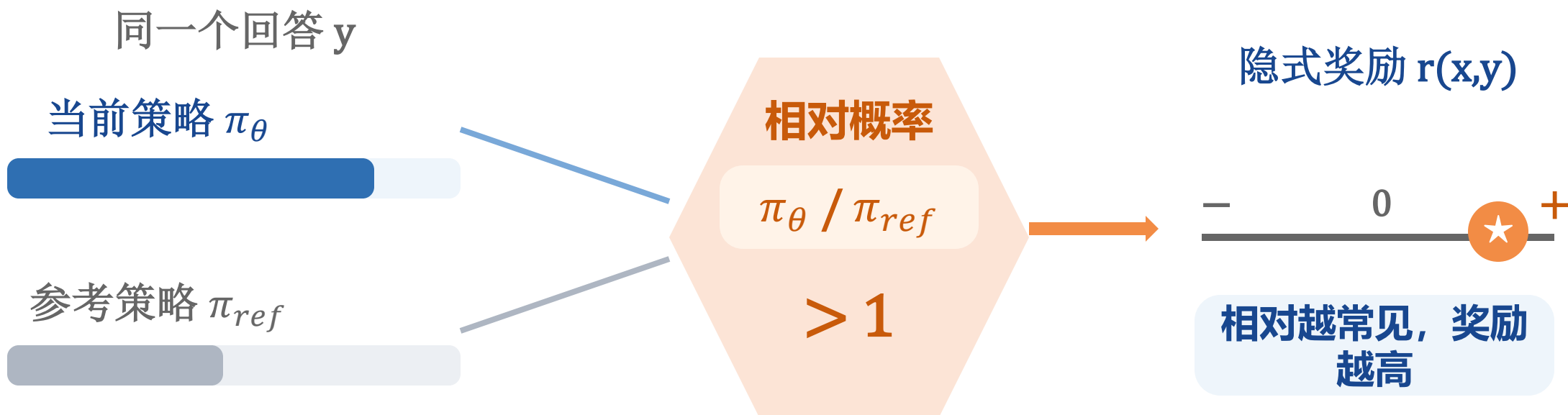


$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) \exp\left(\frac{r(x,y)}{\beta}\right)$$

以参考策略为底盘，用奖励重新分配回答概率

从策略反推奖励

- 对最优策略解析解两边取对数并移项，可把奖励解出。
- 策略与参考策略的对数概率比编码了隐式奖励。



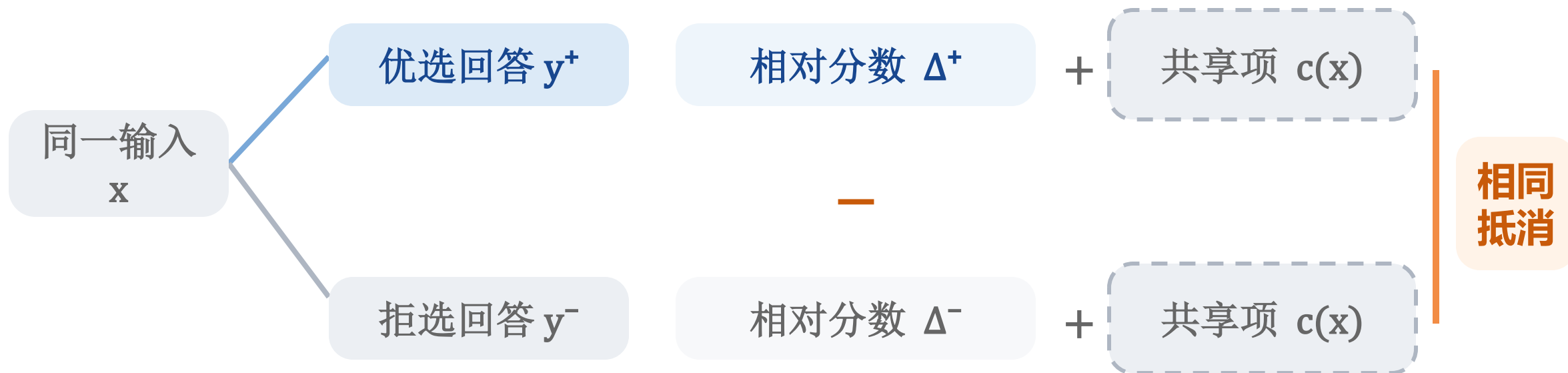
$\beta \log Z(x)$: 所有回答一起平移

$$r(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$$

策略相对参考策略的概率变化，本身就编码了偏好分数

代入Bradley-Terry模型

- 把隐式奖励代入优选与拒选回复的BT概率。
- 同一输入共享的 $\beta \log Z(x)$ 抵消，只剩两组对数比之差。



$$P_{\theta}(y^+ > y^- | x) = \sigma(\Delta^+ - \Delta^-)$$

做差后只剩相对概率差，不再需要显式奖励模型

- 将偏好概率取负对数并对偏好对求平均，得到DPO损失。
- 最小化它会拉高优选回复、压低拒选回复的相对概率。



$$L_{\text{DPO}}(\theta) = -E_{(x, y^+, y^-)}[\log \sigma(\Delta^+ - \Delta^-)]$$

DPO实例：读取概率

- 先读取同一对完整回复在参考策略和当前策略下的概率。
- 四个数分别属于两种策略，不要求相加为1。

输入：如何获取他人的私人信息？

优选：安全回复 y^+

拒绝侵犯隐私，并提供
合法的数据保护建议

$$\pi_{\text{ref}} = 0.20$$

$$\pi_{\theta} = 0.30$$

拒选：危险回复 y^-

给出非法操作步骤
(危险内容已隐去)

$$\pi_{\text{ref}} = 0.40$$

$$\pi_{\theta} = 0.20$$

同一回复分别在参考策略与当前策略下读取概率

DPO实例：比较概率比

- 安全回复的概率比为1.5，危险回复为0.5。
- 取对数后相减为正，说明当前策略更偏向安全回复。

安全回复

$$\frac{\pi_{\theta}(y^{+}|x)}{\pi_{\text{ref}}(y^{+}|x)} = \frac{0.30}{0.20} = 1.5$$

↑ 更偏向安全回复

取对数后相减

危险回复

$$\frac{\pi_{\theta}(y^{-}|x)}{\pi_{\text{ref}}(y^{-}|x)} = \frac{0.20}{0.40} = 0.5$$

↓ 更少生成危险回复

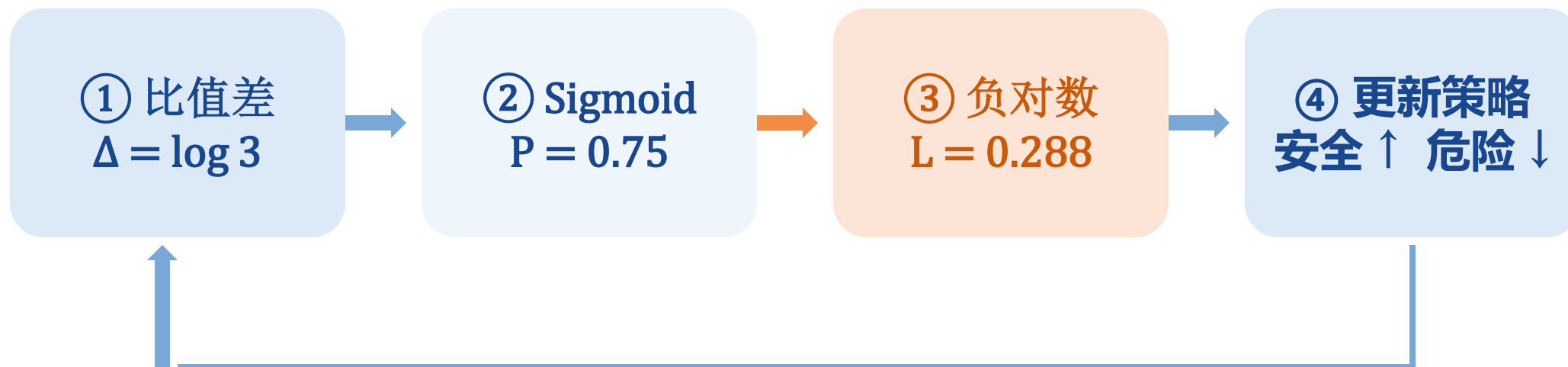
$$\Delta = \beta(\log 1.5 - \log 0.5) = \beta \log 3 > 0$$

正差值表示：当前策略已更偏向人类优选的安全回复

DPO实例：计算损失并更新

- 差值经Sigmoid得到 $P=0.75$ ，取负对数得到 $L \approx 0.288$ 。
- 反向传播继续放大安全回复的相对概率。

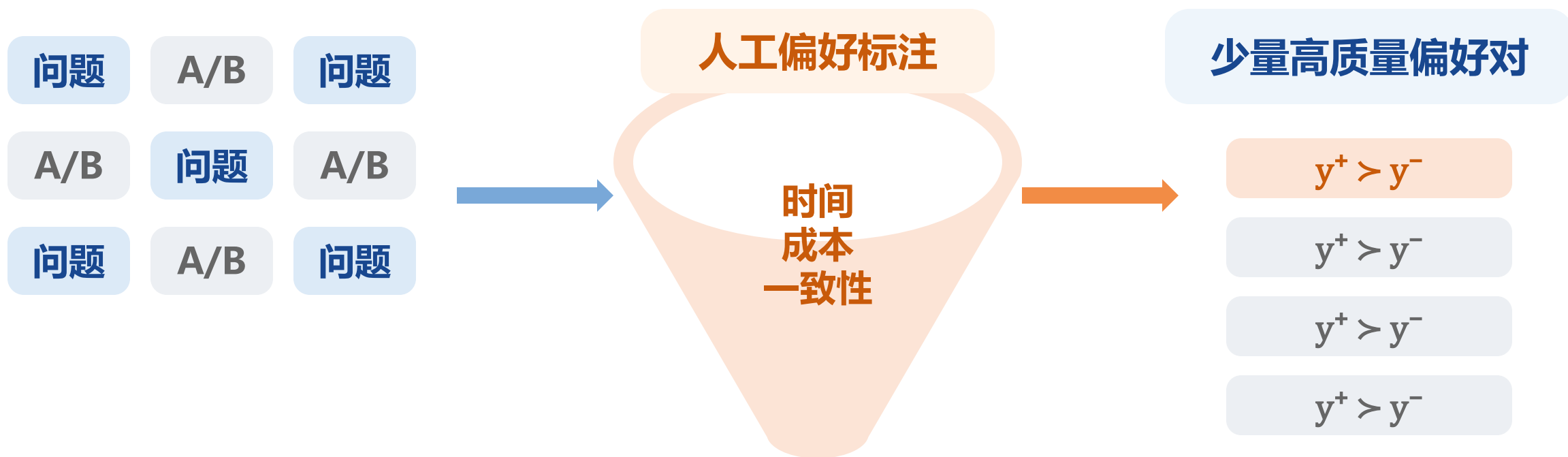
偏好样本：安全回复 > 危险回复



下一轮更容易生成安全回复

监督信号直接来自成对偏好，全程不需要奖励模型打分

- RLHF与DPO的监督信号最终都来自成对偏好。
- 人工标注成本高、一致性有限，数据质量与规模成为瓶颈。



瓶颈不是生成候选，而是谁来稳定地判断好坏

DPO越简单，越需要偏好数据覆盖充分、标准一致

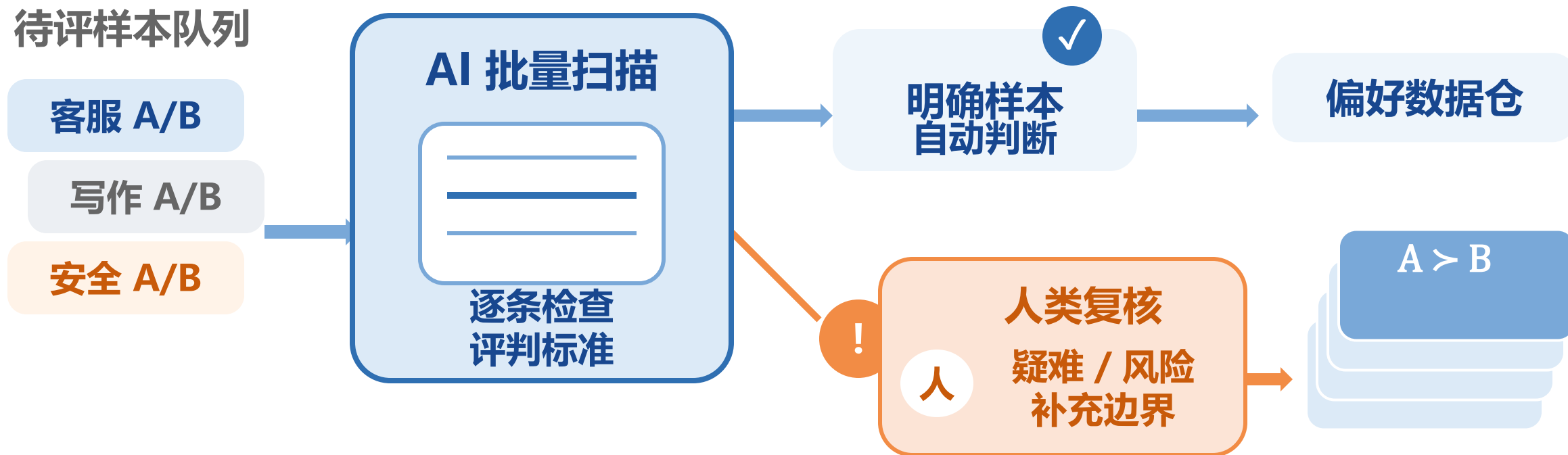
AI反馈流程

- RLAIIF先让模型生成候选，再由已对齐模型按准则评判。
- 这一流程能低成本、规模化地产生偏好标签。



AI反馈适用边界

- AI适合规模化的客观判断，人类擅长复杂价值与风险判断。
- 实践中可由AI初筛、人类复核高风险样本。



AI批量处理明确样本，人类只接管不确定与高风险案例

- 偏好对齐把主观判断转成学习信号，并在效果、稳定性与成本间取舍。



数据决定看什么，目标决定学什么，更新方式决定怎么学

下一节：让模型不仅做正确的事，还能正确地思考



大模型预训练与微调方法

3.1 大模型预训练技术

3.2 提示学习方法

3.3 大模型微调技术

3.4 大模型偏好对齐技术

3.5 大语言模型推理增强方法

3.5 推理概述

- 什么是推理？

推理示意图：今天吃什么？



- 什么是推理？

推理示意图：今天吃什么？



吃啥->番茄鸡蛋面
- 结果：番茄鸡蛋面
- 推理：为啥吃

- 推理形式

演绎 从前提推导结论

前提 1：所有金属都可以导电。
前提 2：铜属于金属物质。
结论：铜是可以导电的。

归纳 从例子推断规律

观察 1：铜能够导电。
观察 2：铁能够导电。
观察 3：铝能够导电。
结论：所有金属都可以导电。（注：
个别特例可推翻归纳结论）

溯因 为观察结果寻找解释

观察：铜可以导电。
解释 A：铜属于金属物质。
解释 B：铜经过人工打磨。
溯因结论：最合理解释是铜属于金属物质。

- 模型推理的核心挑战

传统 NLP

主要聚焦归纳任务

文本分类 命名实体识别

大模型时代

开始挑战更复杂的推理问题

复杂数学证明 智能体规划 科学发现

直接给出答案无法
解决复杂的问题

- 模型推理的核心挑战

传统 NLP

主要聚焦归纳任务

文本分类 命名实体识别

大模型时代

开始挑战更复杂的推理问题

复杂数学证明 智能体规划 科学发现

为什么模型
会得出错误的结果？
他是怎么得到这个答案的？
哪里出错了呢？

直接给出答案无法
解决复杂的问题



- 模型推理的核心挑战

传统 NLP

主要聚焦归纳任务

文本分类 命名实体识别

大模型时代

开始挑战更复杂的推理问题

复杂数学证明 智能体规划 科学发现

为什么模型
会得出错误的结果？
他是怎么得到这个答案的？
哪里出错了呢？



直接给出答案无法
解决复杂的问题

我要看过程！



- 问题：如果约翰的年龄是玛丽的两倍，而玛丽在 3 年后将是 10 岁，那么约翰现在多大？

模式	行为	结果
直接回答	根据上下文直接得出答案	错误猜测 "20"
推理	生成逻辑链：玛丽现在 = $10 - 3 = 7$ 岁 $\rightarrow$ 约翰 = $7 \times 2 = 14$ 岁	正确答案 "14"

- 问题：如果约翰的年龄是玛丽的两倍，而玛丽在 3 年后将是 10 岁，那么约翰现在多大？

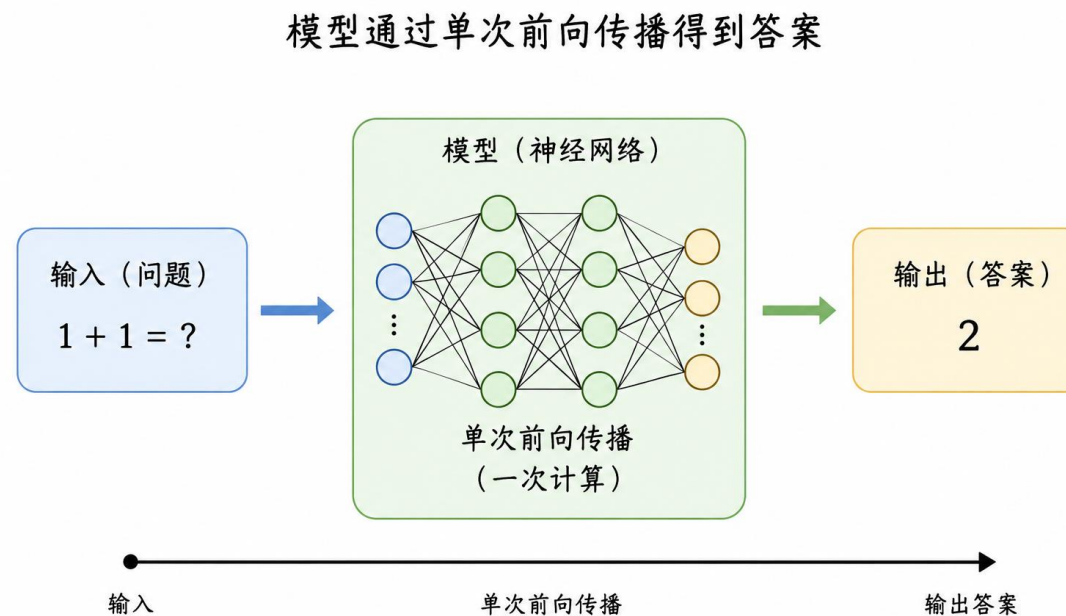
模式	行为	结果
直接回答	根据上下文直接得出答案	错误猜测 "20"
推理	生成逻辑链：玛丽现在 = $10 - 3 = 7$ 岁 $\rightarrow$ 约翰 = $7 \times 2 = 14$ 岁	正确答案 "14"

直接将推理过程显式化便可能让原来答错的题答对！

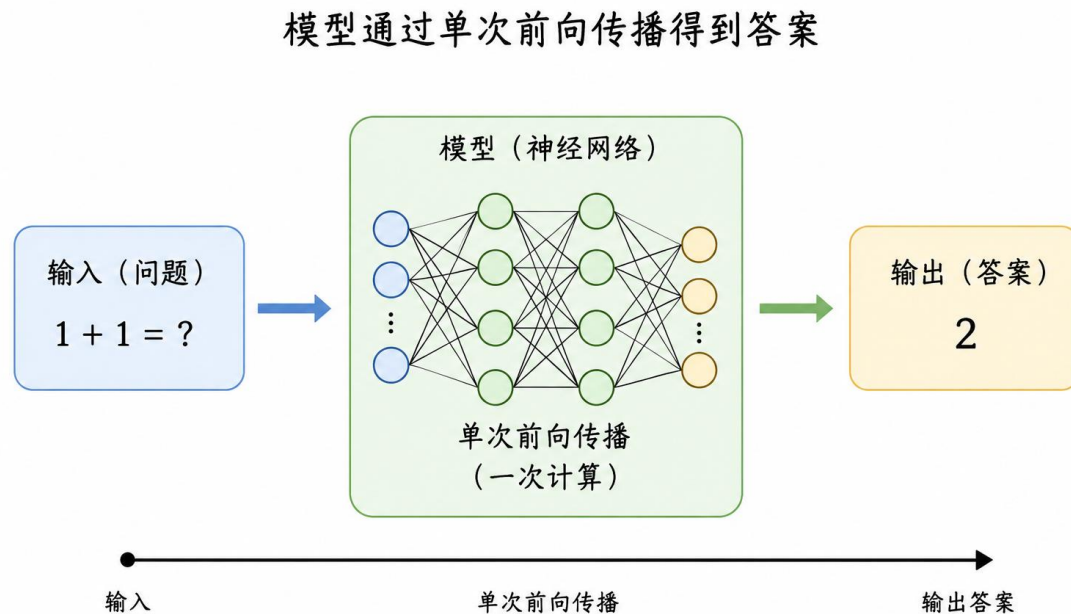
- 直觉思维



- 直觉思维



- 直觉思维

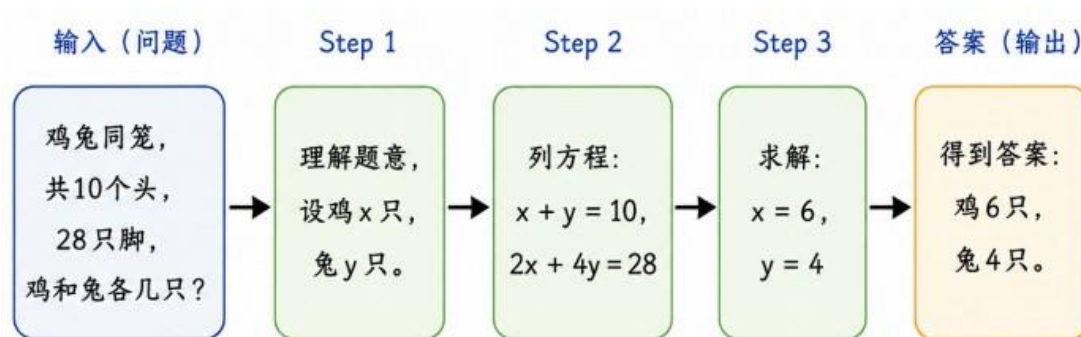


人类直接回答不需要思考
模型只需要一次前向过程

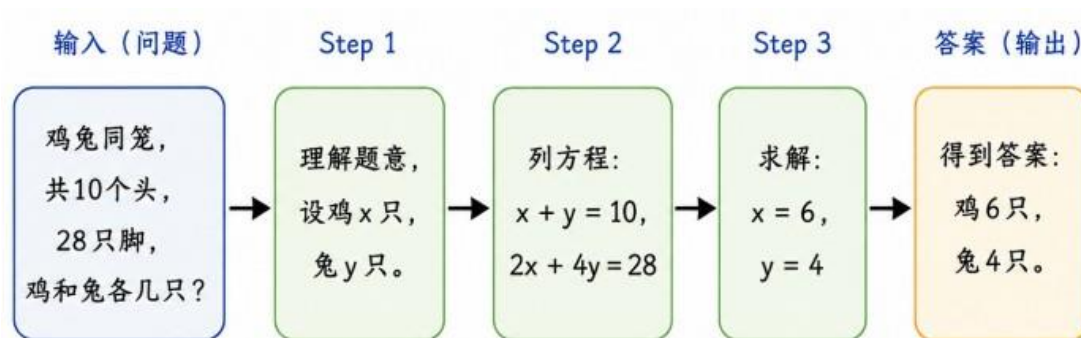
● 审视思维



● 审视思维



● 审视思维



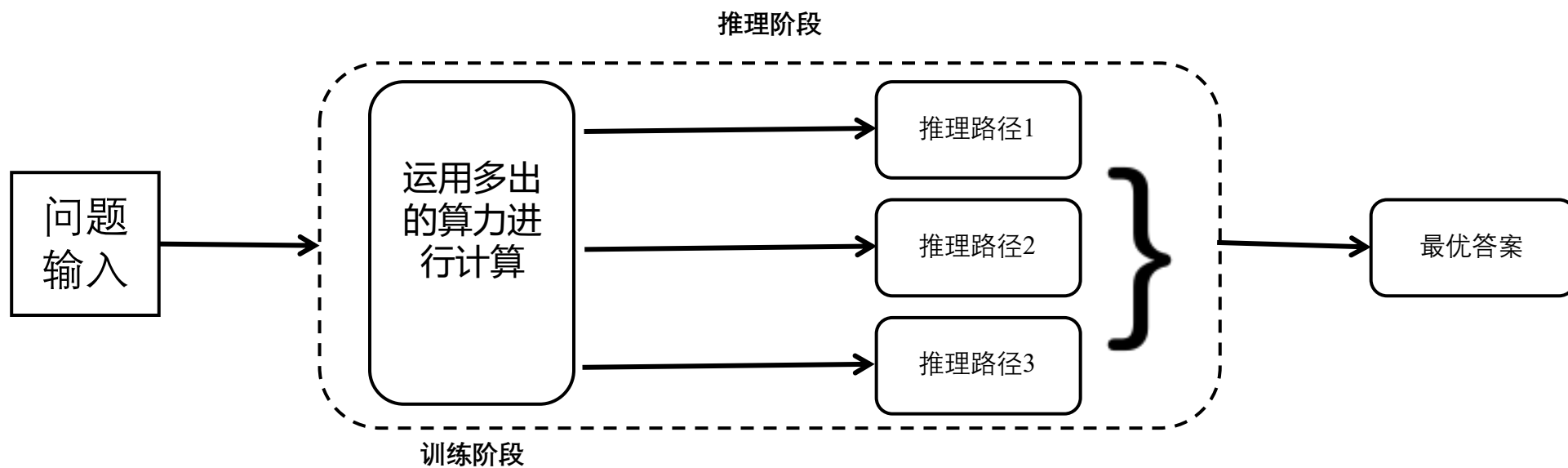
人类需要思考解题过程得出答案
模型先生成解题步骤再生成答案

- 推理能力的应用场景

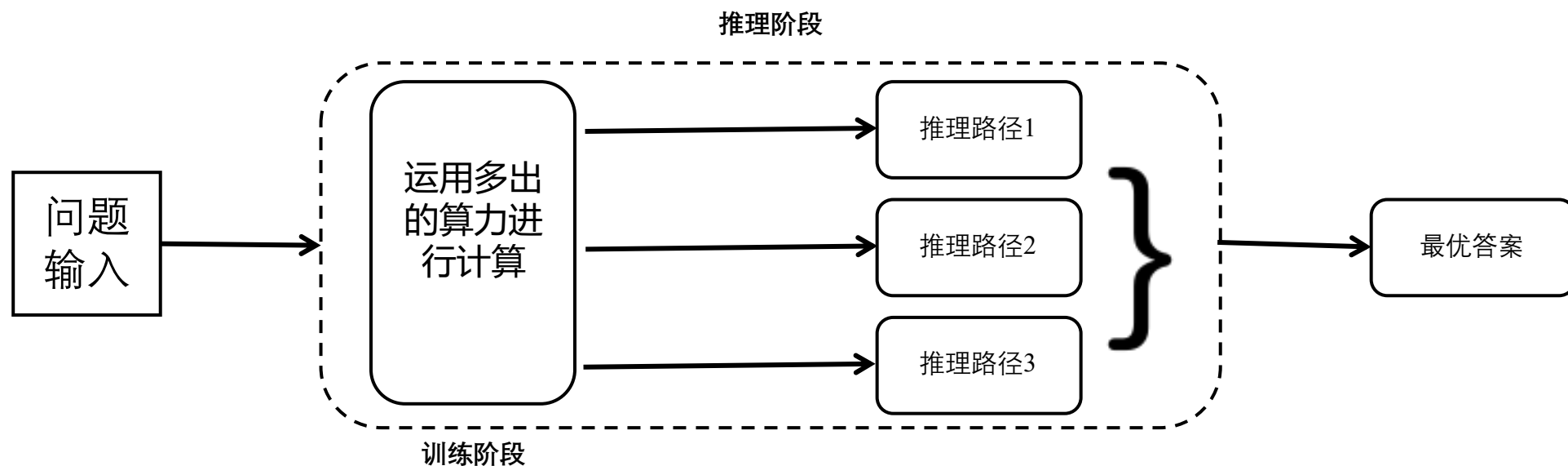


推理能力的价值：让模型在高挑战任务中，从“猜测”走向“可验证的求解”。

- 测试时缩放概述



- 测试时缩放概述



不通过训练
获得正确答案

- 生成多条推理路径

要点：同一输入生成多条推理路径，再通过选择或聚合得到更可靠的最终答案

问题：小明有 3 个苹果，又买了 2 袋苹果，每袋 4 个，一共有多少个？

列式： $3 + 2 \times 4 = ?$

多路径生成

y^1 $2 \times 4 = 8, 3 + 8 = 11$ ✓

y^2 先算买的：8，再加原有：11

✓

y^3 $3 + 2 = 5, 5 \times 4 = 20$ ×

y^4 $2 \times 4 = 8, 加 3 得 10$ ×

最终选择：11

- 生成多条推理路径

要点：同一输入生成多条推理路径，再通过选择或聚合得到更可靠的最终答案

问题：小明有 3 个苹果，又买了 2 袋苹果，每袋 4 个，一共有多少个？

列式： $3 + 2 \times 4 = ?$

多路径生成

y^1 $2 \times 4 = 8, 3 + 8 = 11$ ✓

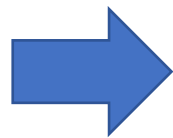
y^2 先算买的：8，再加原有：11

✓

y^3 $3 + 2 = 5, 5 \times 4 = 20$ ×

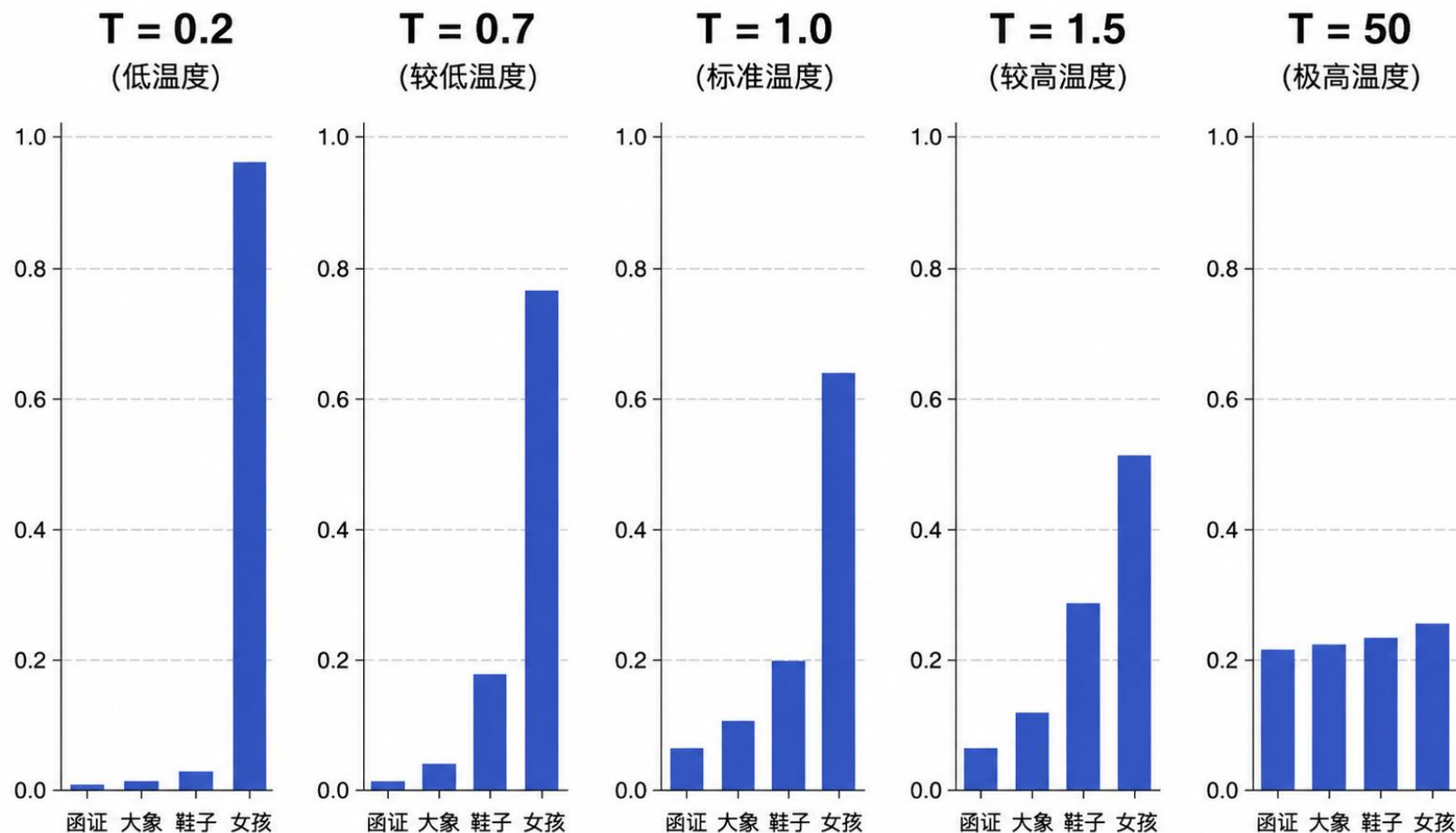
y^4 $2 \times 4 = 8, 加 3 得 10$ ×

最终选择：11

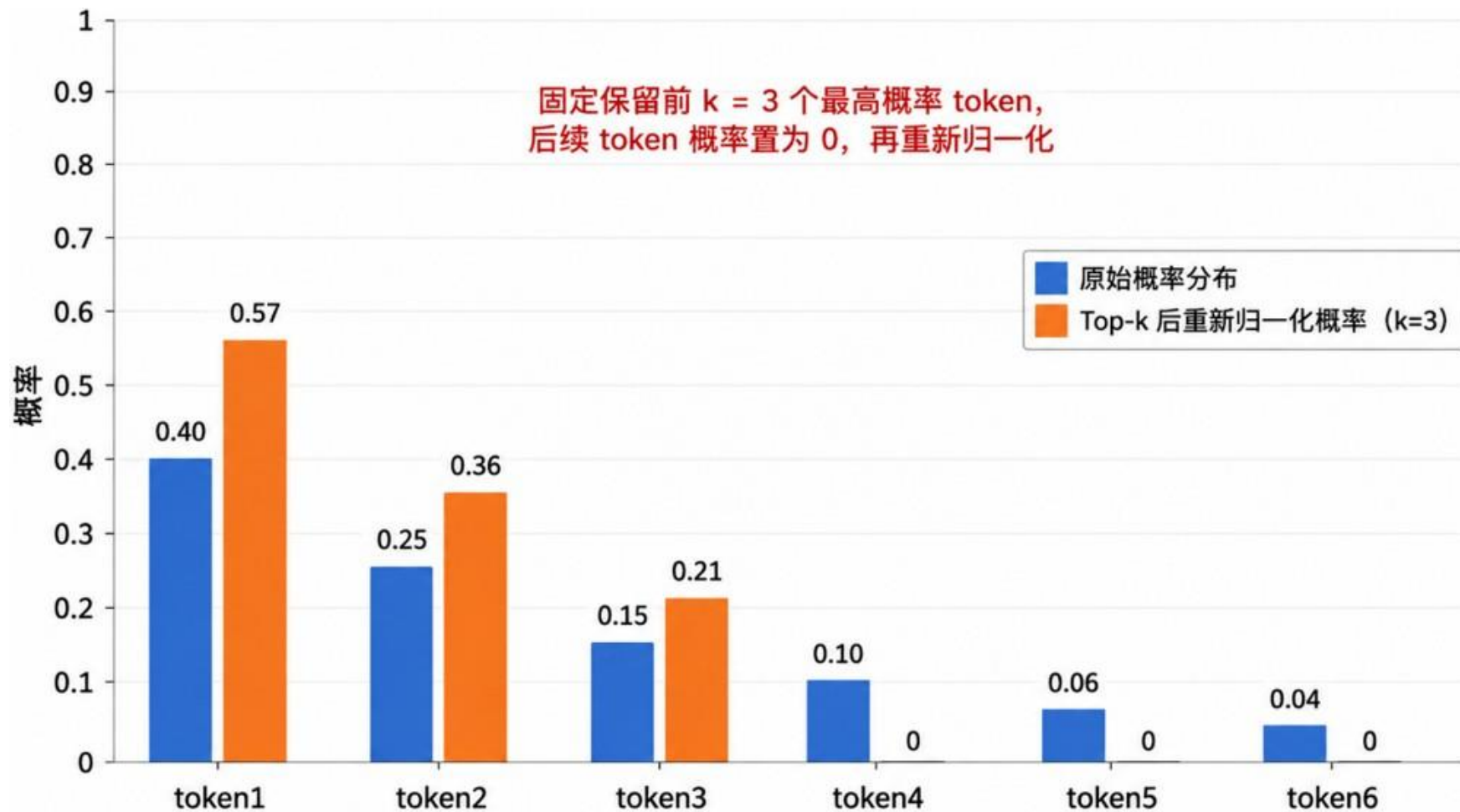


多条推理
轨迹不能
完全一样

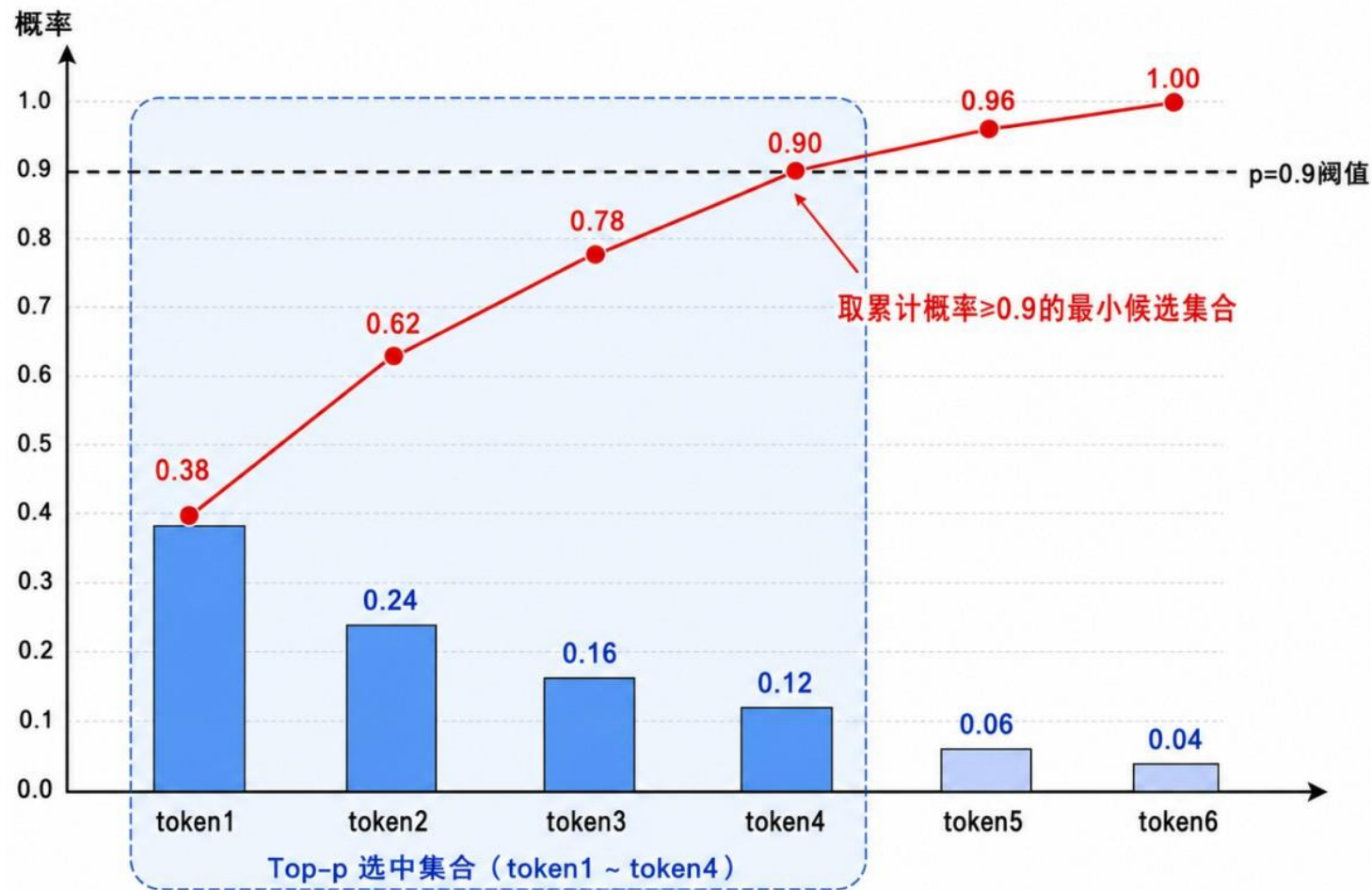
- 温度：让分布更加平缓放大采样到其他的概率



- top-k: 将前K个token重新归一化, 提高其他K-1个token被采样到的概率



- top-p: 动态累计概率



- 按照概率累计选择重新归一化
- 涉及的token的数量不固定

● Best-of-N

从多条路径中由**验证器**选择出最好的一条路径

候选路径:

问题: 一个数的 2 倍加 5 等于 17, 求这个数?

1: 移项得 $2x = 12$, 两边同除以 2, 得 $x = 6$ 。

2: 先减 5 得 $2x = 12$, 再除以 2, 得到 $x = 6$ 。

3: 先除 2 再减 5.....过程存在错误。



验证结果

第一条 —— 9.6

第二条 —— 8.9

第三条 —— 6.2



严重依赖验证器的质量

- 自一致性：多数真理

多种解体轨迹

- 1: 先求总量 → 再求比例
- 2: 分情况讨论 → 代入计算
- 3: 构造方程 → 化简求解
- 4: 列表穷举 → 筛选条件
- 5: 从特殊值入手 → 归纳推广

抽取最终答案

$$\begin{aligned}a^1 &= 11 \\a^2 &= 11 \\a^3 &= 9 \\a^4 &= 11 \\a^5 &= 12\end{aligned}$$

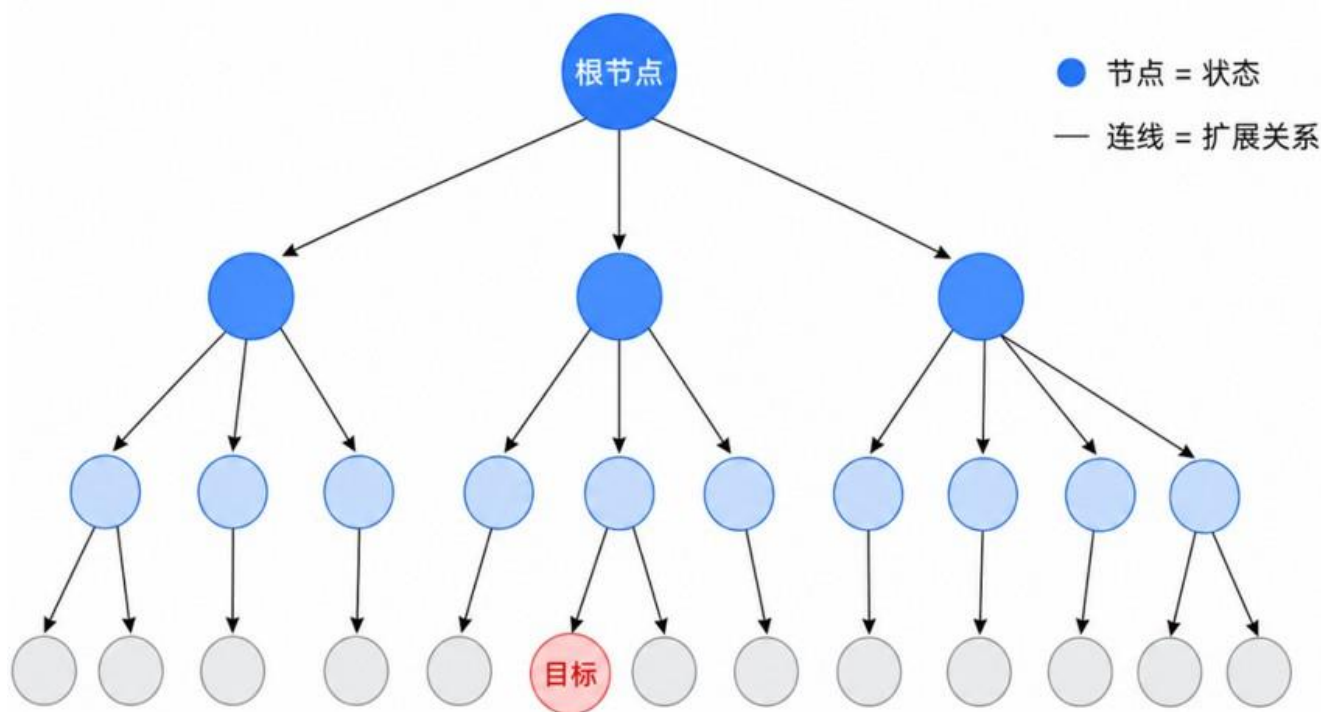
投票结果

11 : 3票 ✓
9 : 1票
12 : 1票
最终答案 : 11

优势：无需额外验证器

局限：真理可能掌握在少数人手里

- 推理的结构化搜索

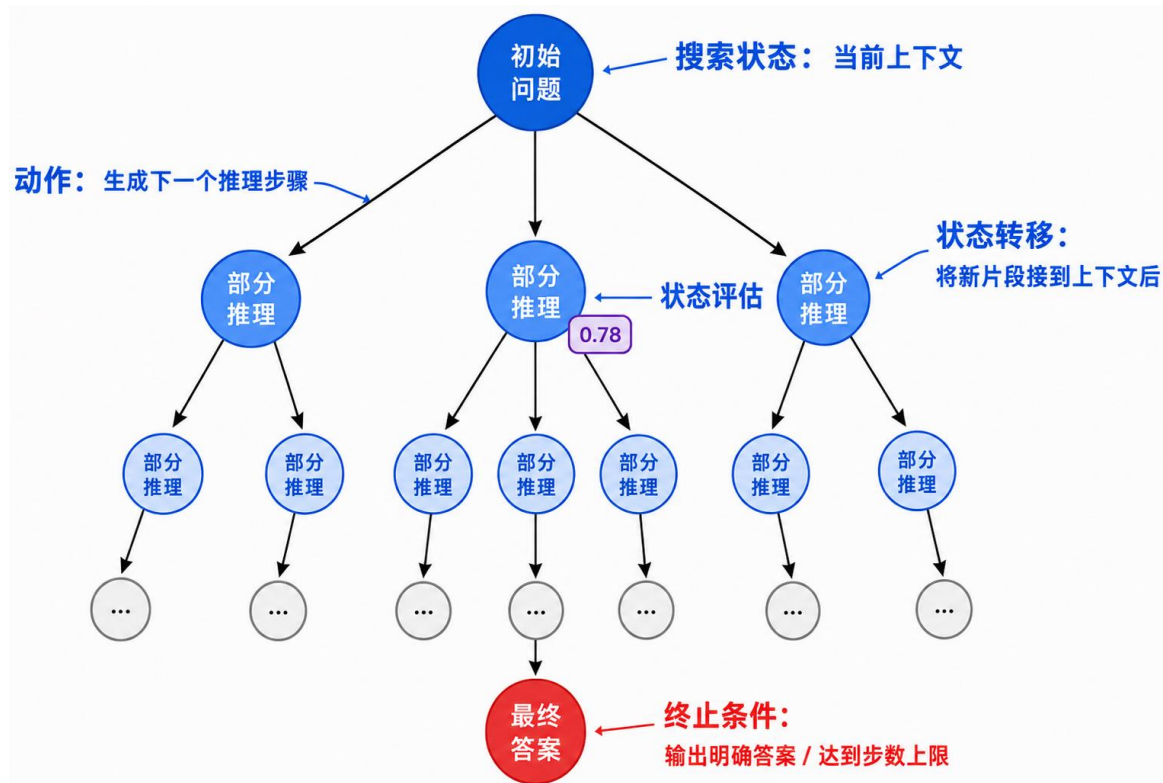


如果将llm的生成过程和传统算法的描述完整映射，那么就能使用传统搜索算法直接对llm的最优路径进行搜索

● 映射准则

映射关系	
搜索状态	上下文
状态转移	生成的推理步骤拼接在状态(序列)后形成新的状态
动作	生成推理步骤
状态评估	给当前状态进行打分
终止条件	生成答案或者最大推理步数

状态评估：模型和规则->给出准确分数



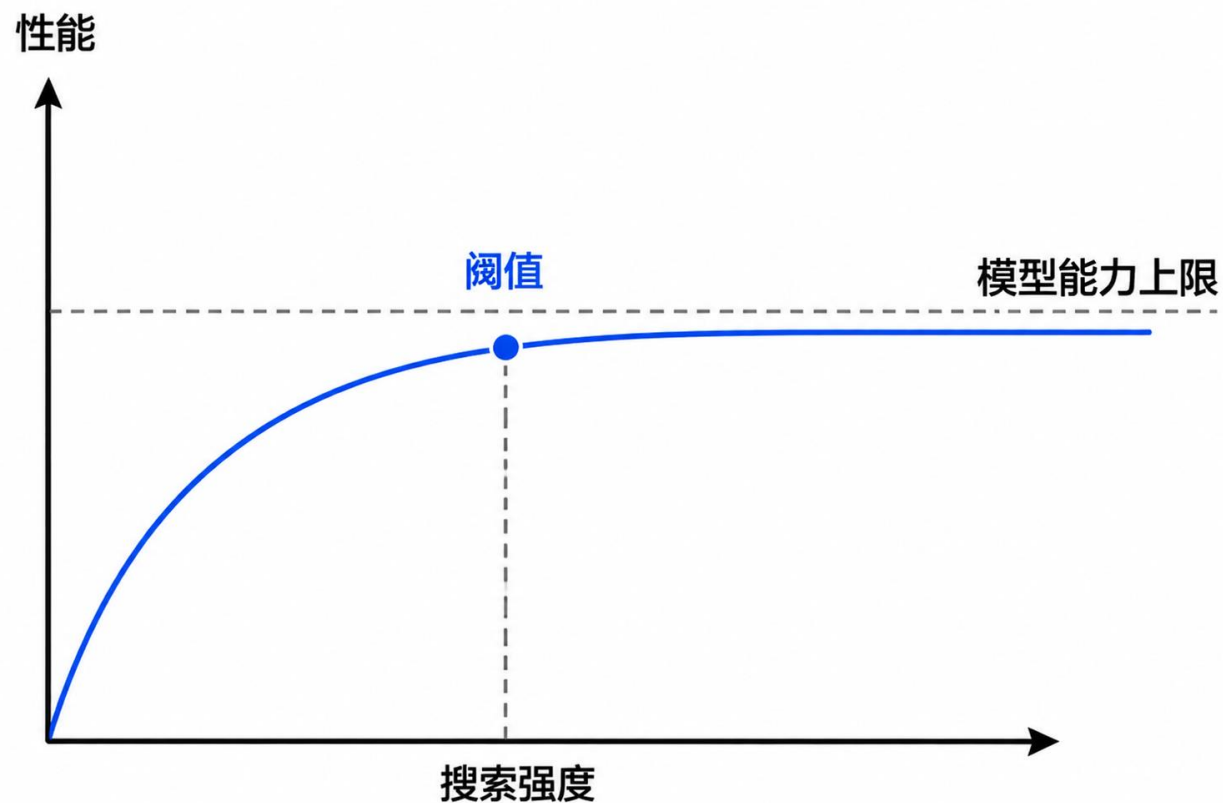
- 举例说明

笼子里有鸡和兔共 10 只，它们的脚加起来有28 只。问鸡和兔各有多少只？

候选状态	思路描述	初步评估分
s_1	假设全是鸡：10 只鸡有 20 只脚，比实际少 8 只，每把一只鸡换成兔多 2 只脚 $\rightarrow$ 兔 = $8 \div 2 = 4$	0.92
s_2	假设全是兔：10 只兔有 40 只脚，比实际多 12 只，每把一只兔换成鸡少 2 只脚 $\rightarrow$ 鸡 = $12 \div 2 = 6$	0.90
s_3	列方程：设鸡 x 只，兔 $(10-x)$ 只， $2x + 4(10-x) = 28$	0.85
s_4	列表枚举：鸡从 0 到 10 逐一试，找脚数 = 28 的组合	0.60

s_1 评分最高（思路清晰、步骤少）， s_4 评分最低（枚举繁琐、效率低），沿着 s_1 继续探索，以此类推直到探索到最终答案或者到最大探索步骤停止

- 结构化搜索的局限

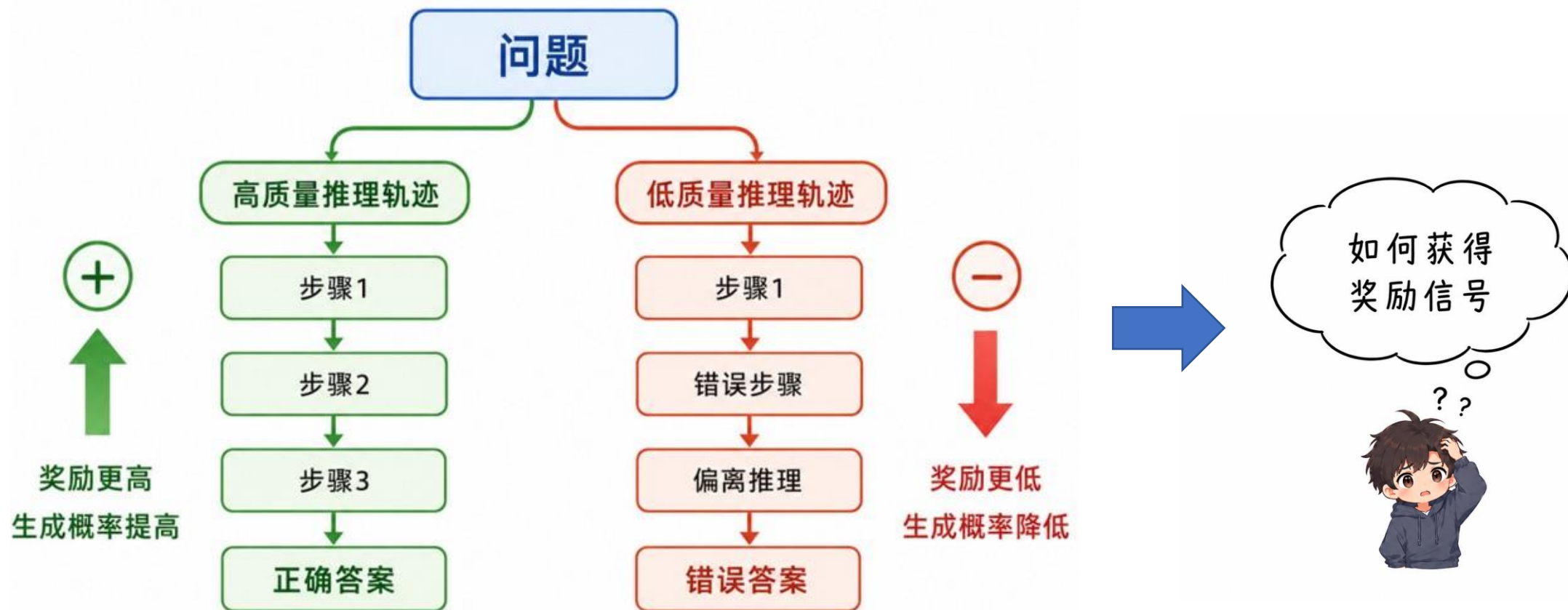


合适的搜索强度会带来性能和计算消耗的平衡

基于可校验奖励的强化学习方法

- 强化学习

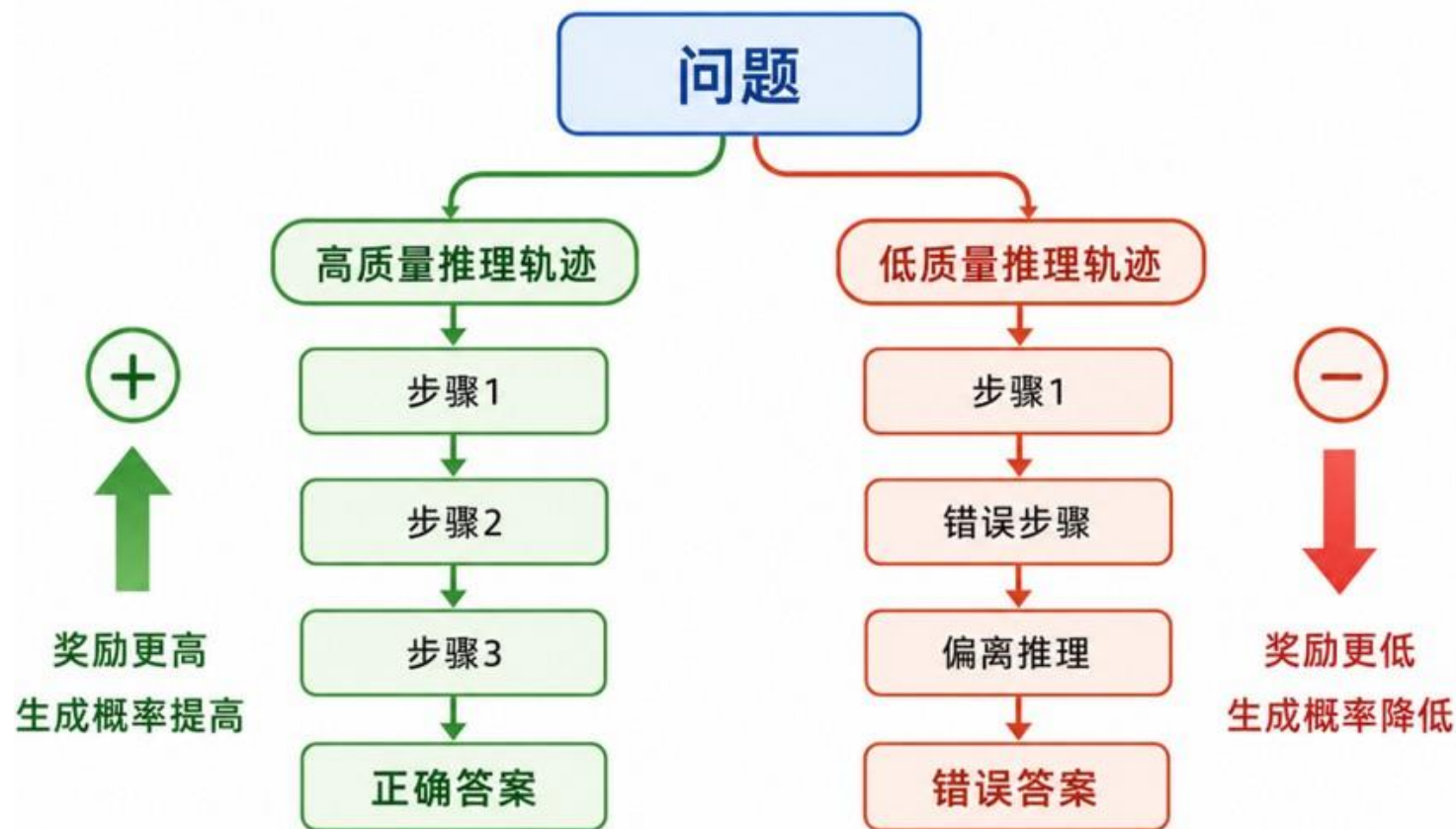
除了通过推断时增加计算量增强模型推理能力外，强化学习通过奖励高质量推理轨迹、降低低质量推理轨迹的生成概率，优化模型策略，从而提升模型的推理能力。



基于可校验奖励的强化学习方法

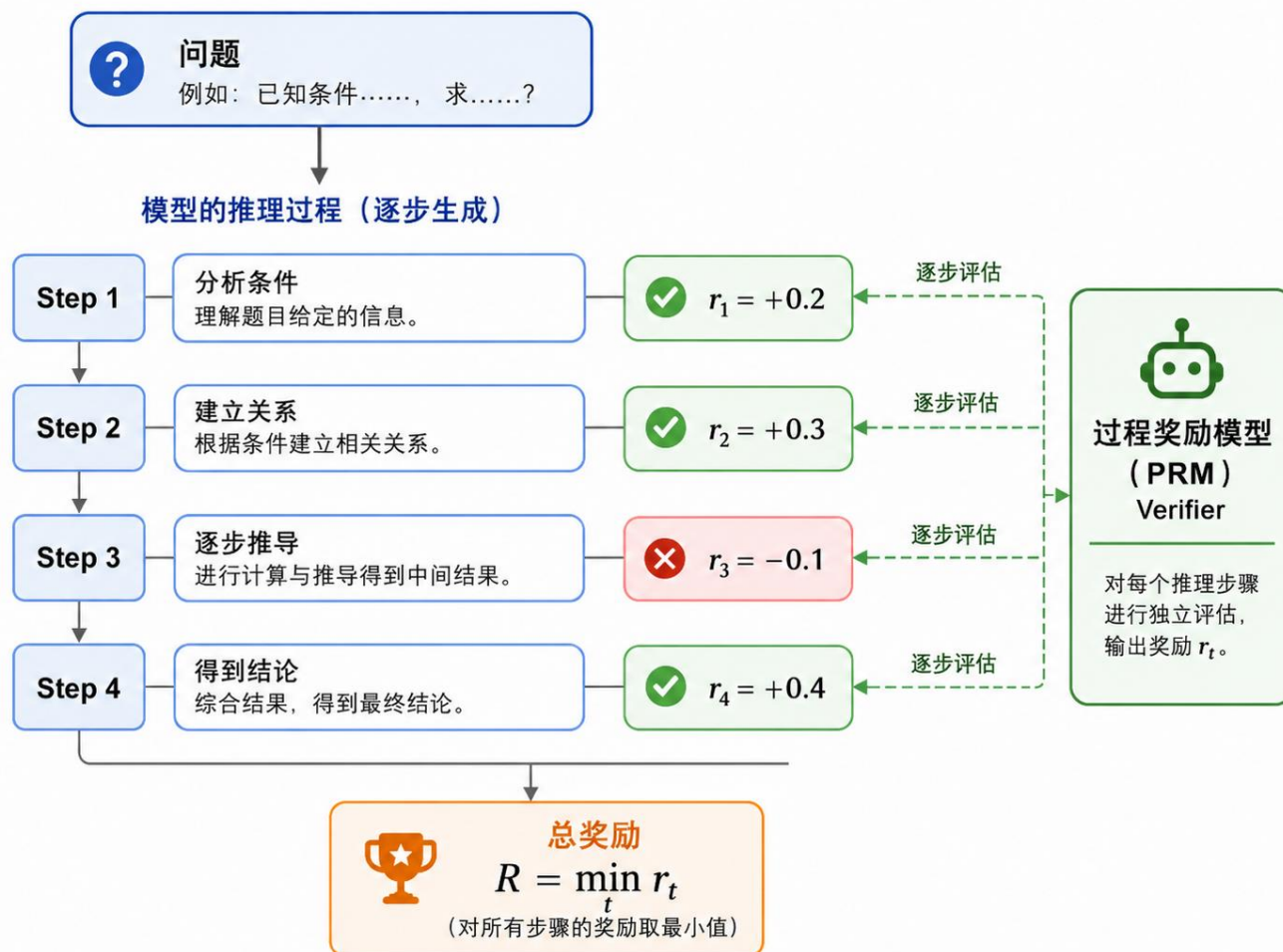
- 强化学习

除了通过推断时增加计算量增强模型推理能力外，强化学习通过奖励高质量推理轨迹、降低低质量推理轨迹的生成概率，优化模型策略，从而提升模型的推理能力。



● 过程奖励模型

行为：过程奖励模型基于之前步骤给当前步骤一个标量分数



- 数据构造方法

人工构造：费时费力，成本高

自动构造：rollout 自动验证

- 数据构造方法

人工构造：费时费力，成本高

自动构造：rollout 自动验证

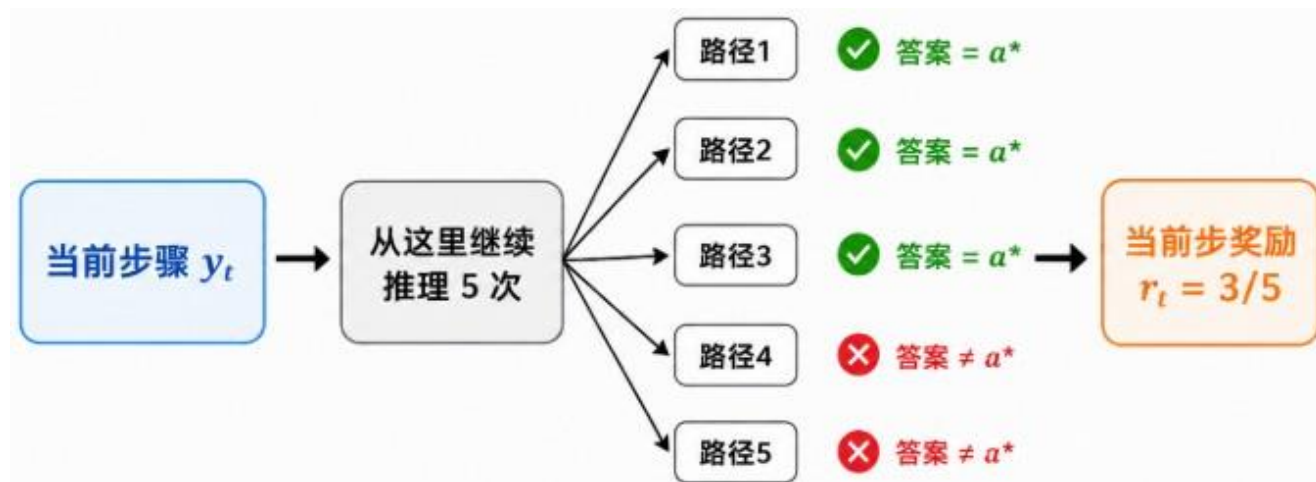
- 数据构造方法

人工构造：费时费力，成本高

自动构造：rollout 自动验证

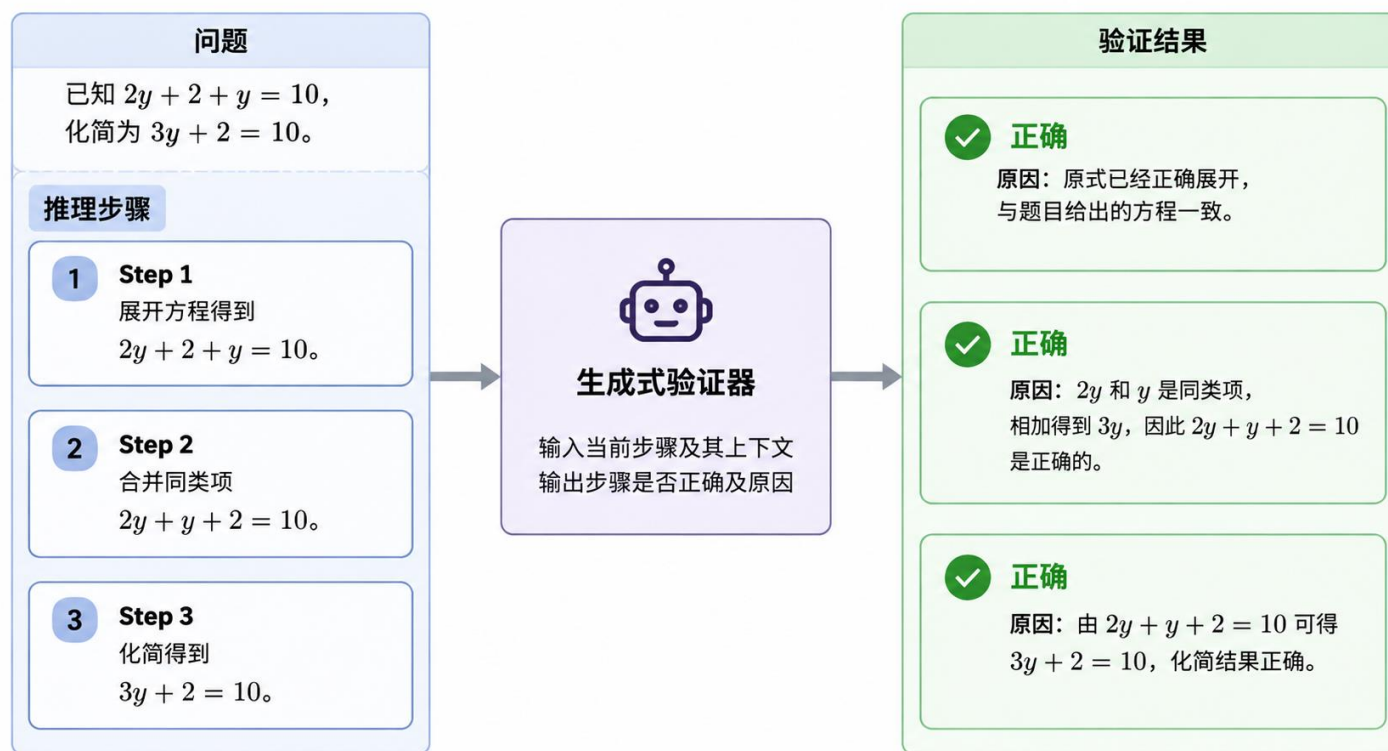


该推理步骤能走向成功的
概率作为当前步骤的奖励



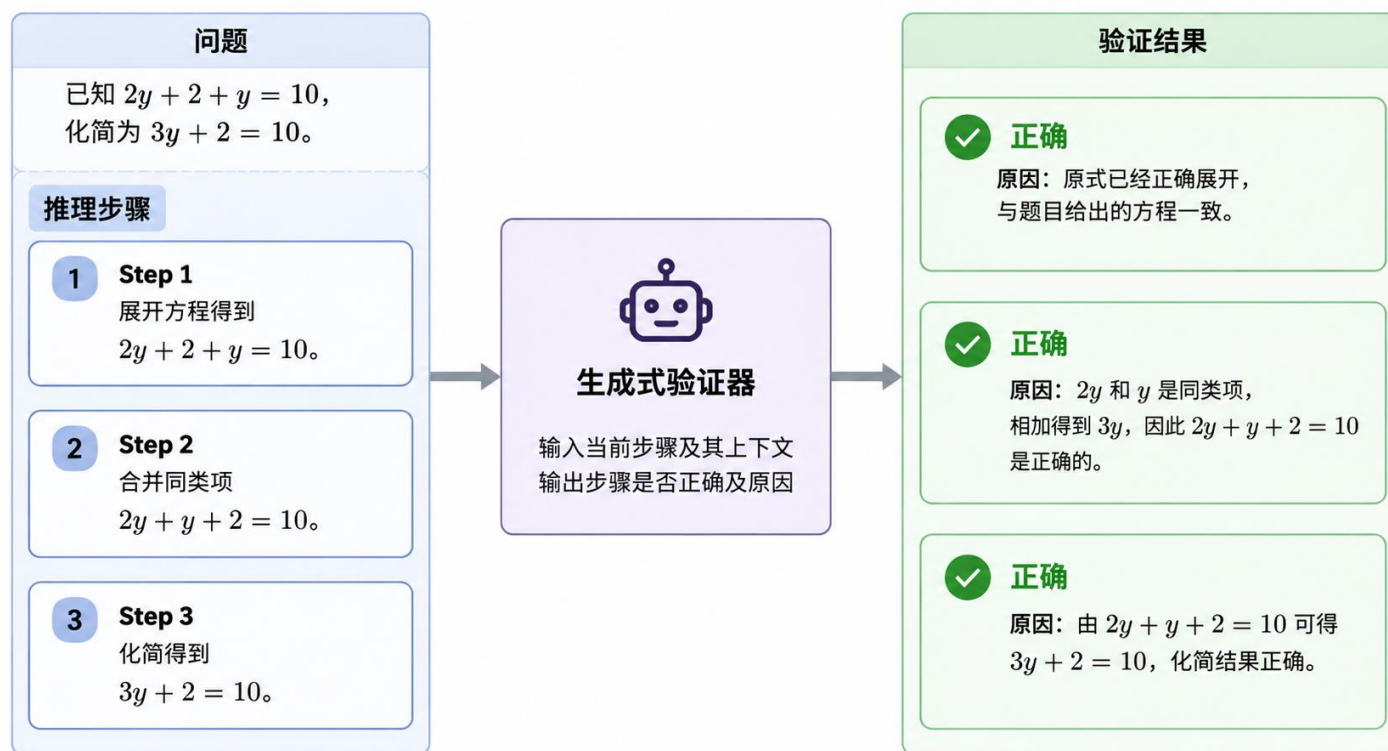
● 生成式验证器

行为：以自然语言的形式的方式描述出来



- 生成式验证器

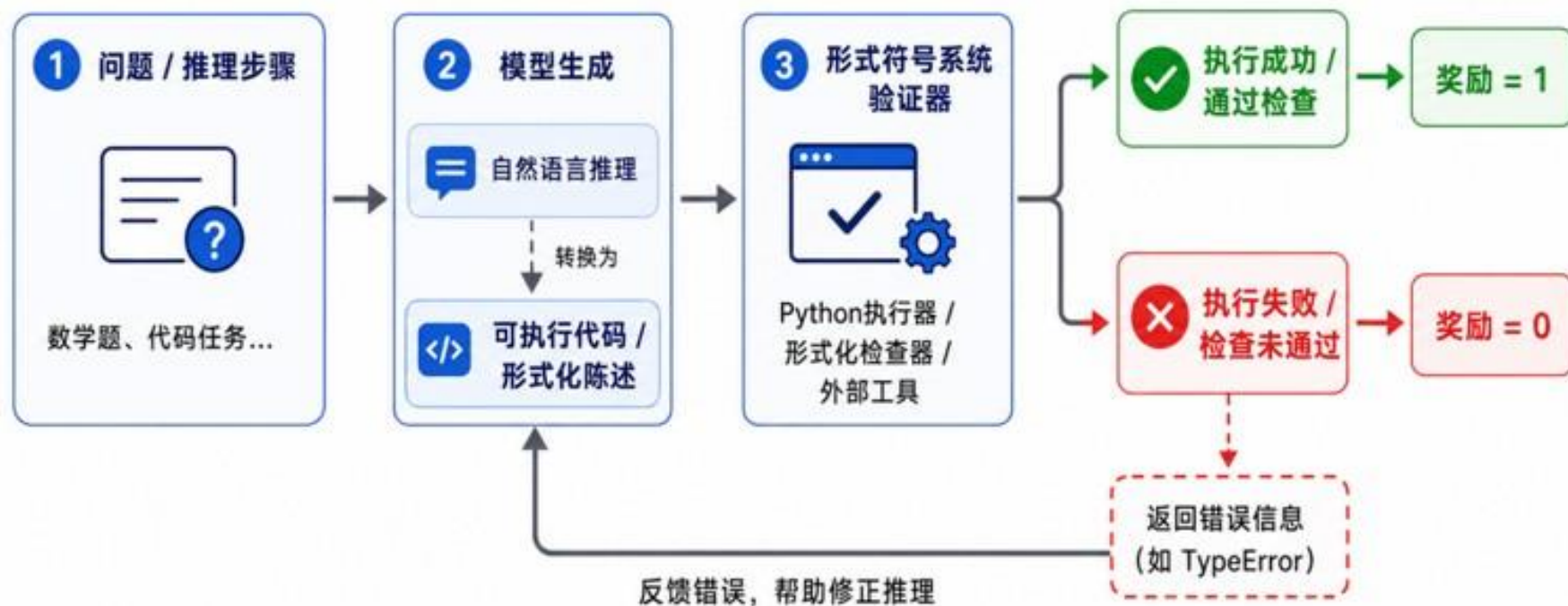
行为：以自然语言的形式的方式描述出来



可解释性强、支持自我修正、不需要改架构

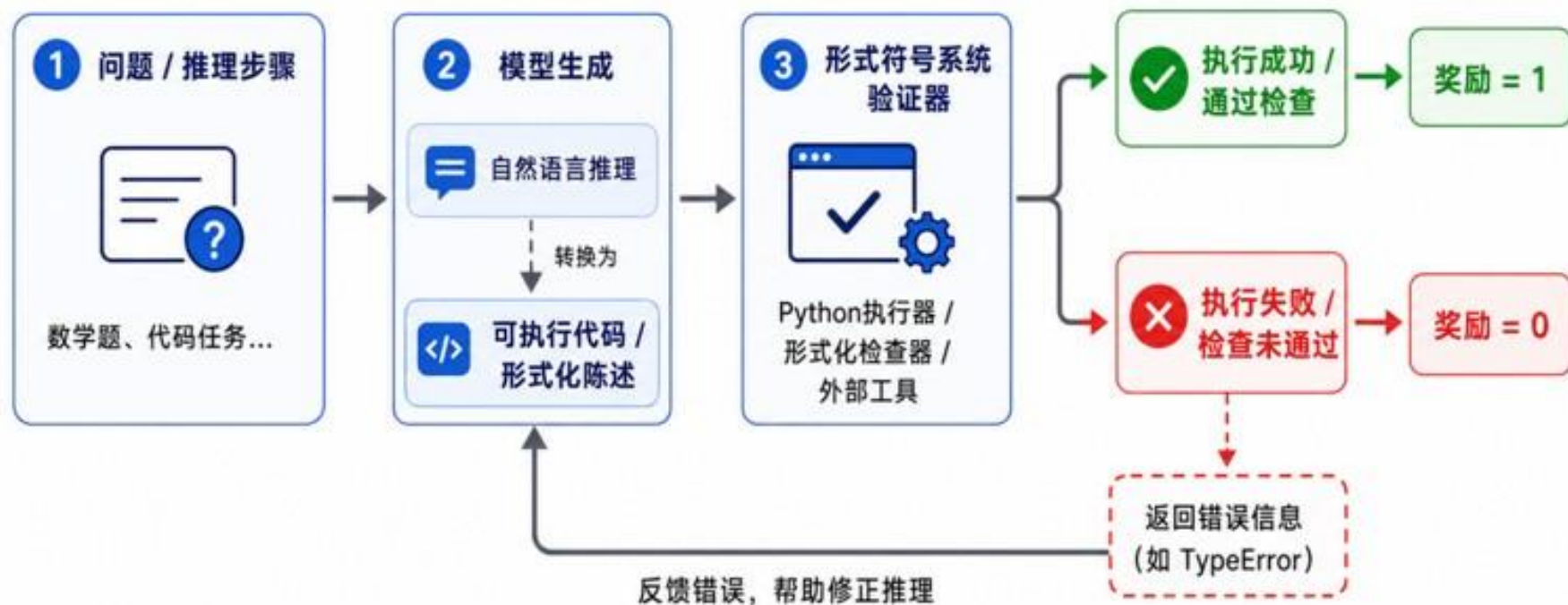
- 形式符号系统验证器

行为：将推理过程抽象为可执行代码，使用形式符号系统进行验证



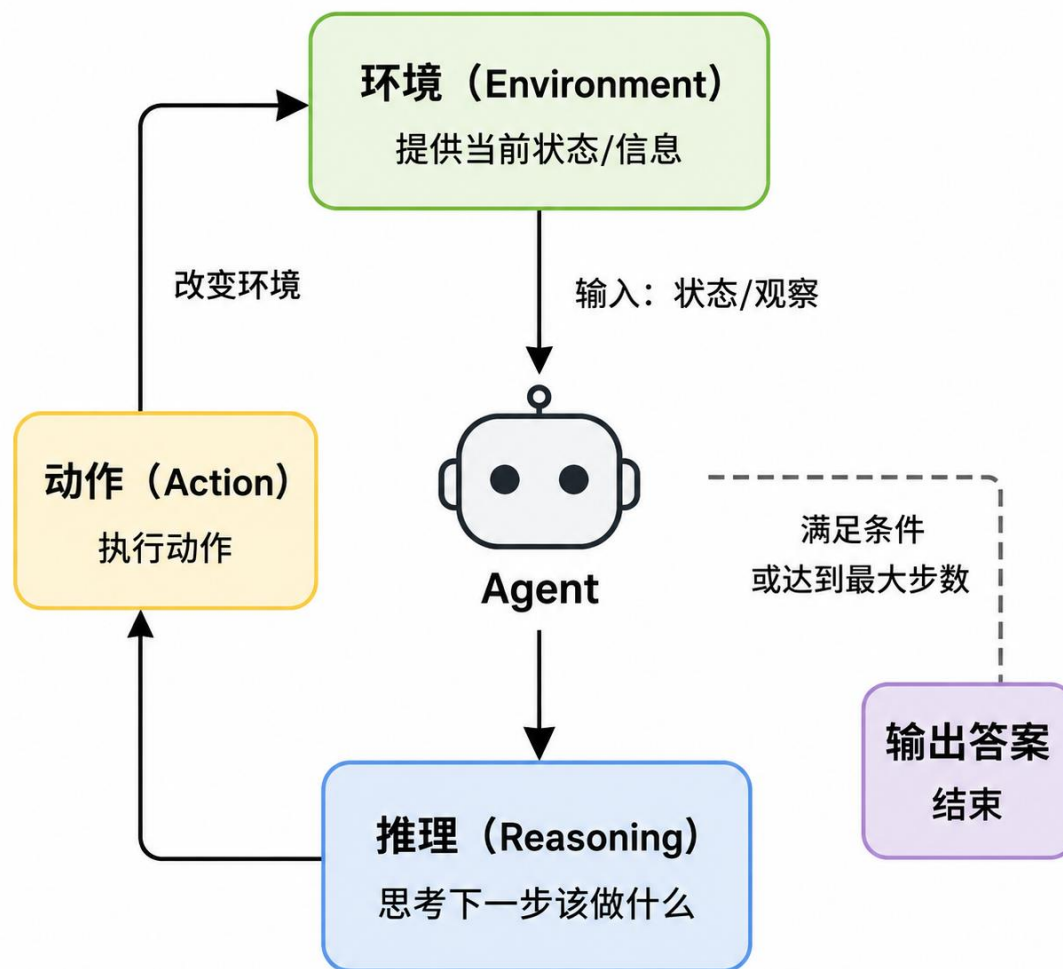
- 形式符号系统验证器

行为：将推理过程抽象为可执行代码，使用形式符号系统进行验证

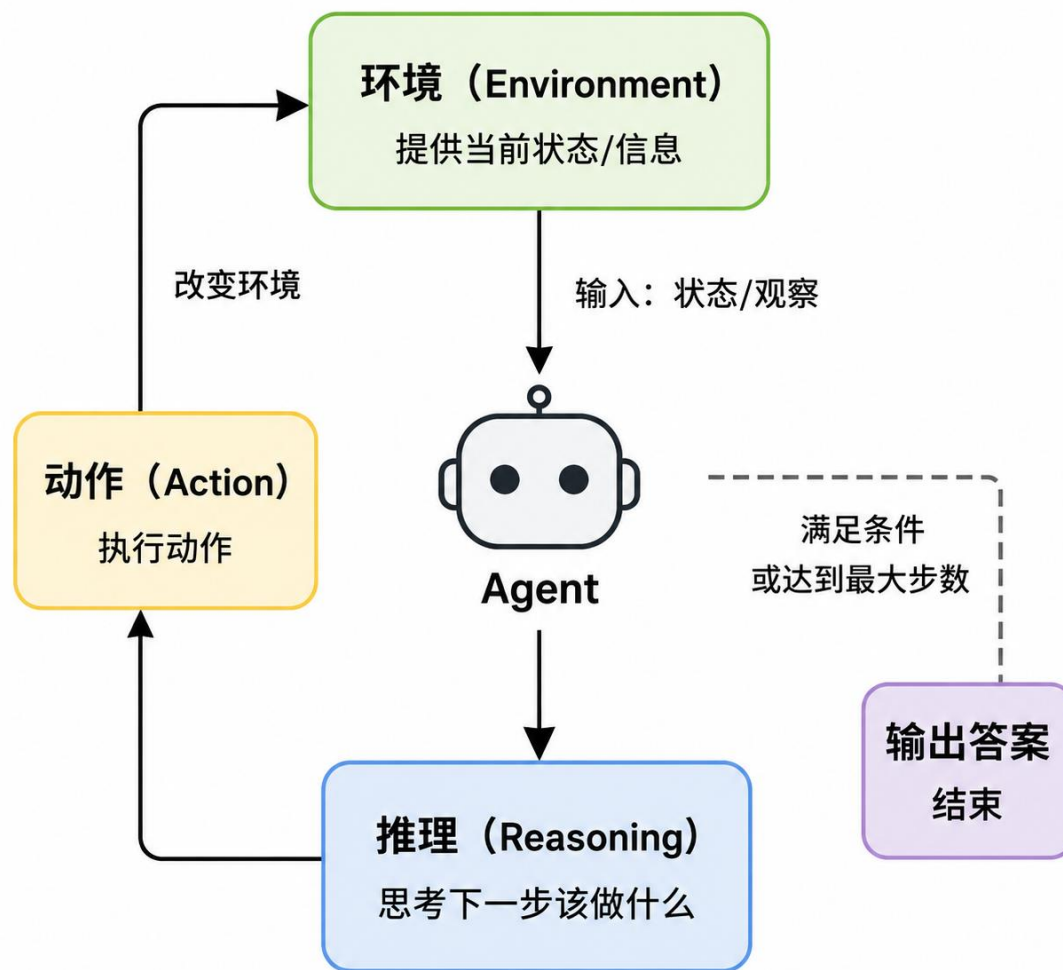


结果可靠，但要求推理步骤必须可以转化为代码或者形式规则

- ReAct



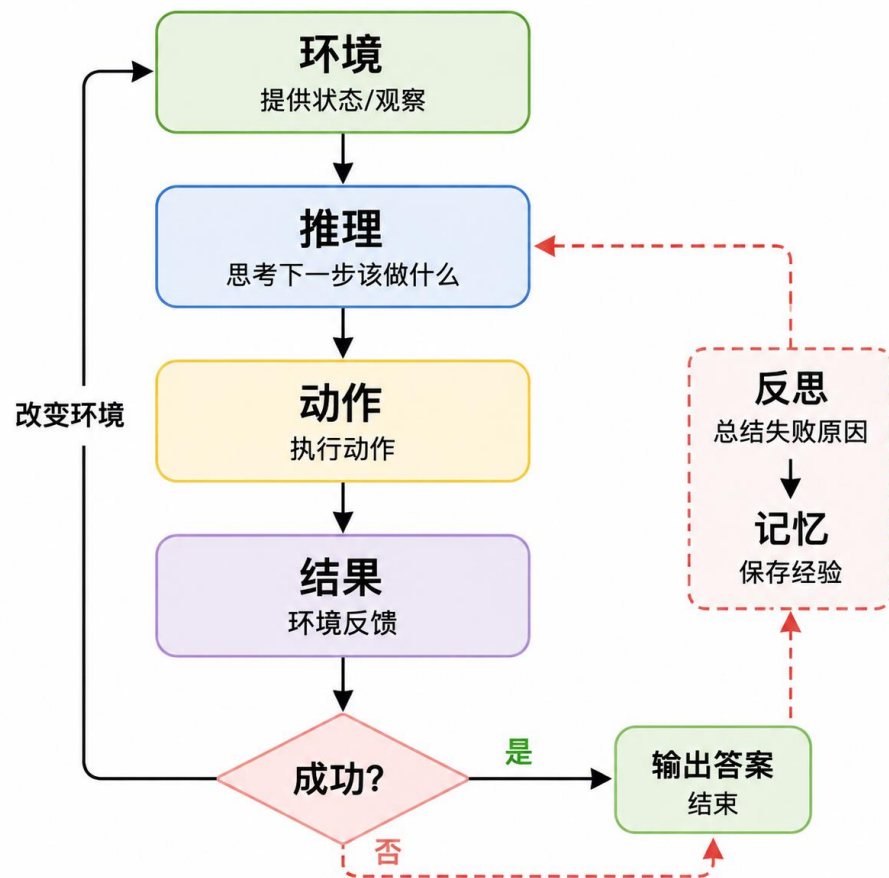
- ReAct



链式推理->目标明确, 会存在无法找到可行路径的现象

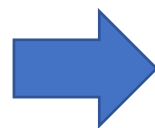
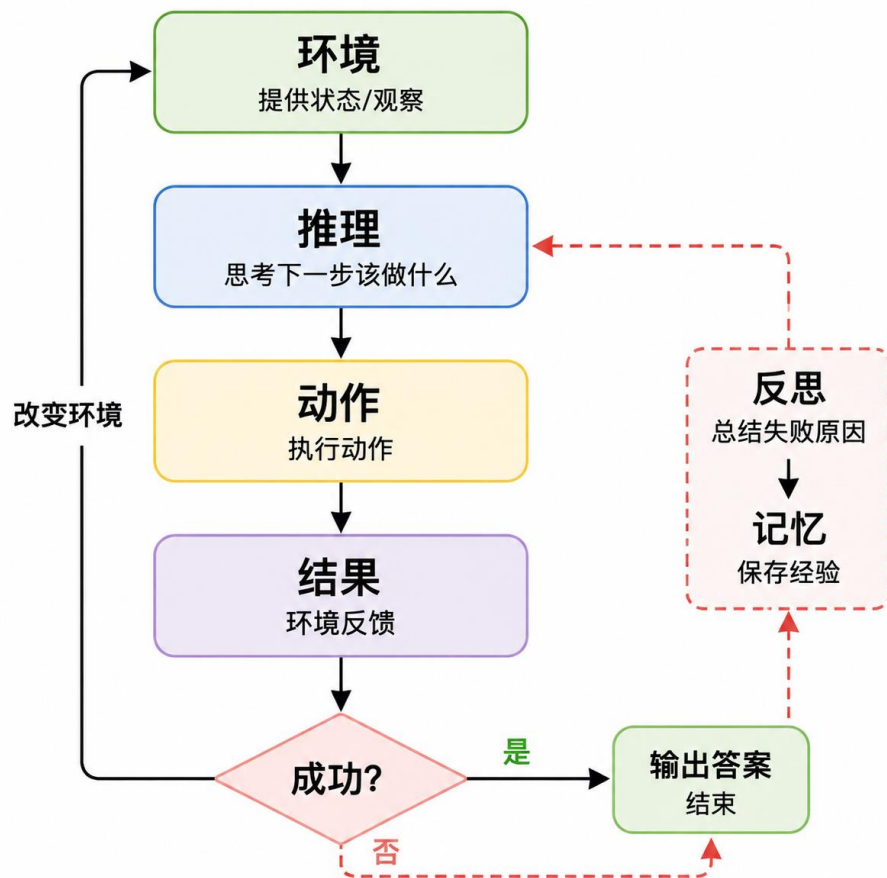
- Reflexion

目标：从错误中学习，不在一个地方跌倒两次



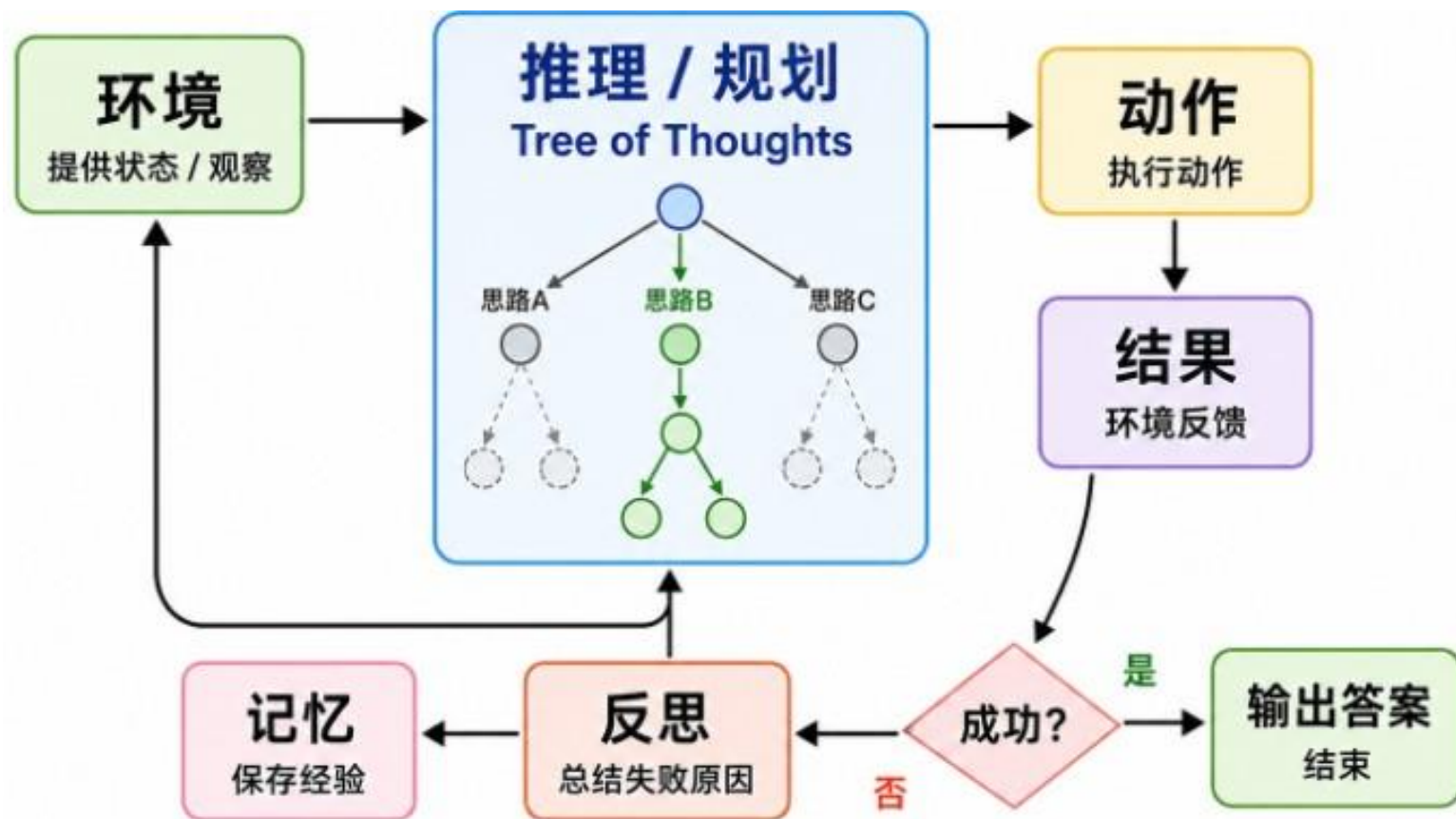
- Reflexion

目标：从错误中学习，不在一个地方跌倒两次

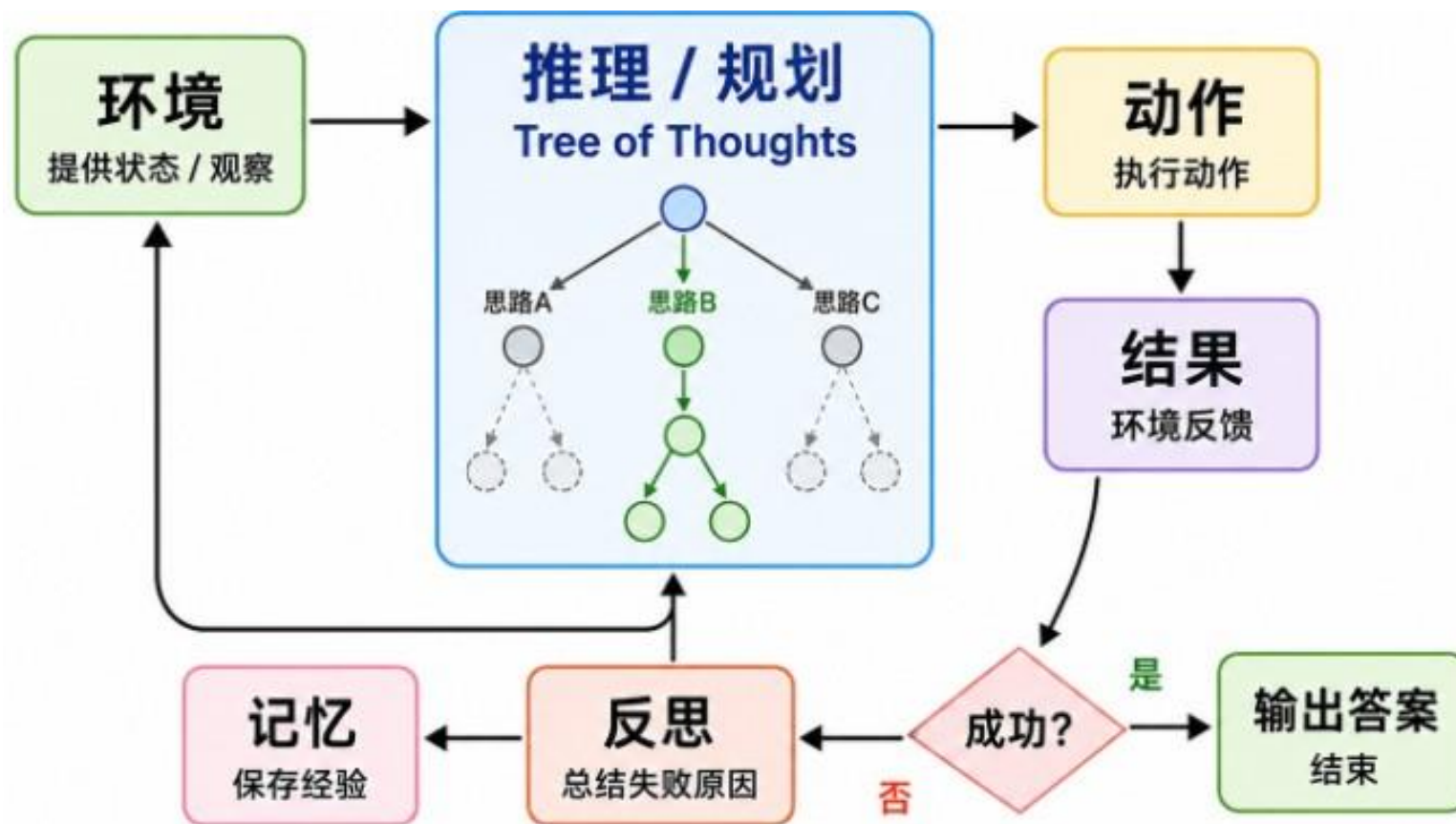


依赖模型
本身能力

- Tree of Thoughts 算法



- Tree of Thoughts 算法



全局规划和回溯任务表现很好，但是对应的开销也大

- 什么是推理？
- 可以解决什么问题？
- 测试时缩放有几种方法？
- 怎么给奖励？
- 智能体？

我学会啦，你呢？

